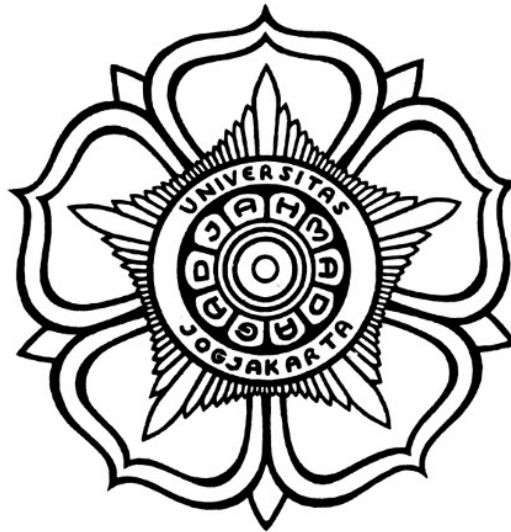


**IMPLEMENTASI SISTEM AKSELERATOR CNN BERBASIS
KOMPUTASI FIXED-POINT 16-BIT PADA BOARD FPGA**

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Affordable and Clean Energy
Decent Work and Economic Growth
Industry, Innovation and Infrastructure

Disusun oleh:

Alex Cinatra Hutasoit
22/505820/TK/55377

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2022**

HALAMAN PENGESAHAN

IMPLEMENTASI SISTEM AKSELERATOR CNN BERBASIS KOMPUTASI FIXED-POINT 16-BIT PADA BOARD FPGA

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Alex Cinatra Hutasoit
22/505820/TK/55377

Telah disetujui dan disahkan

Pada tanggal 15 Juli 2026

Dosen Pembimbing I

Dosen Pembimbing II

Dr.Eng. Ir. Igi Ardiyanto, S.T., M.Eng.
NIP 1987 0109 2020 12 1 005

Agus Bejo, S.T., M.Eng., D.Eng.
NIP 1980 0101 2015 04 1 002

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama : Alex Cinatra Hutasoit
NIM : 22/505820/TK/55377
Tahun terdaftar : 2022
Program : Sarjana
Program Studi : Teknologi Informasi
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 30 Juni 2026

Materai Rp10.000

(Tanda tangan)

Alex Cinatra Hutasoit
22/505820/TK/55377

HALAMAN PERSEMBAHAN

*Untuk Ibu, Bapak,
dan Adik-adikku tercinta.*

KATA PENGANTAR

Contoh: Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Bapak Prof. Ir. Hanung Adi Nugroho, S.T., M.Eng., Ph.D., IPM., SMIEEE., selaku Ketua Jurusan Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada,
2. Bapak Dr.Eng. Ir. Igi Ardiyanto, S.T., M.Eng., IPM., ASEAN Eng., SMIEEE., selaku dosen pembimbing pertama yang telah memberikan banyak bantuan, bimbingan, serta arahan dalam Tugas Akhir ini,
3. Bapak Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM., selaku dosen pembimbing kedua yang juga telah memberikan banyak bantuan, bimbingan, serta arahan dalam Tugas Akhir,
4. Bapak Widyawan, S.T., M.Sc., Ph.D., selaku dosen pembimbing akademik yang telah memberikan pendampingan dan arahan selama masa perkuliahan,
5. Seluruh Dosen di Jurusan Teknik Elektro dan Teknologi Informasi FT UGM, yang tidak bisa disebutkan satu-satu, atas ilmu dan bimbingannya selama penulis berkuliah di JTETI,
6. Ibu dan Bapak yang selama ini telah sabar membimbing, mengarahkan, dan mendukung penulis tanpa kenal lelah untuk selama-lamanya,
7. T Hutasoit dan L Sianturi yang sudah percaya dan mendukung penulis untuk selalu berusaha dan rendah hati serta sabar dalam menjalani semuanya

Akhir kata penulis berharap semoga skripsi ini dapat memberikan manfaat dalam bidang teknologi informasi untuk penelitian selanjutnya di masa mendatang.

Yogyakarta, 25 Juni 2026

Penulis

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	v
DAFTAR ISI	vi
DAFTAR TABEL	ix
DAFTAR GAMBAR	x
DAFTAR SINGKATAN.....	xi
INTISARI.....	xii
ABSTRACT	xiii
BAB I LATAR BELAKANG	1
1.1 Latar Belakang Masalah	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	2
1.4 Batasan Masalah	3
1.5 Manfaat Penelitian	3
1.6 Sistematika Penulisan.....	3
BAB II TINJAUAN PUSTAKA DAN DASAR TEORI	5
2.1 Tinjauan Pustaka	5
2.2 Landasan Teori	7
2.2.1 <i>Neural Network</i>	7
2.2.2 CNN	8
2.2.3 Konvolusi	9
2.2.4 <i>Rectified Linear Unit</i> (ReLU)	10
2.2.5 <i>Max Pool</i>	11
2.2.6 Kuantitasi	12
2.2.7 Multiply Accumulate	14
2.2.8 Time Multiplexer.....	15
2.2.9 Shift Register	15
2.2.10 First In First Out (FIFO)	16
2.2.11 Finite State Machine	17
2.2.12 Backpressure	19
2.2.13 Double Data Rate (DDR)	19
2.2.14 FPGA ARTIX 7	21
2.2.15 OV7670.....	22
2.2.16 VGA	22

2.3	Analisis Perbandingan Metode	23
BAB III Metode Penelitian.....		25
3.1	Alat dan Bahan	25
3.1.1	Perangkat Keras	25
3.1.2	Perangkat Lunak	25
3.2	Rancangan Sistem	25
3.2.1	Rancangan Umum Sistem	25
3.2.2	Dataset	26
3.2.3	Rancangan Arsitektur CNN	26
3.2.4	Rancangan Representasi Fixed-Point	27
3.2.5	Rancangan Akselerator CNN pada FPGA	29
3.2.6	Rancangan Double Frame Memory	31
3.3	Metode Implementasi	32
3.3.1	Implementasi Model CNN di Python	32
3.3.2	Implementasi Kuantisasi Fixed-Point	32
3.3.3	Implementasi CNN ke Verilog	33
3.3.4	Implementasi DDR sebagai Double Frame Memory	34
3.4	Metode Integrasi	34
3.4.1	Integrasi Model CNN ke FPGA	34
3.4.2	Integrasi DDR sebagai Double Frame Memory	34
3.4.3	Integrasi Double Frame Memory dan CNN	34
3.5	Metode Pengujian	35
3.5.1	Pengujian Model CNN di Python	35
3.5.2	Pengujian Perbedaan Hasil Python dan Verilog	35
3.5.3	Pengujian Penggunaan Kapasitas FPGA.....	36
3.5.4	Pengujian Double Frame Memory	36
3.5.5	Pengujian Sistem Terintegrasi	36
3.6	Metode Evaluasi	37
3.6.1	Evaluasi Akurasi	37
3.6.2	Evaluasi Perbedaan Bit	37
3.6.3	Evaluasi Latensi Inferensi	39
3.6.4	Evaluasi Penggunaan Resource FPGA.....	40
3.6.5	Evaluasi Daya	40
3.6.6	Evaluasi Tampilan Real Time	40
3.7	Tahapan Pelaksanaan	41
BAB IV HASIL DAN PEMBAHASAN		43
4.1	Hasil Implementasi Model CNN	43
4.1.1	Dataset yang Digunakan	43
4.1.2	Arsitektur Target Model CNN	44

4.1.3	Hasil Pelatihan Model CNN	45
4.1.4	Hasil Evaluasi Model CNN di Python	47
4.1.5	Kuantisasi Parameter Model CNN	49
4.2	Hasil Implementasi CNN Accelerator pada FPGA.....	50
4.2.1	Arsitektur CNN Accelerator pada FPGA	51
4.2.2	Hasil Penggunaan Resource CNN Accelerator	52
4.2.3	Hasil Timing CNN Accelerator	52
4.2.4	Hasil Latensi dan Throughput CNN Accelerator	53
4.3	Hasil Pengujian Modul Perangkat Keras	54
4.3.1	Hasil Uji Double Frame Memory	54
4.4	Hasil Integrasi Sistem.....	54
4.4.1	Hasil Integrasi Double Frame Memory	54
4.4.2	Hasil Integrasi CNN Accelerator dengan Double Frame Memory .	55
4.4.3	Hasil Penggunaan Resource Sistem Terintegrasi	56
4.4.4	Hasil Timing Sistem Terintegrasi	56
4.4.5	Hasil Estimasi Daya Sistem Terintegrasi	56
4.5	Hasil Evaluasi dan Pembahasan	58
4.5.1	Perbandingan Hasil Python dan Verilog	58
4.5.2	Evaluasi Hasil Inferensi Model pada FPGA	60
4.5.3	Pembahasan Resource, Latensi, dan Daya	61
4.5.4	Pembahasan Ketercapaian Sistem terhadap Tujuan Penelitian	61
BAB V	KESIMPULAN DAN SARAN	62
5.1	Kesimpulan.....	62
5.2	Saran.....	62
	DAFTAR PUSTAKA.....	64
	LAMPIRAN	L-1
L.1	Block Design Sistem OV7670, DDR3, dan VGA.....	L-1
L.2	Kode Implementasi Kuantitasi Parameter 16 Bit	L-1
L.3	Kode Evaluasi Hasil Perbedaan Bit	L-2
L.4	Kode Verilog <i>AXI Read</i>	L-5
L.5	Kode Verilog <i>AXI Write</i>	L-16
L.6	Kode Verilog <i>Base Adress Controller</i>	L-26
L.7	Kode Verilog <i>Camera Capture</i>	L-33
L.8	Kode Verilog <i>VGA Stream</i>	L-36

DAFTAR TABEL

Tabel 2.1	Spesifikasi utama Arty A7-100T	21
Tabel 2.2	Perbandingan implementasi sistem pada penelitian terkait.....	23
Tabel 2.3	Perbandingan performa penelitian CNN Accelerator.....	24
Tabel 3.1	Target penggunaan resource pada implementasi CNN	29
Tabel 4.1	Ukuran Parameter Pada Model Python	48
Tabel 4.2	Parameter Model pada Python	50
Tabel 4.3	Hasil Kuantisasi Parameter	51
Tabel 4.4	Hasil <i>CNN Implement Running</i>	52
Tabel 4.5	Hasil Routing CNN Accelerator Design	52
Tabel 4.6	Hasil <i>CNN Accelerator Delayed Design</i>	52
Tabel 4.7	Hasil Latensi CNN Accelerator.....	53
Tabel 4.8	Hasil <i>Double Frame Memory Implement Running</i>	54
Tabel 4.9	Detail Hasil <i>Implement Running</i>	56
Tabel 4.10	Hasil Routing Design	56
Tabel 4.11	Hasil <i>Delayed Design</i>	56
Tabel 4.12	Penggunaan Daya dan Temperature	57

DAFTAR GAMBAR

Gambar 2.1	Operasi Konvolusi	10
Gambar 2.2	Operasi Max Pool	12
Gambar 2.3	Ilustrasi data <i>input shift register</i>	16
Gambar 2.4	Ilustrasi FIFO	17
Gambar 2.5	Aliran sinyal <i>valid</i> dan <i>ready</i> pada sistem pipeline	19
Gambar 2.6	Ilustrasi hubungan AXI, MIG, dan DDR pada sistem pemindah- an data berbasis memori eksternal.....	20
Gambar 3.7	Diagram Alir Penelitian	42
Gambar 4.8	Distribusi <i>Training Dataset</i> yang digunakan	43
Gambar 4.9	Arsitektur Model CNN	44
Gambar 4.10	<i>Loss training</i> model.....	45
Gambar 4.11	<i>Accuracy training</i> model	46
Gambar 4.12	<i>Confusion Matrix Training</i> model.....	47
Gambar 4.13	Distribusi <i>Test Dataset</i> yang digunakan	48
Gambar 4.14	<i>Confusion Matrix testing</i> model.....	49
Gambar 4.15	Desain CNN Accelerator pada FPGA	51
Gambar 4.16	Hasil <i>debug</i> ILA pada <i>double frame memory</i>	54
Gambar 4.17	Desain Akhir Integrasi	55
Gambar 4.18	Detail Hasil Penggunaan Daya	57
Gambar 4.19	Perbandingan <i>Bit</i> dan <i>Float value</i> pada <i>Conv 1</i> dan <i>Pool 1</i>	58
Gambar 4.20	Perbandingan <i>Bit</i> dan <i>Float value</i> pada <i>Conv 2</i> dan <i>Pool 2</i>	58
Gambar 4.21	Perbandingan <i>Bit</i> dan <i>Float value</i> pada <i>Conv 3</i> dan <i>Pool 3</i>	59
Gambar 4.22	Perbandingan <i>Bit</i> dan <i>Float value</i> pada <i>Dense 1</i> dan <i>Dense 2</i>	59
Gambar 4.23	Perbandingan Akurasi Verilog Simulation dan Actual Accuracy Dataset.....	60
Gambar L.1	Block Design Alur Inferensi Camera Sampai VGA.....	L-1

DAFTAR SINGKATAN

<i>AXI</i>	=	Advanced eXtensible Interface
<i>BD</i>	=	Block Design
<i>BRAM</i>	=	Block Random Access Memory
<i>BUFG</i>	=	Global Clock Buffer
<i>CLK</i>	=	Clock
<i>CNN</i>	=	Convolutional Neural Network
<i>CONV</i>	=	Convolution
<i>CPU</i>	=	Central Processing Unit
<i>DRC</i>	=	Design Rule Check
<i>DSP</i>	=	Digital Signal Processing
<i>DDR</i>	=	Double Data Rate
<i>FF</i>	=	Flip Flop
<i>FIFO</i>	=	First In First Out
<i>FWFT</i>	=	First Word Fall Through
<i>FPGA</i>	=	Field Programmable Gate Array
<i>FPS</i>	=	Frame Per Second
<i>GPIO</i>	=	General-Purpose Input/Output
<i>GPU</i>	=	Graphics Processing Unit
<i>HDL</i>	=	Hardware Descriptive Language
<i>I/O</i>	=	Input Output
<i>I2C</i>	=	Inter-Integrated Circuit
<i>IOB</i>	=	Input/Output Block
<i>LUT</i>	=	Look-Up Table
<i>MIG</i>	=	Memory Interface Generator
<i>MNIST</i>	=	Modified National Institute of Standards and Technology
<i>MUX</i>	=	Multiplexer
<i>SOC</i>	=	System On Chip
<i>SPI</i>	=	Serial Peripheral Interface
<i>TNS</i>	=	Total Negative Slack
<i>THS</i>	=	Total Hold Slack
<i>UART</i>	=	Universal Asynchronous Receiver-Transmitter
<i>UI</i>	=	User Interface
<i>URAM</i>	=	Ultra Random Access Memory
<i>WNS</i>	=	Worst Negative Slack
<i>WHS</i>	=	Worst Hold Slack
<i>WPWS</i>	=	Worst Pulse Width Slack

INTISARI

Implementasi *Convolutional Neural Network* (CNN) pada sistem tertanam membutuhkan rancangan yang akurat, rendah latensi, hemat daya, dan dapat terintegrasi dengan jalur akuisisi citra. Penelitian ini merancang dan mengintegrasikan akselerator CNN berbasis komputasi *fixed-point* 16-bit pada board FPGA Digilent Arty A7-100T, dengan kamera OV7670 sebagai sumber citra, DDR3 SDRAM sebagai *frame buffer*, modul *region of interest*, CNN accelerator berbasis Verilog, modul *argmax*, dan keluaran VGA. Model CNN dilatih menggunakan Python pada dataset MNIST berukuran 28×28 piksel, kemudian bobot dan bias dikuantisasi ke format Q2.14 yang tidak menunjukkan saturasi. Hasil implementasi menunjukkan CNN accelerator menghasilkan keluaran inferensi dalam 3802 siklus clock atau $38,02 \mu s$ pada 100 MHz, dengan throughput sekitar 26302 inferensi per detik. Estimasi daya total on-chip sistem terintegrasi adalah 1,099 W, dan penggunaan DSP mencapai 235 dari 240 DSP yang tersedia atau sekitar 97% pada board Arty A7-100T, sehingga integrasi kamera, memori, akselerator CNN, dan tampilan dapat direalisasikan dengan batas utama pada margin DSP yang sangat kecil.

Kata kunci : *CNN, FPGA, fixed-point* 16-bit, Arty A7-100T, OV7670, akselerator CNN.

ABSTRACT

The implementation of a Convolutional Neural Network (CNN) in an embedded system requires a design that is accurate, low latency, low power, and integrable with an image acquisition pipeline. This research designs and integrates a 16-bit fixed-point CNN accelerator on the Digilent Arty A7-100T FPGA board, with an OV7670 camera as the image source, DDR3 SDRAM as the frame buffer, a region-of-interest module, a Verilog-based CNN accelerator, an argmax module, and VGA output. The CNN model was trained in Python using the 28×28 MNIST dataset, then its weights and biases were quantized into Q2.14 without saturation. The implementation shows that the CNN accelerator produces inference output in 3802 clock cycles, or $38.02 \mu\text{s}$ at 100 MHz, with a throughput of approximately 26302 inferences per second. The estimated total on-chip power of the integrated system is 1.099 W, and DSP usage reaches 235 out of 240 available DSP blocks or about 97% on the Arty A7-100T board, indicating that the camera, memory, CNN accelerator, and display can be integrated on FPGA with the main limitation being the very small remaining DSP margin.

Keywords : CNN, FPGA, 16-bit fixed-point, Arty A7-100T, OV7670, CNN accelerator.

BAB I

LATAR BELAKANG

1.1 Latar Belakang Masalah

Kemampuan CNN dalam melakukan ekstraksi fitur hirarkis pada spatial data matriks menjadikannya sebagai salah satu metode yang paling banyak digunakan dalam proses pengolahan citra dan visi. Proses ekstraksi fitur pada data gambar berskala besar dilakukan oleh secara otomatis dan menjadikan CNN sebagai fondasi utama dalam implementasi pengolahan citra dan visi untuk pengenalan pola, objek hingga karakter [1,2]. Keunggulan ini mendorong adopsi dan implementasi di berbagai bidang terutama sistem tertanam. Saat ini implementasi CNN banyak dilakukan di GPU dan CPU. Konsumsi daya pada GPU sangat besar dan boros baik pada saat proses pelatihan model maupun pada saat mode inferensial [3,4]. Hal ini terjadi terutama pada GPU, karena overhead komunikasi data bus pada batch berukuran kecil sehingga penggunaan daya sangat tidak efisien.

Keterbatasan implementasi CNN pada CPU dan GPU menjadi semakin penting ketika CNN diterapkan pada sistem tertanam yang membutuhkan pemrosesan citra secara langsung, cepat, dan hemat daya [5]. Pada aplikasi seperti kamera cerdas, robotika, dan kendaraan otonom, data visual diperoleh secara terus-menerus dari sensor sehingga proses komputasi tidak cukup hanya akurat, tetapi juga harus memiliki latensi rendah. Oleh karena itu, pendekatan *edge computation* dan *embedded vision* menjadi salah satu solusi karena proses inferensi dapat dilakukan lebih dekat dengan sumber data.

Salah satu alternatif perangkat keras yang dapat digunakan untuk mempercepat komputasi CNN adalah FPGA. FPGA memiliki keunggulan karena arsitektur perangkat kerasnya dapat dikonfigurasi sesuai kebutuhan algoritma. Berbeda dengan CPU dan GPU yang menggunakan arsitektur komputasi umum, FPGA memungkinkan perancang untuk membangun jalur data khusus, menerapkan paralelisme pada tingkat operasi, serta menggunakan teknik *pipeline* untuk meningkatkan *throughput*. Karakteristik ini sesuai dengan operasi CNN yang bersifat berulang, terstruktur, dan memiliki pola komputasi yang dapat dipetakan ke dalam perangkat keras pola [6,7].

Penelitian ini berfokus pada perancangan dan implementasi akselerator CNN berbasis FPGA dengan komputasi *fixed-point* yang berjalan dengan daya rendah. Akselerator dirancang untuk menjalankan proses inferensi CNN secara perangkat keras dengan memanfaatkan paralelisme, *pipeline*, dan optimasi representasi data. Selain itu, penelitian ini juga membahas integrasi sistem dengan perangkat kamera dan jalur tampilan citra, sehingga sistem tidak hanya diuji sebagai modul komputasi terpisah, tetapi juga sebagai bagian dari sistem *embedded vision* yang menerima data visual dari lingkungan nyata.

1.2 Rumusan Masalah

Implementasi CNN pada sistem tertanam tidak hanya menuntut akurasi klasifikasi, tetapi juga efisiensi komputasi, latensi rendah, konsumsi daya yang kecil, serta kemampuan integrasi dengan perangkat akuisisi dan tampilan citra. CPU dan GPU memiliki fleksibilitas tinggi, tetapi konsumsi daya dan pola eksekusinya tidak selalu sesuai untuk sistem *embedded vision* yang bekerja secara langsung di sisi perangkat. FPGA menjadi alternatif karena jalur data, paralelisme, dan representasi numeriknya dapat dirancang sesuai kebutuhan inferensi CNN. Namun, implementasi pada FPGA harus mempertimbangkan keterbatasan DSP, LUT, FF, BRAM, akses memori, serta sinkronisasi antarperiferal.

Berdasarkan latar belakang tersebut, rumusan masalah dalam penelitian ini adalah sebagai berikut.

1. Bagaimana merancang akselerator CNN berbasis komputasi *fixed-point* 16-bit pada FPGA untuk klasifikasi citra MNIST?
2. Bagaimana mengintegrasikan kamera OV7670, DDR SDRAM sebagai *frame buffer*, VGA Pmod Digilent, ROI, CNN accelerator, dan argmax menjadi sistem *embedded vision* end-to-end pada board Arty A7-100T?
3. Bagaimana performa sistem yang dihasilkan dari sisi akurasi, latensi inferensi, penggunaan *resource* FPGA, estimasi konsumsi daya, timing, dan kemampuan kerja *real-time*?

1.3 Tujuan Penelitian

Tujuan penelitian ini adalah merancang, mengimplementasikan, dan mengevaluasi sistem akselerator CNN berbasis FPGA yang terintegrasi dengan subsistem kamera, memori, dan tampilan. Secara khusus, tujuan penelitian ini adalah sebagai berikut.

1. Merancang akselerator CNN dengan representasi *fixed-point* 16-bit untuk inferensi citra MNIST pada FPGA.
2. Mengintegrasikan subsistem OV7670, DDR SDRAM, VGA Pmod Digilent, ROI, CNN accelerator, dan argmax dalam satu alur sistem *embedded vision*.
3. Mengevaluasi akurasi model sebagai acuan kebenaran fungsi inferensi.
4. Mengevaluasi latensi dan *throughput* inferensi CNN accelerator.
5. Mengevaluasi penggunaan *resource* FPGA, meliputi LUT, FF, BRAM, dan DSP.
6. Mengevaluasi hasil timing dan estimasi konsumsi daya sistem terintegrasi.
7. Mengevaluasi ketercapaian sistem terhadap kebutuhan pemrosesan citra secara *real-time* sesuai hasil implementasi yang diperoleh.

1.4 Batasan Masalah

Batasan masalah pada penelitian ini adalah:

1. Pada penelitian ini *board* FPGA yang digunakan adalah ARTY 7 100T.
2. Kamera yang digunakan adalah OV7670.
3. Streaming video dilakukan lewat VGA, dengan modul VGA PMOD Digilent.
4. Keterbatasan *resource* pada FPGA memungkinkan penelitian menggunakan model yang tidak terlalu besar.
5. *Dataset* yang digunakan adalah *Dataset* MNIST, dengan ukuran 28x28 *pixel channel* 1.
6. *Training* model dilakukan di CPU python.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan implementasi CNN pada perangkat FPGA, khususnya pada penggunaan komputasi *fixed-point*, pemetaan paralelisme operasi konvolusi, dan penerapan *pipeline* untuk mempercepat proses inferensi. Selain itu, penelitian ini dapat menjadi referensi dalam memahami proses perancangan akselerator CNN berbasis perangkat keras untuk aplikasi *embedded vision*.

1.6 Sistematika Penulisan

BAB I : PENDAHULUAN

Pada bab ini dijelaskan latar belakang, rumusan masalah, batasan, tujuan, manfaat, dan sistematika penulisan.

BAB II : TINJAUAN PUSTAKA DAN LANDASAN TEORI

Pada bab ini dijelaskan teori-teori dan penelitian terdahulu yang digunakan sebagai acuan dan dasar dalam penelitian.

BAB III : METODOLOGI PENELITIAN

Pada bab ini dijelaskan metode yang digunakan dalam penelitian meliputi langkah kerja, pertanyaan penelitian, alat dan bahan, serta tahapan dan alur penelitian.

BAB IV : HASIL DAN PEMBAHASAN

Pada bab ini dijelaskan hasil penelitian dan pembahasannya.

BAB V : KESIMPULAN DAN SARAN

Pada bab ini ditulis kesimpulan akhir dari penelitian dan saran untuk pengembangan penelitian selanjutnya.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Penelitian Che et al. dan Qasaimeh et al. menunjukkan bahwa pemilihan platform komputasi untuk aplikasi visi sangat dipengaruhi oleh karakteristik algoritma, latensi, dan kebutuhan energi [8, 9]. CPU memiliki fleksibilitas tinggi, tetapi kurang efisien untuk komputasi CNN yang membutuhkan paralelisme besar. GPU mampu memberikan throughput tinggi, namun umumnya membutuhkan daya dan bandwidth memori yang lebih besar. Dalam konteks tersebut, FPGA menjadi alternatif karena operasi CNN seperti konvolusi, *multiply-accumulate*, *buffering*, dan *pipeline* dapat dipetakan langsung ke sumber daya perangkat keras seperti DSP, BRAM, LUT, dan FF, sehingga sesuai untuk sistem *embedded vision* yang membutuhkan latensi rendah dan konsumsi daya efisien [10].

Berdasarkan penelitian Solovyev et al., implementasi CNN pada FPGA untuk *real-time processing* dilakukan dengan mengubah representasi *floating-point* menjadi *fixed-point* agar lebih sesuai dengan komputasi perangkat keras [11]. Penelitian ini menggunakan dataset MNIST dengan augmentasi berupa rotasi acak 10 derajat, inversi warna, serta penambahan *noise* dari 0% sampai 10%. Setelah proses pelatihan, parameter CNN disimpan ke dalam blok memori FPGA dan digunakan pada proses inferensi. Implementasi dilakukan pada board *DE0-Nano* berbasis FPGA *Intel Cyclone IV*, dengan masukan citra dari kamera OV7670 dan keluaran tampilan melalui VGA. Citra dari kamera direduksi menjadi *grayscale* berukuran 28×28 piksel untuk diproses oleh CNN, sehingga sistem mampu berjalan secara *real-time* dengan kecepatan lebih dari 150 *frame per second*. Penelitian ini menghasilkan akurasi model yang tidak berubah dengan model training sebelum kuantisasi.

Pada sisi kamera, penelitian Solovyev et al. juga memperlihatkan penggunaan OV7670 sebagai sumber citra untuk sistem CNN berbasis FPGA [11]. Konfigurasi kamera OV7670 dilakukan melalui *Serial Camera Control Bus* atau SCCB untuk mengatur format keluaran citra, resolusi, dan mode operasi sensor [12]. Data piksel kemudian dikirim melalui *pixel clock*, bus data paralel, serta sinyal sinkronisasi *HREF* dan *VSYNC*. Sinyal *HREF* menandai data piksel valid pada satu baris, sedangkan *VSYNC* menandai pergantian *frame*. Sistem inferensi *end-to-end* CNN berbasis kamera tidak hanya membutuhkan akselerator inferensi, tetapi juga akuisisi citra yang sinkron terhadap tampilan VGA.

Penelitian Qiu et al. mengimplementasikan CNN pada FPGA *Xilinx ZC706* dengan model *VGG16-SVD* untuk klasifikasi citra ImageNet menggunakan representasi 16-bit *fixed-point* [13]. Hasil implementasi menunjukkan akurasi *top-5* sebesar 86.66%,

performa inferensi sebesar 4.45 FPS, *throughput* sekitar 137 GOPS untuk keseluruhan jaringan, konsumsi daya sebesar 9.63 W. Penelitian ini menunjukkan bahwa kuantisasi 16-bit *fixed-point* masih dapat mempertahankan akurasi yang layak pada model CNN berskala besar. Dari sisi arsitektur, operasi konvolusi menjadi bagian komputasi dominan sehingga pemetaan operasi *multiply-accumulate* ke DSP, penggunaan *buffer*, serta pengaturan akses memori menjadi faktor penting dalam meningkatkan *throughput*.

Penelitian Dua et al. melalui arsitektur *Systolic-CNN* menunjukkan implementasi CNN berbasis *OpenCL* pada FPGA dengan pendekatan yang lebih fleksibel terhadap banyak model [14]. Berbeda dari Solovyev et al. dan Qiu et al. yang menggunakan *fixed-point*, penelitian ini menggunakan format 32-bit *floating-point*. Model yang diuji meliputi *AlexNet*, *ResNet-50*, *ResNet-152*, *RetinaNet*, dan *Light-weight RetinaNet*. Implementasi dilakukan pada board *Intel Arria 10 GX1150* dan *Intel Stratix 10 GX2800*. Pada *Arria 10*, inferensi *AlexNet* membutuhkan waktu sekitar 7 ms dan *ResNet-50* sekitar 84 ms, sedangkan pada *Stratix 10* latensinya menjadi sekitar 2 ms dan 33 ms. Paper ini tidak melaporkan konsumsi daya maupun efisiensi energi dalam GOPS/W, sehingga lebih tepat digunakan sebagai referensi arsitektur *systolic array*, pemanfaatan DSP, dan fleksibilitas *OpenCL*.

Selain penelitian CNN, integrasi kamera dan tampilan juga ditunjukkan oleh Navaneethan et al. melalui implementasi *video streaming* dari kamera OV7670 ke VGA pada board FPGA *Spartan-6* dan *Artix-7* menggunakan Verilog dan Xilinx Vivado [15]. Penelitian ini tidak berfokus pada inferensi CNN, tetapi menunjukkan bahwa jalur akuisisi kamera, sinkronisasi piksel, dan tampilan VGA dapat diimplementasikan dengan penggunaan sumber daya yang relatif kecil, yaitu 530 *slice registers*. Hasil tersebut relevan sebagai pembanding bagian antarmuka kamera dan display, terutama karena sistem CNN berbasis kamera tetap membutuhkan jalur video yang stabil sebelum citra diproses oleh akselerator.

Pada sistem yang memproses citra berukuran besar, penggunaan *frame buffer* menjadi penting karena data kamera dan data tampilan bekerja pada aliran waktu yang berbeda. Untuk resolusi VGA 640×480 dengan format RGB, kapasitas BRAM pada FPGA umumnya tidak cukup untuk menyimpan satu atau dua *frame* penuh [16]. Oleh karena itu, pada rancangan ini DDR digunakan sebagai *frame buffer*, sedangkan akses baca dan tulis dikendalikan melalui antarmuka AXI menuju MIG. AXI bertindak sebagai jalur transaksi antara modul *master* dan MIG sebagai pengendali DDR, dan proses akses memori dilakukan setelah MIG selesai dikalibrasi [17, 18].

Berdasarkan penelitian-penelitian tersebut, dapat disimpulkan bahwa setiap pendekatan memiliki fokus yang berbeda. Solovyev et al. telah menunjukkan sistem CNN *fixed-point* berbasis Cyclone IV dengan kamera OV7670, VGA, dan dataset MNIST, tetapi pembahasannya masih menggunakan arsitektur CNN yang sederhana serta belum

menekankan integrasi DDR sebagai frame buffer maupun strategi resource multiplexing secara mendalam [11]. Qiu et al. menunjukkan bahwa representasi fixed-point 16-bit dapat digunakan pada model besar seperti VGG16 di platform ZC706, tetapi fokus penelitian tersebut berada pada akselerator CNN dan bukan pada integrasi sistem embedded vision dengan kamera dan display [13]. Sementara itu, Navaneethan et al. membahas integrasi kamera OV7670 dan VGA, tetapi tidak menyertakan akselerator CNN sebagai blok inferensi [15].

Celah penelitian yang muncul adalah kebutuhan untuk mengintegrasikan blok akuisisi citra, penyimpanan frame, akselerator CNN, dan tampilan visual dalam satu sistem FPGA yang berjalan end-to-end. Penelitian ini mengisi celah tersebut dengan menggabungkan OV7670, DDR, VGA, ROI, CNN accelerator berbasis fixed-point 16-bit, argmax, serta strategi time multiplexing pada board Arty A7-100T. Dengan demikian, kontribusi penelitian tidak hanya berada pada implementasi komputasi CNN, tetapi juga pada integrasi subsistem embedded vision yang dapat menerima citra, menyimpan frame, menjalankan inferensi, dan menampilkan keluaran dalam satu platform perangkat keras.

2.2 Landasan Teori

2.2.1 Neural Network

Neural network merupakan model komputasi yang tersusun dari beberapa lapisan neuron buatan. Setiap neuron menerima sejumlah masukan, melakukan operasi penjumlahan berbobot, menambahkan bias, kemudian melewatkan hasilnya ke fungsi aktivasi. Mekanisme ini membuat *neural network* mampu membentuk hubungan non-linear antara data masukan dan keluaran. Dalam konteks klasifikasi citra, lapisan-lapisan tersebut digunakan untuk mengubah data piksel menjadi representasi fitur yang kemudian dipetakan menjadi kelas tertentu [19].

Operasi dasar pada sebuah neuron dapat dituliskan sebagai berikut:

$$z_j^{(l)} = \sum_{i=1}^N w_{j,i}^{(l)} a_i^{(l-1)} + b_j^{(l)} \quad (2-1)$$

$$a_j^{(l)} = \phi(z_j^{(l)}) \quad (2-2)$$

dengan $a_i^{(l-1)}$ adalah keluaran neuron dari lapisan sebelumnya, $w_{j,i}^{(l)}$ adalah bobot penghubung, $b_j^{(l)}$ adalah bias, $z_j^{(l)}$ adalah hasil penjumlahan berbobot, dan $\phi(\cdot)$ adalah fungsi aktivasi. Dalam bentuk matriks, operasi pada satu lapisan dapat ditulis sebagai:

$$\mathbf{a}^{(l)} = \phi(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}) \quad (2-3)$$

Pada proses klasifikasi, keluaran akhir jaringan biasanya merepresentasikan skor untuk masing-masing kelas. Skor tersebut dapat diproses menggunakan fungsi *softmax* agar menjadi nilai probabilitas:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{r=1}^C e^{z_r}} \quad (2-4)$$

dengan $\hat{y}_k$ adalah probabilitas untuk kelas ke- k dan C adalah jumlah kelas. Kelas prediksi kemudian dapat ditentukan dengan mengambil indeks yang memiliki nilai terbesar:

$$\hat{c} = \underset{k}{\operatorname{argmax}}(\hat{y}_k) \quad (2-5)$$

Pada CNN, konsep *neural network* tetap digunakan, tetapi lapisan awal jaringan tidak langsung berbentuk *fully connected*. Lapisan awal CNN menggunakan konvolusi agar proses ekstraksi fitur citra lebih efisien dan tetap mempertahankan informasi spasial dari masukan.

2.2.2 CNN

Convolutional Neural Network (CNN) merupakan pengembangan dari *neural network* yang dirancang khusus untuk mengolah data berbentuk grid, seperti citra digital. CNN memanfaatkan operasi konvolusi untuk mengekstraksi fitur lokal dari citra, kemudian menggabungkannya melalui beberapa lapisan agar jaringan dapat mengenali pola yang lebih kompleks. Pada lapisan awal, CNN biasanya menangkap fitur sederhana seperti tepi atau garis. Pada lapisan yang lebih dalam, fitur tersebut dapat berkembang menjadi bentuk, tekstur, atau pola yang lebih spesifik terhadap kelas tertentu [19, 20].

Secara umum, alur komputasi CNN dapat dipandang sebagai komposisi dari beberapa fungsi lapisan:

$$\mathbf{y} = f_{\theta}(\mathbf{x}) = g_L(g_{L-1}(\cdots g_2(g_1(\mathbf{x})) \cdots)) \quad (2-6)$$

dengan $\mathbf{x}$ adalah citra masukan, $\mathbf{y}$ adalah keluaran jaringan, g_l adalah operasi pada lapisan ke- l , dan θ adalah seluruh parameter yang dipelajari, seperti bobot dan bias. Lapisan pada CNN umumnya terdiri dari kombinasi konvolusi, fungsi aktivasi, *pooling*, dan *fully connected layer*. Kombinasi tersebut membuat CNN dapat melakukan ekstraksi fitur sekaligus klasifikasi dalam satu rangkaian komputasi.

Keunggulan CNN dibandingkan *fully connected neural network* biasa adalah adanya konsep *local connectivity* dan *weight sharing*. *Local connectivity* berarti neuron hanya terhubung pada area lokal dari citra, sedangkan *weight sharing* berarti kernel yang sama digunakan berulang pada banyak posisi. Kedua konsep ini membuat jumlah parameter CNN lebih efisien dibandingkan jika seluruh piksel langsung dihubungkan ke neuron pada lapisan berikutnya. Hal ini menjadi salah satu alasan CNN banyak digunak-

an pada tugas pengenalan citra dan klasifikasi objek [2].

Pada penelitian ini, CNN digunakan sebagai model utama untuk melakukan klasifikasi citra. Operasi yang paling dominan pada CNN adalah konvolusi karena melibatkan banyak operasi perkalian dan penjumlahan. Oleh karena itu, ketika CNN diimplementasikan pada FPGA, bagian konvolusi menjadi fokus utama untuk dioptimalkan melalui pemetaan operasi MAC, penggunaan format bilangan *fixed-point*, dan pemanfaatan paralelisme perangkat keras. Dengan pendekatan tersebut, CNN dapat dijalankan sebagai akselerator perangkat keras yang lebih sesuai untuk sistem *embedded vision* dan aplikasi *real-time*.

2.2.3 Konvolusi

Konvolusi merupakan operasi utama pada *Convolutional Neural Network* (CNN) yang digunakan untuk mengekstraksi fitur dari citra masukan. Pada operasi ini, sebuah *kernel* atau *filter* digeser pada area lokal citra untuk menghasilkan nilai fitur baru. Setiap nilai keluaran diperoleh dari hasil perkalian antara elemen citra dan elemen kernel, kemudian seluruh hasil perkalian tersebut dijumlahkan. Dengan mekanisme ini, konvolusi dapat menangkap pola lokal seperti tepi, garis, lengkungan, atau bentuk sederhana yang kemudian dapat digunakan oleh lapisan berikutnya untuk membentuk fitur yang lebih kompleks [19, 20].

Secara matematis, operasi konvolusi dua dimensi pada masukan dengan banyak kanal dapat dituliskan sebagai berikut:

$$y_{k,i,j} = b_k + \sum_{c=0}^{C_{in}-1} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} x_{c,iS+m-P,jS+n-P} \cdot w_{k,c,m,n} \quad (2-7)$$

dengan x adalah data masukan, w adalah bobot kernel, b_k adalah bias pada kanal keluaran ke- k , C_{in} adalah jumlah kanal masukan, K_h dan K_w adalah ukuran tinggi dan lebar kernel, S adalah *stride*, dan P adalah *padding*. Indeks i dan j menunjukkan posisi spasial pada *feature map* keluaran.

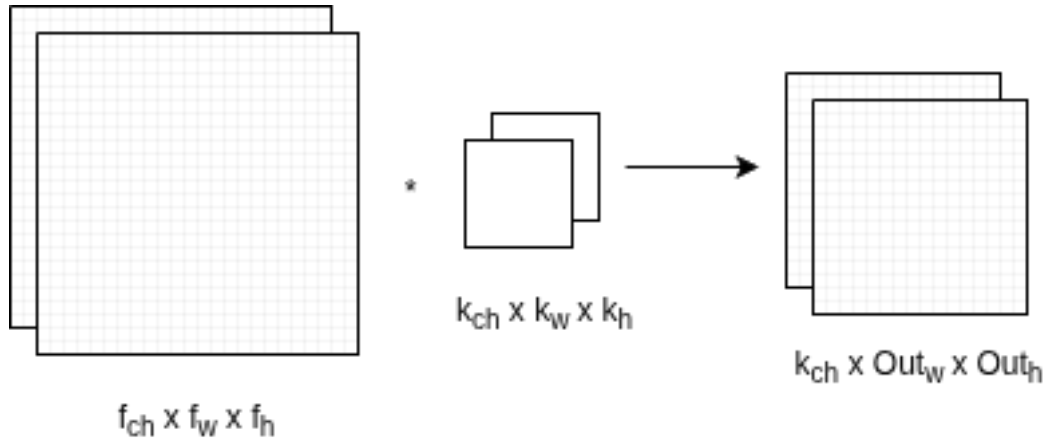
Ukuran keluaran dari operasi konvolusi dipengaruhi oleh ukuran masukan, ukuran kernel, *stride*, dan *padding*. Jika tinggi dan lebar masukan adalah H_{in} dan W_{in} , maka ukuran keluarannya dapat dihitung dengan:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K_h}{S} \right\rfloor + 1 \quad (2-8)$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2P - K_w}{S} \right\rfloor + 1 \quad (2-9)$$

Pada implementasi perangkat keras, persamaan konvolusi tersebut direalisasikan sebagai operasi *multiply accumulate* (MAC), yaitu perkalian antara data dan bobot yang diikuti

dengan penjumlahan akumulatif. Karena operasi ini dilakukan berulang pada banyak piksel dan banyak kernel, konvolusi menjadi bagian yang paling dominan dalam beban komputasi CNN.



Gambar 2.1. Operasi Konvolusi

2.2.4 Rectified Linear Unit (ReLU)

Rectified Linear Unit (ReLU) merupakan fungsi aktivasi yang banyak digunakan pada CNN karena bentuknya sederhana dan efisien secara komputasi. Fungsi ini akan meneruskan nilai masukan jika bernilai positif, sedangkan nilai negatif akan diubah menjadi nol. Dengan demikian, ReLU memberikan sifat non-linear pada jaringan tanpa membutuhkan operasi matematika yang kompleks seperti eksponensial atau trigonometri [2, 21].

Fungsi ReLU dapat dituliskan sebagai:

$$f(x) = \max(0, x) \quad (2-10)$$

atau dalam bentuk fungsi potongan:

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases} \quad (2-11)$$

Turunan fungsi ReLU dapat dituliskan sebagai:

$$f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases} \quad (2-12)$$

Pada titik $x = 0$, turunan ReLU tidak terdefinisi secara matematis, tetapi pada implementasi jaringan saraf biasanya dapat dipilih sebagai 0 atau 1 sesuai kebutuhan implementasi.

Dalam implementasi perangkat keras, ReLU sangat sederhana karena hanya membutuhkan pembandingan terhadap nol. Jika nilai masukan lebih besar dari nol, maka nilai tersebut diteruskan. Jika nilai masukan lebih kecil atau sama dengan nol, maka keluaran dibuat nol. Kesederhanaan ini membuat ReLU cocok digunakan pada akselerator CNN berbasis FPGA, terutama ketika sistem dirancang untuk menekan penggunaan sumber daya dan menjaga latensi komputasi tetap rendah.

2.2.5 Max Pool

Max pool merupakan operasi reduksi spasial yang digunakan untuk memperkecil ukuran *feature map* dengan cara mengambil nilai maksimum dari area tertentu. Operasi ini tidak memiliki parameter yang dipelajari seperti bobot atau bias, tetapi berperan penting dalam mengurangi ukuran data yang diteruskan ke lapisan berikutnya. Dengan ukuran *feature map* yang lebih kecil, jumlah operasi pada lapisan selanjutnya dapat dikurangi sehingga komputasi menjadi lebih ringan [2, 19].

Secara matematis, operasi *max pool* dapat dituliskan sebagai:

$$y_{c,i,j} = \max_{\substack{0 \leq m < K_h \\ 0 \leq n < K_w}} x_{c,iS+m,jS+n} \quad (2-13)$$

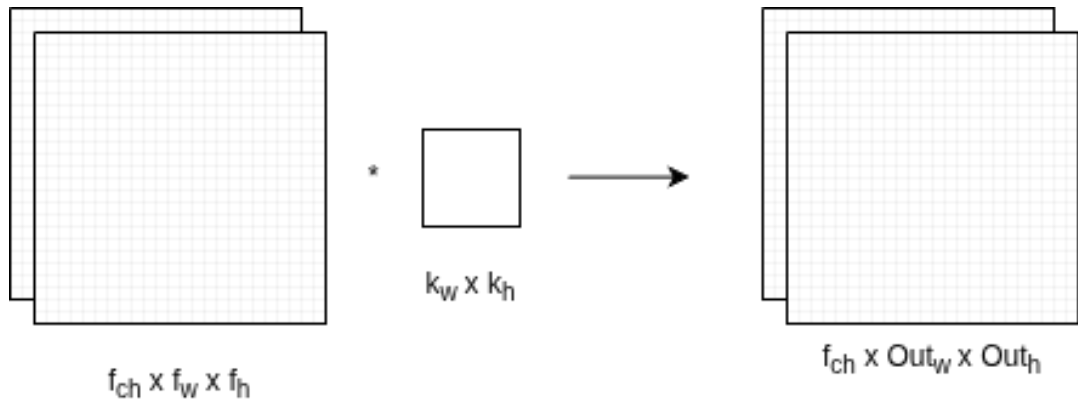
dengan x adalah *feature map* masukan, y adalah *feature map* keluaran, c adalah indeks kanal, K_h dan K_w adalah ukuran jendela pooling, serta S adalah *stride*. Jika digunakan jendela 2×2 dengan *stride* 2, maka ukuran spasial *feature map* akan berkurang menjadi setengah dari ukuran sebelumnya.

Ukuran keluaran dari operasi *max pool* dapat dihitung dengan persamaan yang mirip dengan konvolusi:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K_h}{S} \right\rfloor + 1 \quad (2-14)$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K_w}{S} \right\rfloor + 1 \quad (2-15)$$

Selain mengurangi ukuran data, *max pool* juga membantu mempertahankan fitur yang paling dominan pada suatu area. Hal ini membuat jaringan lebih toleran terhadap sedikit pergeseran posisi objek pada citra, karena nilai fitur yang kuat tetap dapat dipertahankan meskipun posisinya tidak selalu sama persis.



Gambar 2.2. Operasi Max Pool

2.2.6 Kuantitasi

Kuantitasi merupakan proses mengubah representasi numerik dari presisi tinggi menjadi presisi yang lebih rendah. Pada CNN, parameter seperti bobot, bias, dan aktivasi pada awalnya direpresentasikan dalam format *floating-point* karena format tersebut fleksibel untuk proses pelatihan [22]. Namun, format *floating-point* tidak efisien untuk implementasi perangkat keras karena membutuhkan rangkaian aritmetika yang lebih kompleks, ukuran data yang lebih besar, serta konsumsi sumber daya yang lebih tinggi. Oleh karena itu, kuantitasi menjadi tahap penting untuk mengubah model CNN agar lebih sesuai dengan karakteristik FPGA.

Pada implementasi CNN berbasis FPGA, kuantitasi berperan langsung terhadap efisiensi memori, bandwidth, jumlah sumber daya aritmetika, dan latensi inferensi. Jacob et al. menunjukkan bahwa inferensi CNN dapat dijalankan dengan aritmetika integer melalui skema kuantitasi yang tetap menjaga akurasi model [23]. Gupta et al. menunjukkan bahwa jaringan saraf dapat bekerja menggunakan representasi *fixed-point* 16-bit dengan degradasi akurasi yang kecil ketika proses pembulatan dilakukan secara tepat [24]. Lin et al. kemudian membahas kuantitasi *fixed-point* pada jaringan konvolusional dengan menekankan pemilihan *bit-width* yang sesuai pada setiap layer [25]. Pada konteks FPGA, Goyal et al. dan Solovyev et al. memperkuat bahwa perubahan dari *floating-point* menuju *fixed-point* dapat menurunkan kebutuhan komputasi dan memori sehingga model lebih layak diimplementasikan pada platform embedded dan akselerator perangkat keras [11, 26]. Dengan demikian, kuantitasi tidak hanya dilihat sebagai proses kompresi model, tetapi sebagai bagian dari strategi perancangan arsitektur agar operasi CNN dapat dipetakan secara efisien ke dalam jalur data FPGA.

Kuantitasi *int8* berfokus pada pemetaan nilai real ke bilangan integer 8-bit menggunakan faktor skala dan *zero-point*. Jacob et al. menggunakan skema ini untuk mendukung inferensi berbasis aritmetika integer, sedangkan Krishnamoorthi menekankan bahwa kuantitasi 8-bit pada bobot dan aktivasi dapat menurunkan ukuran model serta mem-

percepat inferensi pada perangkat yang mendukung operasi integer [23, 27].

$$S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}} \quad (2-16)$$

$$Z = \text{round} \left(q_{\min} - \frac{x_{\min}}{S} \right) \quad (2-17)$$

$$q_8 = \text{clip} \left(\text{round} \left(\frac{x}{S} \right) + Z, q_{\min}, q_{\max} \right) \quad (2-18)$$

$$\hat{x}_8 = S(q_8 - Z) \quad (2-19)$$

Kuantitasi *fixed-point* 16-bit menggunakan posisi bit pecahan yang tetap, sehingga nilai real dikonversi menjadi bilangan integer bertanda dengan faktor skala 2^F . Pada penelitian ini, format yang digunakan adalah Q2.14, yaitu representasi 16-bit bertanda dengan dua bit untuk bagian integer termasuk bit tanda dan 14 bit untuk bagian fraksi. Konfigurasi ini memberikan rentang nilai real sekitar $[-2, 2)$ dengan resolusi 2^{-14} . Format tersebut sesuai digunakan karena nilai parameter model berada pada rentang kecil dan tidak keluar dari rentang -2 sampai 2 . Gupta et al. membuktikan bahwa representasi 16-bit *fixed-point* dapat digunakan pada jaringan saraf dengan hasil yang tetap kompetitif, sedangkan Lin et al. dan Goyal et al. menunjukkan pentingnya pemilihan jumlah bit dan posisi pecahan agar kesalahan kuantitasi tidak merusak akurasi model [24–26]. Pada format ini, F menyatakan jumlah bit pecahan yang digunakan. Proses kuantitasi dan pendekatan kembali ke nilai real dapat dituliskan sebagai berikut.

$$q_{16} = \text{clip} \left(\text{round}(x \cdot 2^F), -2^{15}, 2^{15} - 1 \right) \quad (2-20)$$

$$\hat{x}_{16} = q_{16} \cdot 2^{-F} \quad (2-21)$$

Untuk parameter CNN, bobot, aktivasi, dan bias dapat dikonversi menggunakan jumlah bit pecahan yang disesuaikan dengan kebutuhan masing-masing parameter. Pada rancangan ini, bobot dan bias model menggunakan konfigurasi Q2.14 yang sama agar format parameter konsisten ketika dimuat ke memori Verilog. Dengan 14 bit fraksi, perubahan kecil pada parameter tetap dapat direpresentasikan, sedangkan dua bit bagian integer sudah cukup karena hasil pengamatan parameter menunjukkan nilai tidak melewati rentang -2 sampai 2 .

$$q_w = \text{clip} \left(\text{round}(w \cdot 2^{F_w}), -2^{15}, 2^{15} - 1 \right) \quad (2-22)$$

$$q_a = \text{clip} \left(\text{round}(a \cdot 2^{F_a}), -2^{15}, 2^{15} - 1 \right) \quad (2-23)$$

$$q_b = \text{clip} \left(\text{round}(b \cdot 2^{F_b}), -2^{31}, 2^{31} - 1 \right) \quad (2-24)$$

Trade-off utama antara *int8* dan *fixed-point* 16-bit terletak pada keseimbangan antara efisiensi dan kestabilan numerik. *Int8* memberikan ukuran data yang lebih kecil, kebutuhan memori yang lebih rendah, dan potensi bandwidth yang lebih hemat, tetapi membutuhkan pemilihan skala, *zero-point*, dan proses kalibrasi yang lebih hati-hati agar akurasi tidak turun secara signifikan. Sebaliknya, *fixed-point* 16-bit membutuhkan sumber daya yang lebih besar dibandingkan *int8*, tetapi memberikan rentang dan ketelitian yang lebih aman untuk menjaga hasil komputasi tetap dekat dengan model *floating-point*. Pada penelitian ini, *fixed-point* 16-bit Q2.14 dipilih karena memberikan titik tengah yang kuat antara efisiensi implementasi FPGA dan kestabilan akurasi inferensi. Pemilihan dua bit integer dan 14 bit fraksi juga didasarkan pada karakter parameter model yang berada di dalam rentang -2 sampai 2 , sehingga bit yang tersedia dapat diprioritaskan untuk meningkatkan resolusi pecahan.

2.2.7 Multiply Accumulate

Multiply Accumulate (MAC) adalah operasi komputasi yang terdiri dari proses perkalian dua nilai dan penjumlahan hasilnya ke nilai akumulasi sebelumnya. Secara umum, operasi MAC dapat dituliskan sebagai berikut:

$$y = \sum_{i=0}^{N-1} x_i w_i + b \quad (2-25)$$

Pada persamaan 2-25, x_i merupakan data masukan, w_i merupakan bobot, dan b merupakan nilai bias. Operasi ini banyak digunakan pada komputasi (CNN), terutama pada proses konvolusi dan *fully connected layer*. Pada proses tersebut, setiap nilai keluaran diperoleh dari hasil perkalian antara data masukan dan bobot, kemudian seluruh hasil perkalian tersebut dijumlahkan.

Pada FPGA, operasi MAC dapat diimplementasikan menggunakan blok logika umum atau menggunakan sumber daya khusus berupa DSP. Pada FPGA seri 7, blok DSP48E1 memiliki struktur yang mendukung operasi perkalian, penjumlahan, dan akumulasi, sehingga sesuai digunakan untuk mempercepat operasi MAC [28]. Penggunaan DSP membuat operasi MAC lebih efisien dibandingkan jika seluruh operasi aritmetika

dibangun hanya menggunakan LUT dan FF.

Namun, jumlah DSP pada FPGA bersifat terbatas. Jika setiap operasi perkalian dan akumulasi pada CNN dibuat secara paralel penuh, maka kebutuhan DSP dapat meningkat sangat besar [29]. Oleh karena itu, perancangan akselerator CNN pada FPGA perlu mempertimbangkan keseimbangan antara jumlah operasi paralel, penggunaan DSP, dan waktu komputasi.

2.2.8 Time Multiplexer

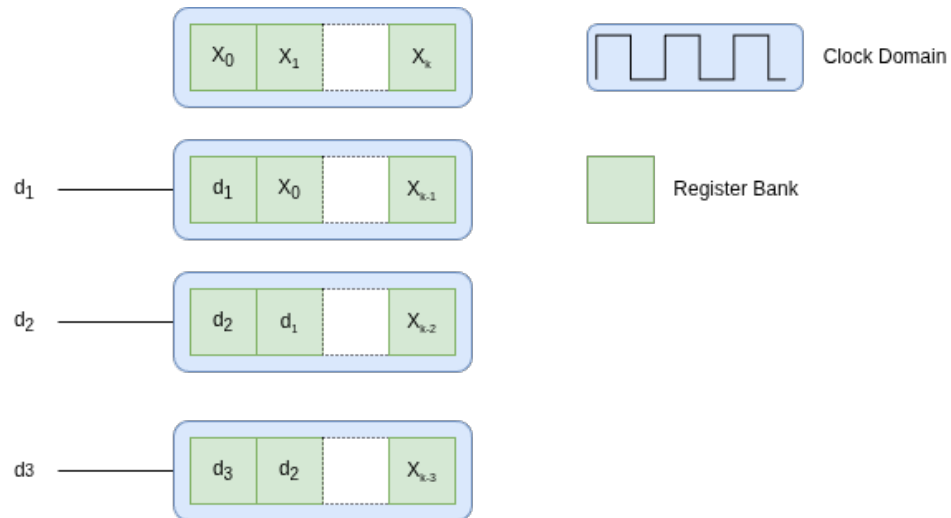
Time multiplexer adalah teknik penggunaan satu sumber daya hardware secara bergantian pada waktu yang berbeda. Dalam akselerator CNN, teknik ini digunakan ketika jumlah operasi MAC yang dibutuhkan lebih banyak dibandingkan jumlah DSP yang tersedia. Dengan teknik ini, satu unit MAC tidak hanya digunakan untuk satu operasi saja, tetapi dapat dipakai berulang untuk memproses beberapa data, channel, filter, atau neuron secara berurutan.

Berbeda dengan paralel penuh yang menyediakan banyak unit MAC sekaligus, time multiplexer menggunakan pendekatan berbasis pembagian waktu. Setiap siklus clock atau beberapa siklus clock digunakan untuk memproses bagian data tertentu, kemudian unit komputasi yang sama digunakan kembali untuk bagian data berikutnya [14, 22, 30]. Pendekatan ini dapat mengurangi penggunaan resource FPGA, tetapi waktu komputasi dapat menjadi lebih panjang karena sebagian operasi dilakukan secara bergantian.

Time multiplexing berkaitan erat dengan keterbatasan resource seperti DSP, LUT, dan memori internal [31]. Beberapa penelitian akselerator CNN pada FPGA menggunakan pendekatan *resource multiplexing* untuk mengurangi konsumsi hardware dengan cara memakai kembali unit komputasi yang sama pada beberapa tahap proses [29]. Pendekatan serupa juga digunakan pada arsitektur akselerator neural network yang menyesuaikan jumlah resource komputasi terhadap kebutuhan throughput tiap layer [30].

2.2.9 Shift Register

Shift register adalah rangkaian register yang digunakan untuk menyimpan dan menggeser data dari satu posisi ke posisi berikutnya pada setiap siklus *clock*. Pada sistem digital, shift register banyak digunakan ketika data harus diproses secara berurutan, terutama pada aliran data yang masuk secara *stream*. Setiap data baru yang masuk akan mendorong data sebelumnya ke posisi berikutnya, sehingga beberapa data terakhir dapat tetap tersedia secara bersamaan di dalam rangkaian register.



Gambar 2.3. Ilustrasi data *input shift register*.

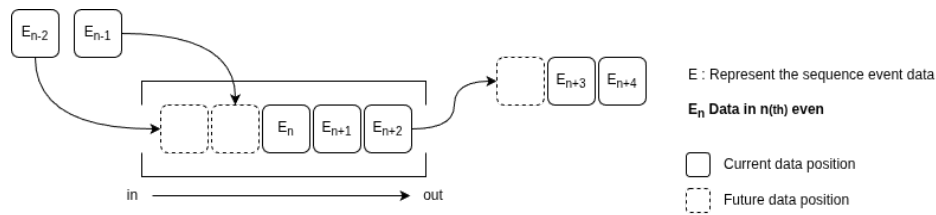
Pada operasi konvolusi citra, shift register memiliki peran penting karena kernel tidak hanya membutuhkan satu piksel, tetapi juga piksel-piksel tetangga di sekitarnya. Misalnya pada kernel berukuran 3×3 , proses konvolusi membutuhkan sembilan nilai piksel yang berasal dari tiga baris dan tiga kolom yang berdekatan [2]. Jika data citra masuk secara berurutan dari kiri ke kanan dan dari atas ke bawah, maka sistem tidak dapat langsung memperoleh sembilan piksel tersebut tanpa mekanisme penyimpanan sementara. Oleh karena itu, shift register digunakan untuk mempertahankan beberapa piksel terakhir dalam satu baris, sehingga susunan piksel horizontal dapat dibentuk secara bertahap.

Dalam akselerator CNN berbasis FPGA, shift register membantu membentuk *window* konvolusi secara langsung dari aliran data. Setelah data piksel masuk ke dalam jalur pemrosesan, nilai-nilai piksel tersebut digeser pada setiap siklus *clock* hingga membentuk susunan data yang sesuai dengan ukuran kernel. Dengan cara ini, operasi *multiply accumulate* pada kernel dapat dilakukan tanpa harus membaca ulang seluruh data citra dari memori utama. Pendekatan seperti ini sejalan dengan implementasi CNN pada FPGA yang menekankan penggunaan *pipeline*, komputasi *fixed-point*, serta pengurangan akses memori eksternal untuk meningkatkan efisiensi sistem [11, 22].

2.2.10 First In First Out (FIFO)

First In First Out atau FIFO adalah struktur penyimpanan data sementara yang bekerja dengan prinsip data yang pertama masuk akan menjadi data pertama yang keluar. FIFO banyak digunakan pada sistem digital untuk mengatur aliran data antarblok, terutama ketika dua bagian sistem tidak selalu bekerja pada kecepatan yang sama [32]. Dalam implementasi FPGA, FIFO dapat digunakan sebagai penyangga data agar proses pengiriman dan penerimaan data tetap stabil meskipun terdapat perbedaan waktu proses

antarblok.



Gambar 2.4. Ilustrasi FIFO

Pada arsitektur konvolusi, FIFO dapat digunakan sebagai *line buffer* untuk menyimpan satu atau beberapa baris data citra [11]. Ketika piksel citra masuk secara berurutan, FIFO menahan data dari baris sebelumnya sehingga data tersebut masih tersedia ketika baris berikutnya sedang diproses. Untuk membentuk *window* 3×3 , sistem umumnya membutuhkan dua FIFO sebagai penyangga dua baris sebelumnya. Data dari baris saat ini, baris sebelumnya, dan dua baris sebelumnya kemudian digabungkan dengan shift register untuk menghasilkan sembilan piksel yang dibutuhkan oleh kernel konvolusi.

Hubungan antara FIFO dan shift register dapat dilihat sebagai dua tingkatan penyimpanan yang saling melengkapi. FIFO bekerja pada tingkat baris citra, sedangkan shift register bekerja pada tingkat piksel dalam satu baris [11]. FIFO menyediakan data vertikal dari baris yang berbeda, sedangkan shift register menyusun data horizontal dari beberapa piksel yang berurutan. Dengan kombinasi ini, sistem dapat membentuk *sliding window* tanpa harus mengambil data berulang kali dari memori utama. Hal ini penting karena operasi konvolusi pada CNN membutuhkan akses data yang intensif, sehingga efisiensi penyimpanan sementara dan penggunaan ulang data menjadi bagian penting dalam rancangan akselerator CNN berbasis FPGA [22].

2.2.11 Finite State Machine

Finite State Machine atau FSM merupakan model kendali sekuensial yang merepresentasikan perilaku sistem dalam sejumlah *state* terbatas. Pada setiap siklus kerja, FSM membaca *input*, mengevaluasi kondisi transisi, menentukan *state* berikutnya, kemudian menghasilkan *output* sesuai dengan aturan transisi yang telah ditentukan seperti pada *Pseudocode* 1. Dengan pendekatan ini, proses kerja yang kompleks dapat dibagi menjadi beberapa tahap yang lebih terstruktur, sehingga alur kendali sistem menjadi lebih mudah dirancang, diamati, dan diverifikasi.

Dalam sistem digital, FSM banyak digunakan untuk mengatur proses yang membutuhkan urutan kerja tertentu, seperti proses inisialisasi, pembacaan data, penulisan data, sinkronisasi sinyal, dan pengendalian jalur *data path*. Pada penelitian ini, FSM digunakan sebagai bagian penting dalam pengendalian proses *input-output*, kendali transaksi AXI, kendali kamera, serta pengaturan tahapan komputasi pada CNN *Accelerator*. Ke-

Algorithm 1 inite State Machine Algorithm

Require: inputSignal, currentState, TransitionSet**Ensure:** updated currentState, outputSignal

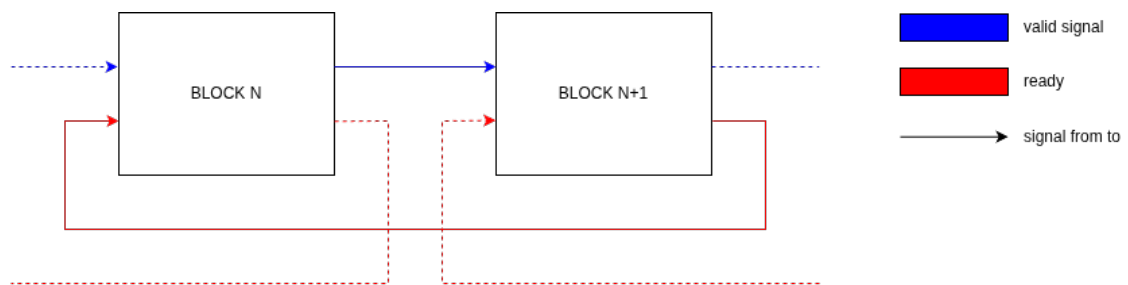
```
1: procedure UPDATEFSM(inputSignal)                                ▷ Initialize default values
2:   nextState ← currentState
3:   outputSignal ← DEFAULT_OUTPUT                                ▷ Evaluate all possible state transitions
4:   for all transition ∈ TransitionSet do
5:     if transition.sourceState == currentState and transition.inputCondition(inputSignal) == true then
6:       nextState ← transition.targetState                                ▷ Generate output
7:       outputSignal ← transition.outputAction(currentState, inputSignal, nextState)
8:       break
9:     end if
10:  end for
11:  currentState ← nextState                                ▷ Update the current state
12:  return outputSignal                                ▷ Return the generated output
13: end procedure
```

butuhan kendali tersebut menjadi semakin penting pada implementasi CNN di FPGA, karena proses komputasi CNN terdiri dari beberapa operasi berurutan seperti konvolusi, aktivasi, *pooling*, dan klasifikasi. CNN sendiri banyak digunakan pada tugas klasifikasi citra karena kemampuannya dalam mengekstraksi fitur secara bertingkat [1, 2]. Oleh karena itu, FSM berperan sebagai pengatur aliran proses agar setiap modul dapat bekerja secara sinkron, mulai dari penerimaan data citra, pemrosesan fitur, hingga keluaran hasil klasifikasi.

Pada sistem yang lebih kompleks, transisi FSM tidak hanya ditentukan oleh *input* utama, tetapi juga oleh kondisi kesiapan antarblok. Mekanisme ini berkaitan dengan *backpressure*, yaitu kondisi ketika sebuah modul harus menahan proses karena modul berikutnya belum siap menerima data. Dalam implementasi *advanced backpressure*, FSM dapat diperluas dengan sinyal kendali seperti *valid*, *ready*, *busy*, *done*, *fifo empty*, dan *fifo full*. Sinyal-sinyal tersebut digunakan sebagai syarat transisi agar perpindahan *state* hanya terjadi ketika data tersedia dan jalur keluaran siap menerima data. Dengan demikian, FSM tidak hanya berfungsi sebagai pengatur urutan kerja, tetapi juga sebagai mekanisme kontrol aliran data untuk mencegah kehilangan data, pembacaan data kosong, atau penulisan data ketika buffer sudah penuh.

2.2.12 Backpressure

Backpressure adalah mekanisme pengendalian aliran data yang digunakan ketika suatu blok belum siap menerima data dari blok sebelumnya. Mekanisme ini umumnya bekerja dengan sinyal *valid* dan *ready*. Sinyal *valid* menunjukkan bahwa data dari pengirim sudah tersedia, sedangkan sinyal *ready* menunjukkan bahwa penerima siap mengambil data tersebut [33, 34].



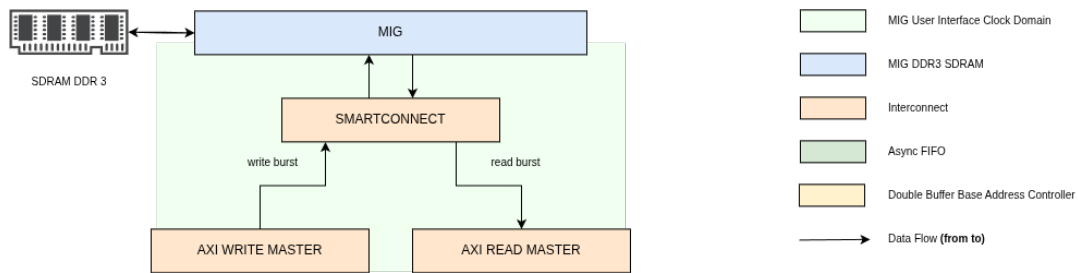
Gambar 2.5. Aliran sinyal *valid* dan *ready* pada sistem pipeline

Berdasarkan Gambar 2.5, sinyal *valid* mengalir searah dengan aliran data, yaitu dari blok proses sebelumnya menuju blok proses berikutnya. Sebagai contoh, ketika blok proses ke- N menghasilkan data, blok tersebut akan mengaktifkan sinyal *valid* ke blok proses ke- $N+1$. Sebaliknya, sinyal *ready* mengalir ke arah belakang, yaitu dari blok penerima menuju blok pengirim. Sinyal ini digunakan untuk memberitahu bahwa blok penerima sudah siap menerima dan memproses data berikutnya.

Jika sinyal *ready* belum aktif, maka pengirim harus menahan data sampai penerima siap. Dengan cara ini, data tidak hilang atau tertimpa oleh data berikutnya. Pada sistem berbasis *pipeline*, konsep backpressure penting untuk menjaga aliran data tetap teratur, terutama ketika setiap blok memiliki waktu proses yang berbeda [33].

2.2.13 Double Data Rate (DDR)

Double Data Rate atau DDR merupakan memori eksternal yang digunakan untuk menyediakan kapasitas penyimpanan lebih besar dibandingkan memori internal FPGA. Pada sistem berbasis citra, kebutuhan memori menjadi penting karena data yang diproses tidak hanya berupa satu piksel, tetapi dapat berupa satu frame penuh atau blok data berukuran besar. Untuk citra RGB565 beresolusi 640×480 , satu frame membutuhkan kapasitas penyimpanan yang jauh lebih besar dibandingkan penyimpanan lokal yang idealnya digunakan untuk buffer kecil, register, atau blok komputasi. Oleh karena itu, DDR ditempatkan sebagai *frame buffer* utama, sedangkan BRAM digunakan untuk menyimpan data sementara pada jalur pemrosesan yang lebih dekat dengan logika FPGA. Gambar 2.6 menunjukkan posisi DDR pada sistem kamera, AXI, MIG, dan VGA.



Gambar 2.6. Ilustrasi hubungan AXI, MIG, dan DDR pada sistem pemindahan data berbasis memori eksternal.

Advanced eXtensible Interface atau AXI merupakan protokol antarmuka yang digunakan untuk menghubungkan modul komputasi dengan subsistem memori atau periferal pada sistem digital. AXI mendukung transaksi *read* dan *write* secara terpisah, memiliki kanal alamat, data, dan respons, serta memungkinkan transfer data dalam bentuk *burst*. Karakter ini membuat AXI sesuai untuk pemindahan data berukuran besar karena beberapa data dapat dikirim atau diambil secara berurutan tanpa harus membentuk alamat baru untuk setiap data. Pada sistem pemrosesan citra, AXI dapat digunakan untuk mengirim data ke memori melalui AXI *write* master dan mengambil kembali data dari memori melalui AXI *read* master. Jika diperlukan, buffer lokal seperti FIFO atau BRAM dapat ditempatkan di sisi modul untuk menyesuaikan laju data, tetapi mekanisme utama pemindahan data tetap dilakukan melalui transaksi AXI [35].

Memory Interface Generator atau MIG merupakan IP yang digunakan sebagai pengendali akses DDR pada FPGA. DDR tidak dapat diakses secara langsung seperti register biasa karena memori ini memiliki aturan waktu, proses inisialisasi, kalibrasi, pengaturan alamat, dan sinyal fisik yang lebih kompleks. MIG menyederhanakan bagian tersebut dengan menyediakan antarmuka yang dapat digunakan oleh logika FPGA, salah satunya melalui AXI. Dengan adanya MIG, modul logika tidak perlu mengatur sinyal fisik DDR secara langsung. Modul cukup membentuk transaksi baca atau tulis melalui antarmuka AXI, sedangkan MIG menerjemahkan transaksi tersebut menjadi operasi memori yang sesuai dengan protokol DDR. Peran ini membuat MIG menjadi penghubung penting antara logika digital di dalam FPGA dan memori eksternal yang memiliki aturan akses lebih ketat [36].

Penggunaan DDR pada akselerator CNN dilakukan pada penelitian yang membutuhkan penyimpanan data berukuran besar dan akses memori yang terstruktur. Solovyeve et al. menggunakan memori eksternal sebagai bagian dari sistem pemrosesan video real-time, sementara memori lokal tetap digunakan untuk mendukung aliran data yang dekat dengan blok komputasi [11]. Zhang et al. menggunakan DDR3 sebagai penyimpanan data utama pada akselerator CNN berbasis FPGA, kemudian mengoptimalkan akses data menggunakan *loop tiling* dan pemanfaatan BRAM agar bandwidth memori dapat digu-

nakan secara lebih efisien [22]. Kedua penelitian tersebut menunjukkan bahwa DDR berperan sebagai penyimpanan eksternal ketika ukuran data melebihi kapasitas memori lokal FPGA, sedangkan memori internal digunakan untuk mempercepat akses data yang sedang aktif diproses.

2.2.14 FPGA ARTIX 7

FPGA Artix-7 merupakan keluarga FPGA dari Xilinx yang dirancang untuk memberikan keseimbangan antara performa, konsumsi daya, dan biaya implementasi. Pada penelitian ini, board yang digunakan adalah Arty A7-100T dengan perangkat FPGA XC7A100T. Board ini menyediakan sumber daya logika, memori internal, DSP, dan DDR eksternal yang cukup untuk membangun sistem akselerator CNN skala embedded. Spesifikasi utama Arty A7-100T ditunjukkan pada Tabel 2.1.

Tabel 2.1. Spesifikasi utama Arty A7-100T

Komponen	Spesifikasi
FPGA	Xilinx Artix-7 XC7A100T
LUT	63400
Flip-Flop	126800
Block RAM	4860 Kb
DSP Slice	240 DSP48E1
DDR	256 MB DDR3L

Jika dibandingkan dengan penelitian sebelumnya, Arty A7-100T berada pada kelas menengah. Board ini memiliki sumber daya lebih besar dibandingkan FPGA lainnya seperti Spartan-6 atau Cyclone IV yang digunakan pada penelitian Solovyev et al., tetapi masih lebih terbatas dibandingkan board kelas tinggi seperti Virtex-7 VC707 yang digunakan pada penelitian Zhang et al. [11, 22]. Kondisi ini membuat Arty A7-100T cukup mampu untuk implementasi CNN *accelerator*, karena desain harus benar-benar memperhatikan pemakaian resource. Operasi MAC dapat dipetakan ke DSP48E1, data sementara dapat disimpan pada BRAM, sedangkan FF dan LUT digunakan untuk membangun jalur kendali, pipeline, FSM, serta register antarblok.

Dari sisi daya dan latensi, FPGA Artix-7 mendukung pendekatan yang lebih hemat daya dibandingkan komputasi umum karena jalur data dapat dibuat khusus sesuai kebutuhan algoritma. Penggunaan komputasi *fixed-point*, DSP untuk operasi MAC, serta pipeline dapat menurunkan latensi komputasi karena operasi tidak perlu dijalankan sebagai instruksi perangkat lunak satu per satu. Namun, performa akhir tetap bergantung pada keseimbangan antara jumlah operasi paralel, akses DDR, dan penggunaan *time mul-*

tiptexting. Semakin banyak operasi dibuat paralel, latensi dapat ditekan, tetapi konsumsi resource dan daya dinamis meningkat. Sebaliknya, pemakaian ulang resource dapat menghemat DSP dan LUT, tetapi waktu komputasi menjadi lebih panjang.

2.2.15 OV7670

OV7670 merupakan sensor kamera CMOS beresolusi VGA yang dapat menghasilkan citra hingga ukuran 640×480 piksel. Sensor ini banyak digunakan pada implementasi FPGA karena menyediakan keluaran data paralel dan sinyal sinkronisasi yang relatif mudah dihubungkan dengan logika digital. Konfigurasi internal kamera dilakukan melalui Serial Camera Control Bus atau SCCB, sedangkan data citra dikeluarkan melalui jalur data paralel yang disertai sinyal *pixel clock*, HREF, dan VSYNC [12].

Pada aliran data kamera, sinyal VSYNC digunakan sebagai penanda pergantian frame, HREF digunakan untuk menunjukkan bahwa data pada satu baris sedang valid, dan *pixel clock* digunakan sebagai acuan pengambilan data piksel. OV7670 dapat menghasilkan beberapa format keluaran, seperti YUV, RGB, dan RGB565. Format RGB565 sering digunakan pada sistem FPGA karena satu piksel dapat direpresentasikan dalam 16-bit, yaitu 5-bit merah, 6-bit hijau, dan 5-bit biru. Format ini cukup ringan untuk disimpan dan diproses, tetapi tetap mampu menampilkan warna yang memadai untuk sistem embedded vision.

Dalam sistem CNN berbasis kamera, OV7670 berperan sebagai sumber data visual yang masuk ke sistem secara kontinu. Hal ini membuat logika penerima kamera harus mampu membaca data berdasarkan sinyal sinkronisasi, menyusun data piksel dengan benar, dan menjaga agar aliran data tidak terputus. Karena data kamera datang mengikuti domain *pixel clock*, rancangan FPGA perlu memperhatikan sinkronisasi, buffering, dan validitas data sebelum data tersebut digunakan untuk pemrosesan atau ditampilkan kembali.

2.2.16 VGA

Video Graphics Array atau VGA merupakan antarmuka tampilan analog yang mengirimkan sinyal warna merah, hijau, dan biru bersama sinyal sinkronisasi horizontal dan vertikal. Pada FPGA, tampilan VGA dibentuk dengan menghasilkan data warna RGB dan sinyal timing yang sesuai dengan resolusi layar. Untuk resolusi 640×480 , sistem perlu menghasilkan urutan piksel, periode *blanking*, sinyal HSYNC, dan sinyal VSYNC agar monitor dapat menampilkan gambar dengan stabil.

Pada penelitian ini, keluaran VGA dapat dibantu menggunakan Pmod VGA dari Digilent. Pmod VGA menyediakan jalur warna 12-bit, yaitu 4-bit untuk merah, 4-bit untuk hijau, dan 4-bit untuk biru, serta dua sinyal sinkronisasi standar berupa HSYNC dan VSYNC [37]. Modul ini mempermudah FPGA untuk menghasilkan keluaran VGA

karena konversi sinyal digital RGB menuju level analog dilakukan melalui rangkaian resistor pada Pmod. Dengan demikian, FPGA cukup menghasilkan data warna digital dan sinyal sinkronisasi yang sesuai dengan timing VGA.

Dalam sistem embedded vision, VGA berfungsi sebagai jalur keluaran visual untuk menampilkan hasil pembacaan kamera atau hasil pemrosesan citra. Keberadaan VGA membuat sistem dapat diamati secara langsung tanpa harus selalu mengirim data ke komputer. Dari sisi perancangan, bagian VGA menuntut aliran data yang stabil karena tampilan layar bergerak mengikuti timing yang tetap. Jika data piksel tidak tersedia pada saat dibutuhkan, tampilan dapat mengalami gangguan seperti warna tidak sesuai, gambar bergeser, atau frame terlihat tidak stabil.

2.3 Analisis Perbandingan Metode

Pada penelitian sebelumnya menunjukkan bahwa hasil inferensi CNN pada FPGA dapat diselesaikan sebelum satu frame dapat selesai terbentuk. Camera OV7670 memiliki 30 FPS [12]. Sedangkan latensi terbesar didapat pada nilai 150 FPS [11]. Sedangkan konsumsi daya pada FPGA relatif lebih rendah dibandingkan dengan CPU dan GPU. Efisiensi daya ini pada penelitian yang dilakukan oleh Qasaimeh et al. dan Guo et al. [9, 38]. Hal ini menunjukkan bahwa FPGA cocok untuk digunakan dalam implementasi CNN dengan daya lebih rendah.

Tabel 2.2. Perbandingan implementasi sistem pada penelitian terkait

Penelitian	CNN	Kamera	VGA	Board/Platform
Solovyev et al. [11]	Ya	Ya	Ya	<i>DE0-Nano & Intel Cyclone IV</i>
Qiu et al. [13]	Ya	–	–	<i>Xilinx ZC706</i>
Dua et al. [14]	Ya	–	–	<i>Arria 10 GX1150 & Stratix 10 GX2800</i>
Navaneethan et al. [15]	–	Ya	Ya	<i>Spartan-6 & Artix-7</i>
Sun et al. [16]	–	Ya	Ya	<i>FPGA MicroBlaze based</i>

Berdasarkan Tabel 2.2, penelitian terkait dapat dikelompokkan menjadi dua arah utama. Penelitian seperti Solovyev et al. sudah menunjukkan implementasi inferensi CNN secara *end-to-end* pada FPGA, sedangkan Qiu et al. dan Dua et al. lebih menekankan optimasi akselerator CNN dari sisi komputasi model, arsitektur paralel, dan efisiensi operasi. Di sisi lain, Navaneethan et al. dan Sun et al. lebih berfokus pada integrasi sistem citra, yaitu pengambilan data dari kamera OV7670 dan penampilan citra melalui VGA, tetapi belum membahas akselerasi inferensi CNN. Hal ini menunjukkan adanya celah penelitian pada integrasi antara sistem akuisisi citra, tampilan video, dan akselerator CNN dalam satu platform FPGA. Oleh karena itu, penelitian ini tidak hanya berfokus pada implementasi CNN berbasis *fixed-point*, tetapi juga pada bagaimana sistem tersebut dapat diintegrasikan dengan periferal citra agar lebih mendekati kebutuhan

aplikasi *embedded vision* secara langsung.

Tabel 2.3. Perbandingan performa penelitian CNN Accelerator

Penelitian	Latensi/FPS	Arsitektur Model	Daya (W)	GOPS	Loss	Akurasi
[11]	>150 FPS	CNN	–	–	–	Tidak berubah setelah kuantisasi
[13]	224.60 ms	VGG16-SVD	9.63	137	–	86.66%
[14]	140ms	AlexNet	2,1	–	–	–

Selain aspek integrasi sistem, perbandingan performa pada Tabel 2.3 juga menunjukkan bahwa konsumsi daya menjadi faktor penting dalam implementasi CNN. Pada penelitian Qiu et al., implementasi VGG16-SVD pada FPGA hanya membutuhkan daya sebesar 9,63 W, sedangkan platform CPU dan GPU yang digunakan sebagai pembanding memiliki konsumsi daya yang jauh lebih tinggi, yaitu 135 W untuk CPU dan 250 W untuk GPU [13]. Meskipun GPU mampu memberikan performa komputasi yang lebih besar, konsumsi dayanya mencapai sekitar 26 kali lebih tinggi dibandingkan FPGA. Kondisi ini memperlihatkan bahwa CPU dan GPU tidak selalu menjadi pilihan paling efisien untuk sistem *real-time* dan *embedded*, terutama ketika batasan daya dan latensi menjadi pertimbangan utama. Dengan demikian, FPGA menjadi alternatif yang relevan karena dapat menyediakan jalur komputasi khusus melalui paralelisme, *pipeline*, dan representasi *fixed-point* sehingga proses inferensi dapat dilakukan dengan konsumsi daya yang lebih rendah.

BAB III

METODE PENELITIAN

3.1 Alat dan Bahan

Alat dan bahan yang digunakan pada penelitian ini terbagi atas perangkat keras dan perangkat lunak yang akan dijelaskan seperti berikut.

3.1.1 Perangkat Keras

Perangkat keras terdiri dari sebuah FPGA board yang digunakan sebagai media inferensi model CNN serta memori DDR SDRAM yang digunakan sebagai *double frame memory*. Detail mengenai perangkat keras yang digunakan terdapat pada daftar berikut.

1. Digilent Arty A7-100T,
2. DDR SDRAM,
3. *USB*,
4. 14 Jumper *Male-Female*,

3.1.2 Perangkat Lunak

Perangkat lunak yang digunakan selama proses pengembangan mencakup lingkungan kerja dan lingkungan uji. Detail mengenai perangkat lunak yang digunakan terdapat pada daftar berikut.

1. Software VIVADO,
2. Linux Operating System,
3. Python,
4. Kaggle,
5. Verilog Icairus,
6. Visual Studio Code.

3.2 Rancangan Sistem

3.2.1 Rancangan Umum Sistem

DDR SDRAM digunakan sebagai *double frame memory* agar dua frame citra dapat disimpan secara bergantian. Satu frame berperan sebagai buffer aktif untuk proses baca dan inferensi, sedangkan frame lainnya digunakan sebagai buffer tulis untuk data berikutnya. Pemisahan ini membantu menghindari konflik akses baca-tulis pada memori yang sama dan membuat aliran data menuju CNN accelerator lebih stabil.

Mekanisme *double frame memory* juga memudahkan verifikasi sistem karena data masukan dan keluaran dapat dipertahankan dalam buffer yang berbeda. Dengan demikian, proses evaluasi tidak bergantung pada periferal akuisisi atau tampilan eksternal, melainkan cukup pada konsistensi data yang disimpan, dibaca, dan dipindahkan antarbuffer oleh pengendali memori.

3.2.2 Dataset

Dataset yang digunakan pada penelitian ini adalah dataset MNIST yang diperoleh dari Kaggle [39]. Dataset MNIST merupakan kumpulan citra angka tulisan tangan dari digit 0 sampai 9 yang umum digunakan sebagai *benchmark* awal pada pengembangan sistem klasifikasi citra [20]. Dataset ini terdiri dari 60.000 citra untuk pelatihan dan 10.000 citra untuk pengujian. Setiap citra memiliki ukuran 28×28 piksel dan direpresentasikan dalam bentuk *grayscale*, sehingga setiap sampel hanya memiliki satu kanal intensitas. Karakteristik ini sesuai dengan kebutuhan penelitian karena rancangan CNN menggunakan satu kanal input dan menghasilkan 10 kelas keluaran, yaitu kelas digit 0 sampai 9.

Data MNIST yang diunduh dari Kaggle disediakan sebagai dataset publik yang berasal dari internet. Data tersebut digunakan sebagai sumber data utama pada proses pelatihan dan evaluasi model CNN di Python. Sebelum digunakan oleh model, citra dibaca sebagai matriks piksel berukuran 28×28 , dinormalisasi, kemudian dipasangkan dengan label kelasnya. Data latih digunakan untuk memperoleh parameter bobot dan bias, sedangkan data uji digunakan untuk mengukur kemampuan model terhadap sampel yang tidak digunakan pada proses pelatihan. Pada tahap implementasi, citra dan hasil inferensi dapat ditempatkan pada *double frame memory* sehingga aliran data tetap terpisah antara buffer aktif dan buffer cadangan.

Pemilihan MNIST juga dilakukan karena ukuran citra relatif kecil dan struktur datanya sederhana, sehingga sesuai untuk tahap validasi awal akselerator CNN berbasis FPGA. Ukuran input 28×28 memungkinkan proses konvolusi, pooling, dan *fully connected* diuji tanpa membutuhkan kapasitas memori dan resource komputasi yang terlalu besar. Hal ini penting karena fokus penelitian bukan hanya pada akurasi model, tetapi juga pada keterlaksanaan implementasi hardware, penggunaan resource FPGA, latensi inferensi, serta perbandingan hasil komputasi antara Python dan Verilog. Dengan demikian, dataset MNIST digunakan sebagai dasar pelatihan dan pengujian model sebelum parameter model dikonversi ke format *fixed-point* untuk implementasi pada FPGA.

3.2.3 Rancangan Arsitektur CNN

Arsitektur CNN yang digunakan pada penelitian ini dirancang sebagai model klasifikasi citra berukuran kecil agar sesuai dengan keterbatasan sumber daya pada FPGA.

Input model berupa citra grayscale berukuran 28×28 piksel yang diperoleh dari hasil pemrosesan citra pada buffer aktif. Model CNN dikembangkan terlebih dahulu menggunakan Python sebagai model acuan sebelum parameter bobot dan biasnya dipindahkan ke implementasi Verilog. Penggunaan CNN dipilih karena metode ini mampu mengekstraksi fitur citra secara bertahap melalui operasi konvolusi, aktivasi, dan pooling [19,20].

Secara umum, model terdiri dari tiga blok konvolusi dan dua lapisan *fully connected*. Blok pertama menerima satu kanal input dan menghasilkan 8 kanal fitur. Blok kedua menghasilkan 16 kanal fitur, sedangkan blok ketiga menghasilkan 32 kanal fitur. Setiap blok konvolusi menggunakan kernel 3×3 yang diikuti oleh fungsi aktivasi ReLU dan proses *max pooling*. Setelah tahap ekstraksi fitur selesai, hasil akhir diproses oleh lapisan *fully connected* untuk menghasilkan 10 nilai keluaran yang merepresentasikan kelas prediksi. Pada implementasi hardware, tahap *softmax* tidak digunakan karena proses klasifikasi cukup dilakukan dengan membandingkan nilai keluaran terbesar menggunakan *argmax*.

Arsitektur ini dipilih karena memiliki jumlah parameter yang relatif kecil, tetapi tetap mampu merepresentasikan proses ekstraksi fitur CNN secara bertingkat. Selain itu, ukuran model yang tidak terlalu besar membuatnya lebih memungkinkan untuk diimplementasikan pada FPGA dengan komputasi *fixed-point*. Parameter model berupa bobot dan bias disimpan sebagai data konstan pada memori internal, sehingga proses inferensi dapat dilakukan secara langsung oleh rangkaian hardware tanpa memerlukan proses pelatihan ulang di dalam FPGA.

3.2.4 Rancangan Representasi Fixed-Point

Representasi numerik pada rancangan akselerator CNN dibuat menggunakan format *fixed-point* 16-bit. Pemilihan format ini dilakukan karena model CNN pada tahap pelatihan masih menggunakan *floating-point*, sedangkan implementasi pada FPGA lebih sesuai apabila operasi aritmetika direalisasikan dalam bentuk bilangan integer atau *fixed-point*. Operasi *floating-point* membutuhkan rangkaian aritmetika yang lebih kompleks dan dapat meningkatkan penggunaan resource, terutama pada operasi perkalian dan akumulasi yang muncul berulang pada layer konvolusi dan *fully connected*. Oleh karena itu, parameter model hasil pelatihan, seperti bobot dan bias, dikonversi terlebih dahulu ke dalam format *fixed-point* sebelum dimasukkan ke dalam memori parameter pada Verilog [11,22].

Pada penelitian ini, representasi *fixed-point* menggunakan format Q2.14, yaitu format 16-bit bertanda dengan dua bit untuk bagian integer termasuk bit tanda dan 14 bit untuk bagian fraksi. Dengan konfigurasi tersebut, nilai *floating-point* dikalikan dengan faktor skala 2^{14} , kemudian dibulatkan dan dibatasi agar tetap berada pada rentang bilangan bertanda 16-bit. Format Q2.14 memiliki rentang nilai real sekitar $[-2, 2)$ dengan

resolusi sebesar 2^{-14} . Pemilihan format ini dilakukan karena nilai parameter model, baik bobot maupun bias, tidak keluar dari rentang -2 sampai 2 , sehingga dua bit bagian integer sudah cukup untuk menampung rentang nilai parameter.

Pemakaian 14 bit fraksi penting karena nilai bobot CNN umumnya berada pada rentang kecil, sehingga dibutuhkan resolusi pecahan yang cukup halus agar informasi parameter tidak banyak hilang setelah proses kuantisasi. Dengan memprioritaskan bit untuk bagian fraksi, perubahan kecil pada parameter masih dapat direpresentasikan dengan lebih baik dibandingkan apabila lebih banyak bit dialokasikan untuk bagian integer. Secara umum, proses konversi dari nilai real x ke representasi *fixed-point* 16-bit dapat dituliskan sebagai berikut.

$$q_{16} = \text{clip}(\text{round}(x \cdot 2^F), -2^{15}, 2^{15} - 1) \quad (3-1)$$

$$\hat{x}_{16} = q_{16} \cdot 2^{-F} \quad (3-2)$$

Pada Persamaan 3-1 dan 3-2, fungsi $\text{round}(\cdot \cdot \cdot)$ digunakan untuk membulatkan hasil perkalian skala ke bilangan integer terdekat, sedangkan fungsi $\text{clip}(\cdot \cdot \cdot)$ digunakan untuk mencegah nilai keluar dari rentang 16-bit bertanda. Dengan $F = 14$, nilai yang disimpan memiliki resolusi sebesar 2^{-14} . Artinya, perubahan kecil pada nilai bobot masih dapat direpresentasikan dengan cukup baik. Konfigurasi ini dipilih sebagai titik tengah antara ketelitian numerik dan efisiensi hardware. Jumlah bit pecahan yang terlalu kecil dapat memperbesar galat kuantisasi, sedangkan jumlah bit pecahan yang terlalu besar dapat mempersempit rentang nilai integer yang dapat direpresentasikan. Pada rancangan ini, penyempitan rentang integer tersebut masih dapat diterima karena parameter hasil pelatihan berada di dalam rentang -2 sampai 2 [25].

Konversi parameter pada setiap layer dilakukan dengan prinsip yang sama [24]. Bobot hasil pelatihan dikonversi menjadi q_w , aktivasi dikonversi menjadi q_a , dan bias dikonversi menjadi q_b . Pada rancangan ini, bobot dan bias disimpan dalam format 16-bit bertanda, sedangkan hasil perkalian dan akumulasi pada jalur MAC dibuat lebih lebar agar tidak langsung mengalami *overflow*. Proses kuantisasi masing-masing parameter dapat dituliskan sebagai berikut.

Proses translasi parameter dari Python ke Verilog dilakukan dengan mengubah nilai *floating-point* hasil pelatihan menjadi data heksadesimal *fixed-point*. Kode translasi tersebut pada dasarnya melakukan tiga tahap utama, yaitu mengalikan nilai parameter dengan 2^F , membulatkan hasilnya ke bilangan integer, kemudian membatasi nilai agar tetap berada pada rentang 16-bit bertanda. Hasil konversi selanjutnya disimpan ke dalam file memori seperti `conv1_w.mem`, `conv1_b.mem`, `conv2_w.mem`, `conv2_b.mem`,

dan file parameter lain yang digunakan oleh modul Verilog. Dengan cara ini, parameter yang digunakan pada simulasi Python dan implementasi Verilog tetap berasal dari model yang sama, tetapi sudah berada dalam bentuk numerik yang sesuai untuk jalur data FPGA.

3.2.5 Rancangan Akselerator CNN pada FPGA

Akselerator CNN pada FPGA dirancang untuk menjalankan proses inferensi secara langsung pada hardware. Operasi utama pada CNN adalah konvolusi, yaitu proses perkalian antara data input dengan bobot kernel yang kemudian dijumlahkan. Oleh karena itu, bagian utama dari akselerator CNN direalisasikan menggunakan struktur *multiply accumulate* (MAC). Operasi MAC dipetakan ke DSP pada FPGA karena DSP lebih sesuai untuk menjalankan operasi perkalian dan akumulasi dibandingkan apabila seluruh operasi dilakukan menggunakan LUT. Namun, karena jumlah DSP pada FPGA terbatas, rancangan akselerator tidak dibuat paralel penuh, melainkan menggunakan kombinasi paralelisme dan *time multiplexing* [28, 29]. Target pemetaan resource pada masing-masing bagian CNN ditunjukkan pada Tabel 3.1.

Tabel 3.1. Target penggunaan resource pada implementasi CNN

Layer CNN	Input	Output	Target DSP	Strategi Implementasi
Conv 1	1 <i>channel</i>	8 <i>channel</i>	72	Paralel untuk 8 output <i>channel</i>
Conv 2	8 <i>channel</i>	16 <i>channel</i>	72	<i>Time multiplexing</i> 16 output <i>channel</i>
Conv 3	16 <i>channel</i>	32 <i>channel</i>	72	<i>Time multiplexing</i> 32 output <i>channel</i>
Dense	32 <i>feature</i>	16 <i>feature</i>	8	MAC digunakan bergantian pada setiap neuron
Dense	16 <i>feature</i>	10 kelas	10	MAC digunakan bergantian pada setiap neuron

Pada rancangan ini, blok konvolusi pertama dibuat lebih paralel karena jumlah kanal input dan output masih kecil. Sementara itu, blok konvolusi kedua dan ketiga menggunakan *time multiplexing* agar jumlah DSP yang digunakan tetap dapat dikendalikan. Conv2 menghitung beberapa output *channel* secara bergantian menggunakan kelompok

MAC yang sama. Conv3 juga menggunakan pendekatan serupa, tetapi kanal input dibagi menjadi dua kelompok sehingga satu kelompok MAC dapat dipakai ulang untuk menyelesaikan seluruh output *channel*. Strategi ini membuat jumlah DSP yang digunakan lebih rendah dibandingkan implementasi paralel penuh, walaupun membutuhkan jumlah siklus yang lebih banyak untuk menyelesaikan satu keluaran fitur. Pendekatan ini sejalan dengan konsep *folding* pada akselerator CNN FPGA, yaitu penggunaan resource komputasi yang lebih sedikit untuk menjalankan operasi yang lebih besar secara bergantian [30].

Selain MAC, bagian penting dari akselerator CNN adalah pembentukan window konvolusi. Karena operasi konvolusi 3×3 membutuhkan sekumpulan data piksel atau fitur yang berdekatan, maka digunakan *shift register* dan *line buffer* untuk membentuk window tersebut. Data input tidak dibaca ulang dari awal untuk setiap operasi konvolusi, tetapi digeser secara berurutan sehingga window baru dapat terbentuk dari data yang sudah tersimpan pada buffer baris sebelumnya. Penggunaan *shift register* dan *line buffer* ini membantu mengurangi kebutuhan akses memori dan membuat proses konvolusi lebih sesuai dengan aliran data streaming pada FPGA. Pendekatan serupa juga digunakan pada beberapa akselerator CNN berbasis FPGA, seperti FINN melalui *Sliding Window Unit* dan AoCStream melalui arsitektur *stream-based line-buffer* [30, 40].

Setiap blok CNN dikendalikan menggunakan *finite state machine* (FSM) [41]. FSM digunakan untuk mengatur urutan kerja pada masing-masing blok, seperti menunggu data masuk, menjalankan proses komputasi, mengumpulkan hasil perhitungan, dan mengirimkan data ke blok berikutnya. Pada blok yang menggunakan *time multiplexing*, FSM berperan penting karena satu unit komputasi digunakan secara bergantian untuk beberapa *channel*. Dengan demikian, FSM memastikan bahwa proses pengambilan data, pemilihan bobot, akumulasi hasil, dan pengiriman output dilakukan pada urutan yang benar [42].

Untuk menjaga kestabilan aliran data antarblok, rancangan akselerator menggunakan FIFO dan mekanisme *valid-ready*. FIFO digunakan sebagai penyangga antara blok yang memiliki kecepatan proses berbeda. Hal ini diperlukan karena tidak semua blok CNN menghasilkan data pada jumlah siklus yang sama. Misalnya, blok konvolusi yang menggunakan *time multiplexing* membutuhkan waktu lebih lama dibandingkan blok pooling atau blok komparator [43]. Dengan adanya FIFO, blok sebelumnya tetap dapat mengirim data selama FIFO belum penuh, sedangkan blok berikutnya dapat mengambil data ketika sudah siap. Mekanisme FIFO seperti ini umum digunakan pada desain hardware berbasis aliran data untuk memisahkan proses produksi dan konsumsi data [32]. Aliran data dengan mekanisme *valid-ready* ditunjukkan pada Gambar 2.5.

Mekanisme *valid-ready* digunakan sebagai bentuk *backpressure*. Sinyal *valid* menunjukkan bahwa data dari blok sebelumnya sudah tersedia, sedangkan sinyal *ready* menunjukkan bahwa blok berikutnya siap menerima data. Perpindahan data hanya terjadi

ketika kedua sinyal tersebut aktif secara bersamaan. Dengan cara ini, counter, *shift register*, FIFO, dan FSM hanya bergerak ketika data benar-benar berhasil diterima oleh blok berikutnya. Hal ini mencegah terjadinya kehilangan data atau pergeseran urutan fitur ketika salah satu blok sedang sibuk.

Data antarblok pada rancangan ini tetap menggunakan bus yang cukup lebar untuk menjaga hasil antara dari proses *fixed-point*. Namun, sebelum data aktivasi masuk ke operasi MAC pada layer berikutnya, lebar aktivasi dibatasi agar operasi perkalian antara aktivasi dan bobot 16-bit tetap efisien pada DSP. Dengan demikian, hasil akumulasi MAC tetap memiliki lebar bit yang cukup besar untuk menjaga ketelitian, tetapi operand utama yang masuk ke multiplier tetap berada pada ukuran yang sesuai dengan karakteristik DSP FPGA.

Secara keseluruhan, akselerator CNN dirancang sebagai rangkaian streaming yang terdiri dari beberapa blok utama, yaitu pembentuk window berbasis *shift register*, unit MAC berbasis DSP, FSM pengendali, FIFO antarblok, serta mekanisme *valid-ready*. Kombinasi ini membuat model CNN dapat dijalankan pada FPGA dengan menyesuaikan antara kebutuhan komputasi, keterbatasan resource, dan kestabilan aliran data. Penggunaan resource yang tinggi, terutama pada DSP, masih dapat diterima selama rancangan memenuhi timing dan menghasilkan keluaran inferensi yang sesuai dengan model acuan.

3.2.6 Rancangan Double Frame Memory

Rancangan *double frame memory* dibuat untuk menyediakan dua area penyimpanan citra yang dapat digunakan secara bergantian. Saat satu buffer digunakan oleh proses baca untuk memasok data ke CNN accelerator, buffer lainnya dapat diisi oleh data baru. Setelah proses selesai, peran kedua buffer ditukar sehingga aliran data berikutnya dapat diproses tanpa menunggu buffer yang sama kosong kembali.

Akses DDR dilakukan melalui MIG dan antarmuka AXI. Jalur tulis digunakan untuk menempatkan data masukan ke buffer aktif, sedangkan jalur baca mengambil kembali data untuk proses inferensi atau evaluasi hasil. Dengan memisahkan buffer aktif dan buffer cadangan, sistem dapat mengurangi risiko *data hazard* dan menjaga agar pengolahan citra tetap konsisten [33,44].

Pada tahap implementasi, data citra dari dataset disimpan ke salah satu buffer, kemudian dibaca oleh CNN accelerator sesuai urutan proses yang dirancang. Hasil inferensi dapat ditulis kembali ke buffer lain untuk keperluan pengamatan atau evaluasi. Dengan rancangan ini, alur sistem dapat dijelaskan sebagai data input, *double frame memory*, CNN accelerator, dan data output [11].

3.3 Metode Implementasi

Metode implementasi dilakukan secara bertahap agar setiap bagian sistem dapat diverifikasi sebelum digabungkan dengan blok lain. Tahapan dimulai dari model acuan di Python, dilanjutkan dengan kuantisasi parameter, translasi ke Verilog, implementasi DDR sebagai *double frame memory*. Urutan ini dipilih untuk memisahkan risiko kesalahan algoritmik dari risiko kesalahan integrasi perangkat keras [11,23].

3.3.1 Implementasi Model CNN di Python

Model CNN terlebih dahulu dibangun dan dilatih menggunakan Python. Tahap ini dilakukan karena Python menyediakan lingkungan yang lebih fleksibel untuk mengembangkan arsitektur jaringan, menjalankan proses pelatihan, mengevaluasi akurasi, serta mengamati keluaran pada setiap layer. Sebelum model dipindahkan ke FPGA, perilaku algoritmiknya harus dipastikan terlebih dahulu agar kesalahan yang muncul pada tahap hardware tidak berasal dari model yang belum tervalidasi.

Implementasi Python digunakan sebagai baseline software. Baseline ini menyediakan bobot, bias, keluaran antara, dan keluaran akhir yang menjadi referensi bagi implementasi fixed-point dan Verilog. Dengan adanya baseline, setiap perubahan nilai akibat kuantisasi, pembulatan, atau pembatasan lebar bit dapat dibandingkan secara langsung terhadap model acuan. Hal ini penting karena implementasi hardware tidak menyediakan fleksibilitas debugging sebesar Python, sehingga referensi numerik harus disiapkan sebelum proses translasi ke Verilog dilakukan.

Tahap ini juga menjadi dasar pemilihan arsitektur model yang sesuai dengan keterbatasan FPGA. Model tidak dirancang terlalu besar karena seluruh operasi inferensi harus dipetakan ke resource seperti DSP, LUT, FF, dan BRAM. Oleh karena itu, implementasi Python tidak hanya bertujuan memperoleh model dengan akurasi yang dapat diterima, tetapi juga memastikan bahwa ukuran model masih layak untuk direalisasikan sebagai akselerator perangkat keras.

3.3.2 Implementasi Kuantisasi Fixed-Point

Parameter hasil pelatihan dikonversi dari floating-point ke fixed-point 16-bit dengan format Q2.14. Format ini menggunakan dua bit untuk bagian integer termasuk bit tanda dan 14 bit untuk bagian fraksi. Tahap ini diperlukan karena operasi floating-point membutuhkan rangkaian aritmetika yang lebih kompleks dan dapat meningkatkan penggunaan resource FPGA. Sebaliknya, fixed-point lebih sesuai untuk FPGA karena operasi perkalian dan akumulasi dapat dipetakan secara lebih efisien ke DSP dan jalur data integer.

Kuantisasi dilakukan sebelum parameter dimasukkan ke file memori Verilog agar

format data yang digunakan pada simulasi hardware sama dengan format data yang akan disintesis pada FPGA. Setiap bobot dan bias dikalikan dengan faktor skala berdasarkan jumlah bit pecahan, dibulatkan ke integer terdekat, kemudian dibatasi agar tetap berada pada rentang 16-bit bertanda. Proses pembatasan ini penting untuk mencegah overflow pada penyimpanan parameter, sedangkan pemilihan 14 bit fraksi bertujuan menjaga resolusi nilai pecahan agar perubahan kecil pada bobot tidak hilang secara berlebihan. Alokasi dua bit integer dinilai cukup karena hasil parameter model tidak keluar dari rentang -2 sampai 2 , sehingga sisa bit dapat difokuskan untuk meningkatkan ketelitian bagian fraksi.

Validasi kuantisasi dilakukan dengan membandingkan rentang nilai, jumlah saturasi, dan kesesuaian keluaran terhadap model Python. Pemeriksaan saturasi diperlukan karena parameter yang tersaturasi dapat berubah jauh dari nilai aslinya dan berpotensi mengubah hasil inferensi. Dengan validasi ini, proses kuantisasi tidak hanya menjadi tahap konversi format, tetapi juga tahap pengendalian galat numerik sebelum model dijalankan pada Verilog.

3.3.3 Implementasi CNN ke Verilog

Implementasi Verilog merealisasikan operasi konvolusi, ReLU, pooling, fully connected, dan argmax sebagai rangkaian perangkat keras. Tahap ini dilakukan karena tujuan utama penelitian adalah menjalankan inferensi CNN sepenuhnya di FPGA, bukan hanya mensimulasikan model di software. Dengan Verilog, operasi CNN dapat dibentuk sebagai jalur data khusus yang memanfaatkan paralelisme, pipeline, dan resource hardware secara langsung.

Blok komputasi menggunakan kombinasi MAC berbasis DSP, shift register, FIFO, FSM, dan mekanisme valid-ready. MAC digunakan karena operasi dominan pada CNN adalah perkalian dan akumulasi antara aktivasi dan bobot. Shift register dan line buffer digunakan untuk membentuk window konvolusi tanpa membaca ulang data dari memori utama. FIFO dan valid-ready digunakan untuk menjaga aliran data antarblok yang memiliki waktu proses berbeda, sedangkan FSM mengatur urutan komputasi, pemilihan channel, pemilihan bobot, dan pengiriman keluaran.

Implementasi dilakukan secara *layer-by-layer* agar kesalahan dapat dilokalisasi. Setiap layer diuji terhadap keluaran Python sebelum dihubungkan ke layer berikutnya. Alasan pendekatan ini adalah galat pada layer awal dapat terakumulasi dan sulit ditelusuri jika seluruh jaringan langsung diuji sebagai satu blok penuh. Dengan pengujian bertahap, perbedaan akibat fixed-point, pemotongan lebar bit, atau kesalahan urutan data dapat dianalisis pada titik yang lebih spesifik.

3.3.4 Implementasi DDR sebagai Double Frame Memory

DDR diimplementasikan sebagai *double frame memory* melalui MIG dan transaksi AXI. Tahap ini diperlukan karena satu frame citra membutuhkan kapasitas penyimpanan yang lebih besar dibandingkan buffer kecil yang idealnya ditempatkan pada BRAM. BRAM lebih sesuai digunakan untuk FIFO, line buffer, dan data antara yang dekat dengan blok komputasi, sedangkan DDR menyediakan ruang yang lebih besar untuk menyimpan dua frame secara bergantian.

Jalur tulis DDR menerima data masukan untuk buffer aktif, sedangkan jalur baca menyediakan data untuk CNN accelerator atau evaluasi hasil. Pemisahan jalur tulis dan baca diperlukan karena proses tulis dan baca dapat berjalan pada waktu yang berbeda, sementara CNN hanya membutuhkan data citra yang sudah siap diproses. Ketika satu buffer sedang dibaca, buffer lainnya dapat diisi sehingga aliran data tetap berlanjut tanpa menghentikan proses.

Penggunaan MIG dan AXI dilakukan agar akses DDR dapat dikendalikan melalui antarmuka yang terstruktur. MIG menangani inisialisasi, kalibrasi, dan detail fisik DDR, sedangkan AXI digunakan untuk membentuk transaksi baca dan tulis. Dengan pendekatan ini, rancangan logika dapat difokuskan pada pengaturan alamat, validitas data, dan sinkronisasi perpindahan buffer. Implementasi DDR sebagai *double frame memory* menjadi tahap penting karena sistem membutuhkan sumber data yang stabil dan dapat diakses kembali sesuai urutan pengolahan.

3.4 Metode Integrasi

3.4.1 Integrasi Model CNN ke FPGA

Integrasi model CNN dilakukan dengan memasukkan parameter fixed-point ke memori parameter Verilog dan menghubungkan blok-blok layer sesuai urutan arsitektur. Keluaran tiap tahap dibandingkan dengan baseline Python agar kesalahan dapat dilokalisasi sebelum sistem dihubungkan dengan memori DDR.

3.4.2 Integrasi DDR sebagai Double Frame Memory

Integrasi DDR sebagai *double frame memory* bertujuan memastikan data citra dapat ditulis ke buffer aktif secara berurutan dan dibaca kembali dari buffer lain tanpa saling mengganggu. Tahap ini memvalidasi validitas alamat, sinkronisasi perpindahan buffer, dan kestabilan aliran data tulis-baca.

3.4.3 Integrasi Double Frame Memory dan CNN

Integrasi penuh menggabungkan *double frame memory* dengan CNN accelerator. Pada tahap ini, sistem diuji sebagai alur end-to-end sehingga data citra dapat disimpan,

dipilih dari buffer yang aktif, diproses oleh CNN, dan hasilnya digunakan sebagai keluaran sistem.

3.5 Metode Pengujian

3.5.1 Pengujian Model CNN di Python

Pengujian model CNN di Python dilakukan untuk memperoleh baseline sebelum model diimplementasikan ke dalam Verilog. Pengujian ini dilakukan terhadap data latih dan data uji untuk melihat kemampuan model dalam mengenali citra digit pada dataset MNIST. Hasil pengujian Python digunakan sebagai acuan utama karena model pada tahap ini masih menggunakan representasi numerik yang lebih fleksibel dibandingkan implementasi fixed-point pada hardware.

Pada tahap ini, setiap citra masukan diproses oleh model CNN untuk menghasilkan kelas prediksi dari sepuluh kemungkinan digit. Data latih digunakan untuk memastikan bahwa model telah mampu mempelajari pola dari dataset yang digunakan selama pelatihan, sedangkan data uji digunakan untuk melihat kemampuan model terhadap data yang tidak digunakan secara langsung dalam proses optimasi bobot.

Keluaran dari pengujian ini berupa label prediksi untuk setiap citra uji, nilai akurasi baseline, serta informasi kesalahan klasifikasi yang nantinya digunakan sebagai pembandingan terhadap hasil implementasi fixed-point pada Verilog dan FPGA.

3.5.2 Pengujian Perbedaan Hasil Python dan Verilog

Pengujian ini dilakukan untuk memperoleh keluaran antara dari implementasi Python dan Verilog pada titik yang sama di dalam arsitektur CNN. Data yang digunakan adalah data latih untuk seluruh label digit, dengan 100 sampel pada setiap label. Susunan data tersebut digunakan agar pengujian tidak hanya mewakili satu kelas tertentu, tetapi mencakup respons model terhadap seluruh kelas pada dataset.

Urutan pengujian dimulai dengan menjalankan model acuan di Python untuk menghasilkan keluaran setiap layer dan blok. Setelah itu, data masukan dan parameter fixed-point yang sama diberikan ke testbench Verilog. Simulasi Verilog dijalankan menggunakan Icarus Verilog secara bertahap pada blok konvolusi, pooling, dan dense, sehingga keluaran tiap tahap dapat diamati sebelum dilanjutkan ke tahap berikutnya.

Output yang dikumpulkan dari pengujian ini adalah nilai setiap *feature* pada setiap layer, baik berupa elemen pada *feature map* maupun keluaran neuron pada layer dense. Selain itu, hasil akhir argmax juga dikumpulkan untuk melihat apakah perbedaan nilai antara Python dan Verilog berpengaruh terhadap keputusan kelas. Dengan alur ini, pengujian menghasilkan data mentah yang dapat digunakan pada tahap evaluasi perbedaan bit dan evaluasi akurasi hasil inferensi.

3.5.3 Pengujian Penggunaan Kapasitas FPGA

Pengujian kapasitas FPGA dilakukan dari laporan implementasi Vivado. Parameter yang diamati meliputi LUT, FF, BRAM, dan DSP karena keempat resource tersebut menentukan kelayakan integrasi pada Arty A7-100T. Pengujian ini tidak hanya digunakan untuk mengetahui persentase penggunaan resource setelah desain selesai, tetapi juga menjadi umpan balik terhadap strategi implementasi yang dipakai selama proses perancangan.

Pada CNN accelerator, hasil penggunaan resource memengaruhi keputusan pemotongan bit pada keluaran MAC. Hasil akumulasi MAC memiliki lebar bit yang lebih besar dibandingkan operand masukan karena merupakan hasil perkalian dan penjumlahan berulang. Jika seluruh lebar hasil akumulasi dipertahankan sampai layer berikutnya, kebutuhan register, jalur data, FIFO, dan logika kontrol akan meningkat. Oleh karena itu, pengujian resource digunakan untuk menentukan apakah lebar bit hasil antara masih dapat dipertahankan atau perlu dipotong pada titik tertentu agar desain tetap muat tanpa menghilangkan ketelitian yang dibutuhkan secara berlebihan.

Pengujian kapasitas juga digunakan untuk mengevaluasi kedalaman FIFO. FIFO diperlukan untuk menahan perbedaan laju data antarblok, tetapi kedalaman FIFO yang terlalu besar akan meningkatkan penggunaan BRAM atau LUT. Sebaliknya, FIFO yang terlalu dangkal dapat menyebabkan aliran data mudah tertahan ketika blok berikutnya belum siap menerima data. Oleh karena itu, kedalaman FIFO ditentukan dengan mempertimbangkan kebutuhan buffering dan batas resource yang tersedia pada FPGA.

Selain pada CNN accelerator, pengujian resource memengaruhi implementasi akses memori dan perpindahan buffer. Proses pemindahan citra ke buffer berukuran 28×28 piksel harus dibuat ringan agar tidak menambah beban komputasi dan memori yang besar. Dengan demikian, pengujian kapasitas FPGA menjadi dasar untuk menyeimbangkan kebutuhan akurasi numerik, kestabilan aliran data, dan kelayakan integrasi seluruh subsistem pada satu board FPGA.

3.5.4 Pengujian Double Frame Memory

Pengujian *double frame memory* dilakukan untuk memastikan transaksi baca dan tulis melalui MIG dan AXI berjalan setelah proses kalibrasi selesai. Validasi dilakukan terhadap kestabilan akses alamat, perpindahan buffer, dan kesesuaian data yang ditulis dan dibaca kembali.

3.5.5 Pengujian Sistem Terintegrasi

Pengujian sistem terintegrasi dilakukan setelah blok CNN dan *double frame memory* lulus pengujian dasar. Pendekatan bertahap ini digunakan agar kesalahan integrasi

dapat ditelusuri berdasarkan blok yang sudah tervalidasi sebelumnya.

3.6 Metode Evaluasi

3.6.1 Evaluasi Akurasi

Evaluasi akurasi digunakan untuk menilai kesesuaian hasil klasifikasi terhadap label data uji dan sebagai pembandingan antara baseline software dan implementasi hardware. Pada penelitian ini, evaluasi akurasi tidak hanya dinyatakan dalam bentuk nilai akurasi keseluruhan, tetapi juga dianalisis menggunakan *confusion matrix*.

Confusion matrix digunakan karena mampu menunjukkan hubungan antara label sebenarnya dan label hasil prediksi untuk setiap kelas digit. Nilai pada diagonal utama menunjukkan jumlah data yang diklasifikasikan dengan benar, sedangkan nilai di luar diagonal menunjukkan kesalahan klasifikasi antar kelas. Dengan cara ini, evaluasi tidak hanya menjawab apakah model benar atau salah secara umum, tetapi juga menunjukkan kelas digit mana yang lebih sering tertukar dengan kelas lain.

Penggunaan *confusion matrix* penting pada penelitian ini karena model CNN diimplementasikan kembali dari Python ke Verilog dengan representasi fixed-point. Jika terjadi penurunan akurasi atau perubahan hasil prediksi, *confusion matrix* dapat membantu melihat apakah perubahan tersebut terjadi merata pada seluruh kelas atau hanya dominan pada kelas tertentu. Analisis ini digunakan untuk menilai apakah hasil inferensi hardware masih konsisten terhadap baseline model Python.

3.6.2 Evaluasi Perbedaan Bit

Evaluasi perbedaan bit digunakan untuk menilai data hasil pengujian Python dan Verilog secara kuantitatif. Metrik yang dihitung meliputi selisih integer fixed-point per *feature*, galat nilai real *float64*, rata-rata galat absolut, dan galat maksimum pada setiap layer atau blok CNN. Jika q_i^{py} adalah keluaran fixed-point dari Python dan q_i^v adalah keluaran fixed-point dari Verilog pada *feature* ke- i , maka perbedaan bit dihitung sebagai berikut.

$$d_i^{bit} = |q_i^v - q_i^{py}| \quad (3-3)$$

Karena implementasi menggunakan fixed-point 16-bit Q2.14, nilai yang terbaca perlu dikembalikan terlebih dahulu ke bilangan bertanda. Jika nilai bit 16 terbaca sebagai b_{16} , konversi ke signed integer dan *float64* dilakukan dengan persamaan berikut.

$$s_{16} = \begin{cases} b_{16}, & b_{16} < 2^{15} \\ b_{16} - 2^{16}, & b_{16} \geq 2^{15} \end{cases} \quad (3-4)$$

$$x_{float64} = \frac{s_{16}}{2^{14}} \quad (3-5)$$

Dengan cara ini, hasil Verilog dan Python dibaca pada skala numerik yang sama, sehingga perbedaan fixed-point dapat diterjemahkan menjadi perbedaan nilai aktivasi.

Setelah nilai Python dan Verilog berada pada skala `float64`, galat pada feature ke- i dihitung sebagai $e_i = x_i^v - x_i^{py}$. Untuk setiap layer dengan N feature, rata-rata galat absolut dan galat maksimum dihitung menggunakan persamaan berikut.

$$MAE = \frac{1}{N} \sum_{i=1}^N |e_i| \quad (3-6)$$

$$E_{max} = \max_{1 \leq i \leq N} |e_i| \quad (3-7)$$

Hasil evaluasi dinilai dengan melihat apakah perbedaan bit dan galat `float64` masih kecil pada setiap layer, tidak meningkat secara tidak wajar pada layer berikutnya, dan tidak mengubah hasil argmax. Dengan demikian, evaluasi tidak hanya melihat apakah terdapat perbedaan numerik, tetapi juga apakah perbedaan tersebut signifikan terhadap alur inferensi CNN.

Potongan kode yang digunakan untuk proses evaluasi tersebut ditunjukkan pada Kode L.3. Fungsi `sign_extend` digunakan untuk mengubah data unsigned two's complement menjadi signed integer sesuai lebar bit tertentu, sedangkan fungsi `python_float_to_fixed` digunakan untuk mengubah keluaran *floating-point* Python menjadi integer fixed-point. Mode pembulatan digunakan agar proses konversi mengikuti prinsip kuantisasi parameter yang digunakan pada implementasi model.

```

1 def sign_extend(arr, bit_width):
2     """
3     Untuk mengubah unsigned two's complement menjadi signed
4     integer.
5     Contoh kalau ACT_SIZE = 24, maka bit_width = 24.
6     """
7     arr = arr.astype(np.int64)
8
9     if bit_width is None:
10         return arr
11
12     sign_bit = 1 << (bit_width - 1)
13     full_range = 1 << bit_width
14
15     return np.where(arr >= sign_bit, arr - full_range, arr)
16
17 def python_float_to_fixed(arr, frac_bits=14, mode="round"):
18     """

```

```

19     Convert float Python ke fixed-point integer .
20     """
21     scale = 1 << frac_bits
22     scaled = arr * scale
23
24     if mode == "round":
25         return np.round(scaled).astype(np.int64)
26
27     elif mode == "trunc":
28         return np.trunc(scaled).astype(np.int64)
29
30     elif mode == "floor":
31         return np.floor(scaled).astype(np.int64)
32
33     else:
34         raise ValueError("mode harus 'round', 'trunc', atau 'floor'")
35

```

Kode evaluasi perbedaan bit Python dan Verilog

3.6.3 Evaluasi Latensi Inferensi

Evaluasi latensi inferensi dihitung berdasarkan jumlah siklus clock yang diperlukan sejak data masukan mulai diproses oleh blok CNN sampai keluaran klasifikasi tersedia. Pengukuran ini dilakukan karena sistem tidak hanya harus menghasilkan prediksi yang benar, tetapi juga harus mampu menghasilkan keluaran dalam batas waktu yang sesuai dengan kebutuhan *real-time*. Pada implementasi sinkron di FPGA, waktu eksekusi lebih tepat dinyatakan berdasarkan jumlah siklus clock karena setiap tahap komputasi, pembacaan FIFO, operasi MAC, dan propagasi antarblok dikendalikan oleh sinyal clock.

Latensi inferensi dihitung menggunakan Persamaan 3-8. Nilai N_{cycle} menyatakan jumlah siklus clock yang dibutuhkan untuk menyelesaikan satu proses inferensi, sedangkan f_{clk} menyatakan frekuensi kerja clock sistem dalam satuan Hz.

$$T_{latency} = \frac{N_{cycle}}{f_{clk}} \quad (3-8)$$

Jika frekuensi clock dinyatakan dalam MHz, maka latensi dalam mikrodetik dapat dihitung menggunakan Persamaan 3-9. Persamaan ini digunakan untuk memudahkan interpretasi hasil pengujian karena nilai latensi inferensi pada FPGA umumnya berada pada orde mikrodetik hingga milidetik.

$$T_{latency}(\mu s) = \frac{N_{cycle}}{f_{clk}(\text{MHz})} \quad (3-9)$$

Throughput dihitung sebagai jumlah inferensi yang dapat diselesaikan dalam satu detik. Untuk satu jalur inferensi tanpa pemrosesan paralel antar citra, throughput dapat diperoleh dari kebalikan latensi seperti ditunjukkan pada Persamaan 3-10. Dengan demikian, semakin kecil jumlah siklus inferensi atau semakin tinggi frekuensi clock yang digunakan, semakin besar throughput yang dapat dicapai oleh akselerator.

$$Throughput = \frac{1}{T_{latency}} = \frac{f_{clk}}{N_{cycle}} \quad (3-10)$$

Apabila pengujian dilakukan untuk beberapa citra uji, throughput rata-rata dihitung menggunakan Persamaan 3-11. Nilai N_{image} menyatakan jumlah citra yang diuji, sedangkan T_{total} menyatakan total waktu yang dibutuhkan untuk menyelesaikan seluruh proses inferensi.

$$Throughput_{avg} = \frac{N_{image}}{T_{total}} \quad (3-11)$$

Hasil latensi dan throughput kemudian digunakan untuk menilai apakah strategi implementasi yang dipilih, seperti pemotongan bit hasil MAC, penggunaan FIFO, dan algoritma prapemrosesan citra, masih memenuhi kebutuhan kecepatan sistem secara keseluruhan.

3.6.4 Evaluasi Penggunaan Resource FPGA

Evaluasi resource FPGA menilai penggunaan LUT, FF, BRAM, dan DSP pada CNN accelerator maupun sistem terintegrasi. Hasilnya digunakan untuk menganalisis trade-off antara paralelisme, time multiplexing, dan kelayakan implementasi.

3.6.5 Evaluasi Daya

Evaluasi daya menggunakan hasil estimasi implementasi untuk menilai kesesuaian sistem dengan motivasi embedded vision yang membutuhkan konsumsi daya rendah.

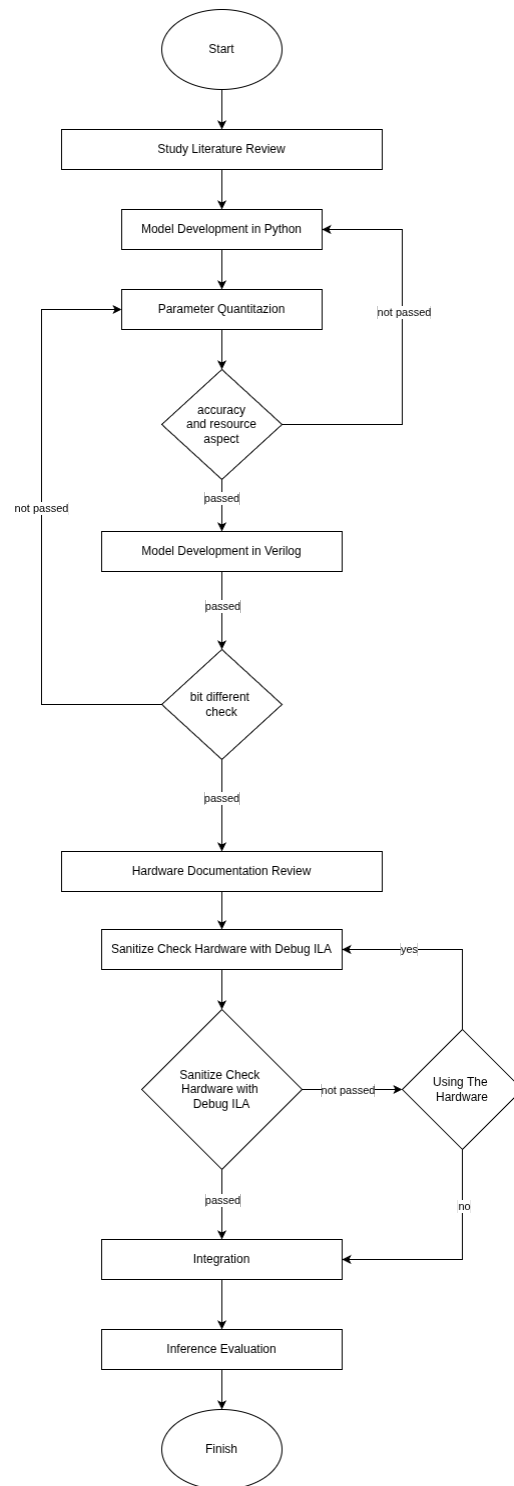
3.6.6 Evaluasi Tampilan Real Time

Evaluasi jalur memori dilakukan dengan melihat apakah *double frame memory* dan CNN mampu bekerja stabil ketika sistem dijalankan. Evaluasi ini melengkapi pengujian CNN karena sistem akhir tidak hanya berupa akselerator, tetapi sistem pemrosesan citra berbasis memori berganda.

3.7 Tahapan Pelaksanaan

Tahapan pelaksanaan penelitian ini ditunjukkan pada Gambar 3.7. Penelitian diawali dengan studi literatur untuk memahami konsep implementasi CNN pada FPGA, teknik kuantisasi *fixed-point*, serta sistem pendukung seperti memori DDR sebagai *double frame memory*. Setelah itu, model CNN dikembangkan terlebih dahulu menggunakan Python sebagai model acuan. Model yang telah diperoleh kemudian dikonversi ke representasi *fixed-point* 16-bit agar lebih sesuai dengan karakteristik komputasi FPGA. Tahap kuantisasi ini divalidasi berdasarkan dua aspek, yaitu akurasi model dan kebutuhan *resource* FPGA, karena implementasi CNN pada FPGA perlu mempertimbangkan keseimbangan antara performa komputasi, penggunaan sumber daya, dan ketelitian numerik [11, 23].

Setelah model hasil kuantisasi dinilai memenuhi kebutuhan, arsitektur CNN diimplementasikan ke dalam Verilog. Hasil implementasi Verilog kemudian dibandingkan dengan hasil model Python melalui pengujian *bit-difference* untuk memastikan bahwa proses komputasi pada hardware tetap mengikuti model acuan. Tahap berikutnya adalah pengujian dasar hardware menggunakan *debug* ILA untuk memastikan blok FIFO, DDR, dan CNN dapat bekerja dengan benar sebelum dilakukan integrasi sistem. Setelah seluruh blok tervalidasi, sistem diintegrasikan menjadi alur lengkap mulai dari penyimpanan citra, pemrosesan CNN, hingga evaluasi hasil inferensi.



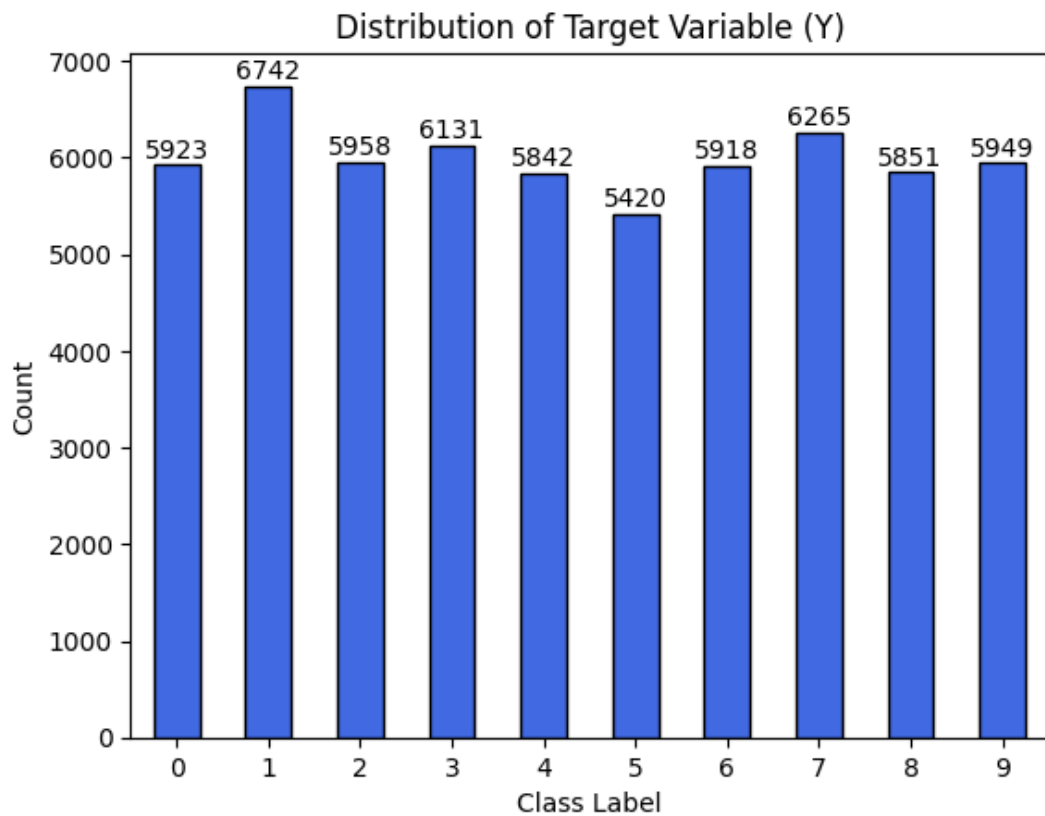
Gambar 3.7. Diagram Alir Penelitian

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Implementasi Model CNN

4.1.1 Dataset yang Digunakan

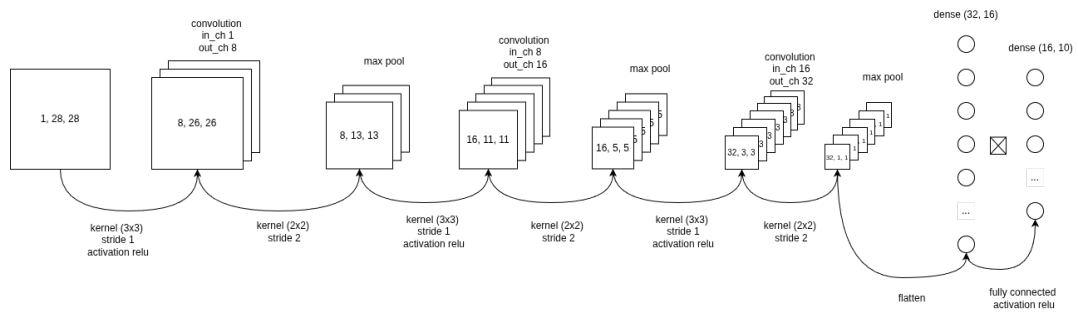


Gambar 4.8. Distribusi *Training Dataset* yang digunakan

Gambar 4.8 menunjukkan distribusi data latih yang digunakan pada proses pelatihan model CNN. Distribusi kelas yang relatif seimbang penting karena model tidak diarahkan untuk lebih sering melihat satu kelas tertentu dibandingkan kelas lain.

Keseimbangan distribusi tersebut membantu mengurangi risiko bias klasifikasi terhadap digit tertentu. Dalam konteks implementasi pada FPGA, tahap ini tetap penting karena kualitas model hardware ditentukan terlebih dahulu oleh model software yang menjadi sumber bobot dan bias.

4.1.2 Arsitektur Target Model CNN

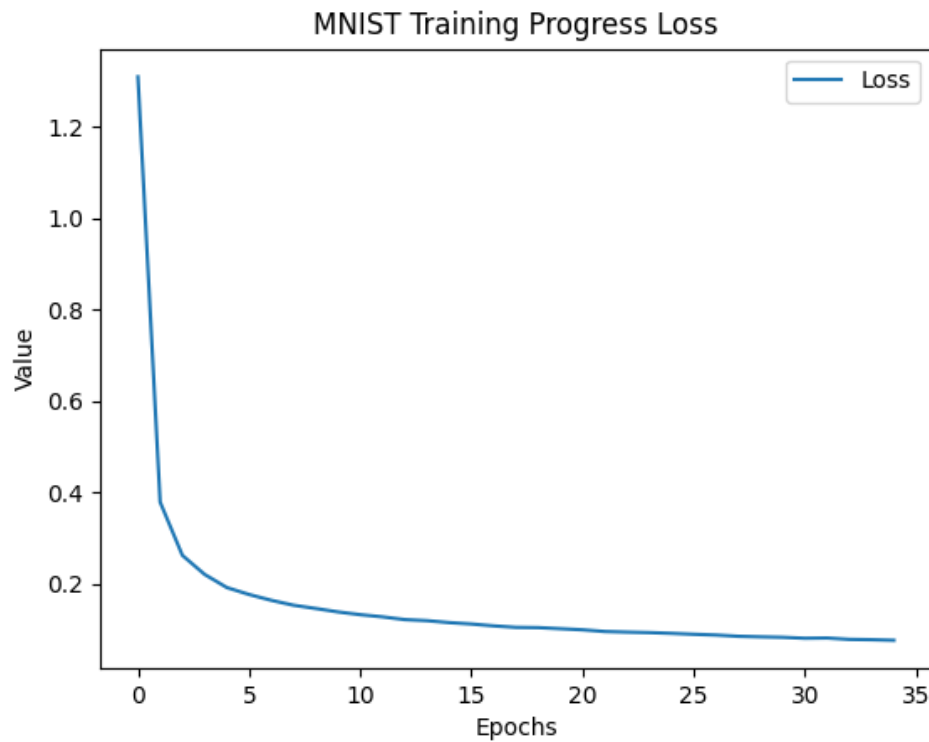


Gambar 4.9. Arsitektur Model CNN

Gambar 4.9 memperlihatkan arsitektur CNN yang digunakan sebagai target implementasi. Model terdiri dari konfigurasi layer konvolusi dan neural network dengan aktivasi ReLU. Layer konvolusi digunakan untuk ekstraksi fitur citra, sedangkan pooling yang digunakan adalah max pool untuk mereduksi dimensi feature map.

Hasil ekstraksi fitur dari blok konvolusi dan max pool kemudian diproses oleh neural network untuk melakukan pemetaan ke sepuluh label keluaran, yaitu digit 0 sampai 9. Pemilihan arsitektur yang relatif kecil merupakan keputusan engineering yang sesuai dengan keterbatasan resource FPGA. Model tetap mempertahankan kemampuan ekstraksi fitur bertingkat, tetapi jumlah parameter dan operasi dijaga agar masih layak dipetakan ke DSP, BRAM, LUT, dan FF pada Arty A7-100T.

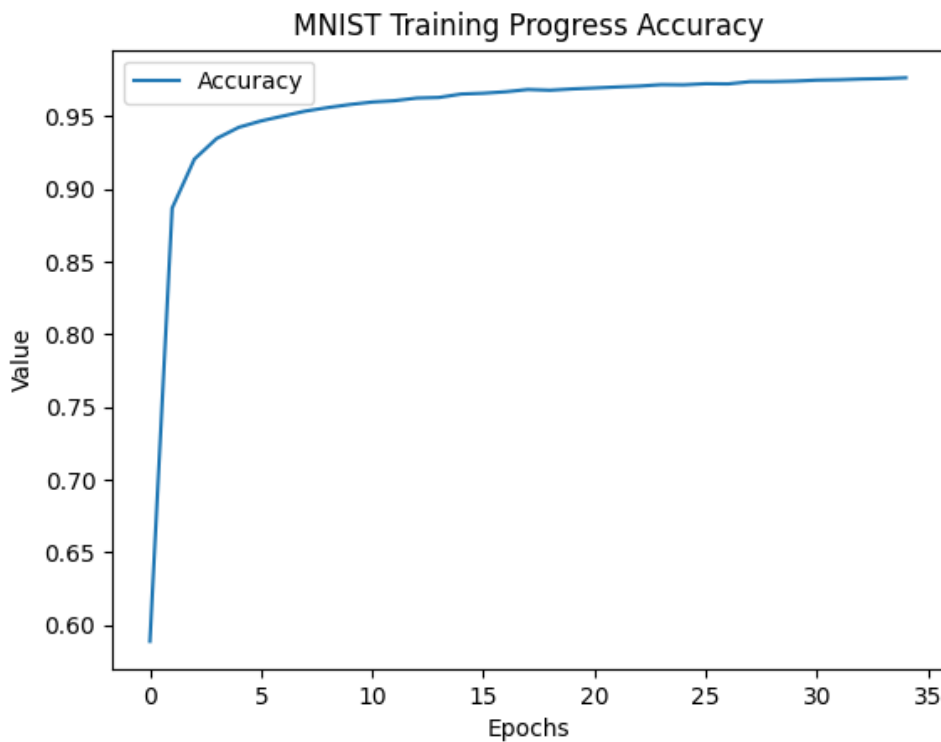
4.1.3 Hasil Pelatihan Model CNN



Gambar 4.10. *Loss training* model

Gambar 4.10 menunjukkan perubahan nilai loss selama proses pelatihan. Tren penurunan loss mengindikasikan bahwa proses optimasi berjalan menuju kondisi yang lebih stabil dan tidak menunjukkan pola divergensi pada kurva yang ditampilkan. Pada akhir pelatihan, nilai loss yang diperoleh adalah 0,01.

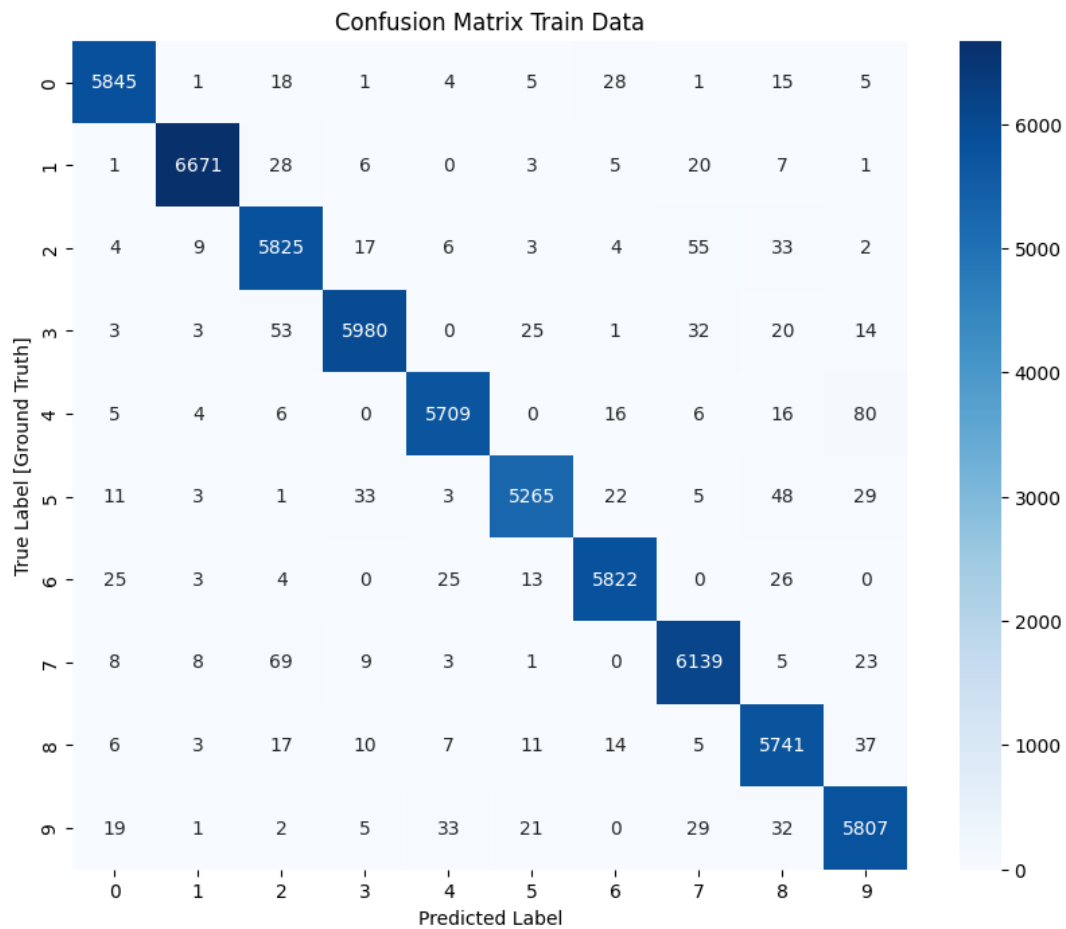
Penurunan loss menjadi dasar bahwa parameter yang dihasilkan dapat digunakan sebagai baseline sebelum dipindahkan ke fixed-point. Dengan demikian, evaluasi hardware tidak dimulai dari model yang belum konvergen, melainkan dari model software yang telah memiliki perilaku pelatihan yang teramati.



Gambar 4.11. *Accuracy training model*

Gambar 4.11 memperlihatkan perubahan akurasi selama pelatihan. Peningkatan akurasi yang mengikuti penurunan loss menunjukkan bahwa model semakin mampu memetakan fitur citra ke kelas digit yang benar. Akurasi akhir model pada data latih mencapai 97.23%.

Stabilitas akurasi pada akhir pelatihan mengindikasikan bahwa model telah konvergen dan dapat digunakan sebagai acuan implementasi hardware. Kemampuan generalisasi kemudian dievaluasi pada data uji sebelum keluaran Verilog dibandingkan terhadap keluaran Python.



Gambar 4.12. *Confusion Matrix Training model*

Gambar 4.12 menampilkan confusion matrix pada data latih. Secara visual, dominasi nilai pada diagonal utama menunjukkan bahwa sebagian besar sampel latih diklasifikasikan pada kelas yang sesuai.

Kesalahan klasifikasi yang muncul pada posisi di luar diagonal dapat dikaitkan dengan kemiripan bentuk antar digit tertentu pada dataset MNIST. Observasi ini penting karena kesalahan model software akan ikut menjadi batas awal bagi hasil implementasi hardware, sehingga perbandingan Python dan Verilog perlu dibaca sebagai kesesuaian terhadap baseline, bukan sebagai pelatihan ulang model.

4.1.4 Hasil Evaluasi Model CNN di Python

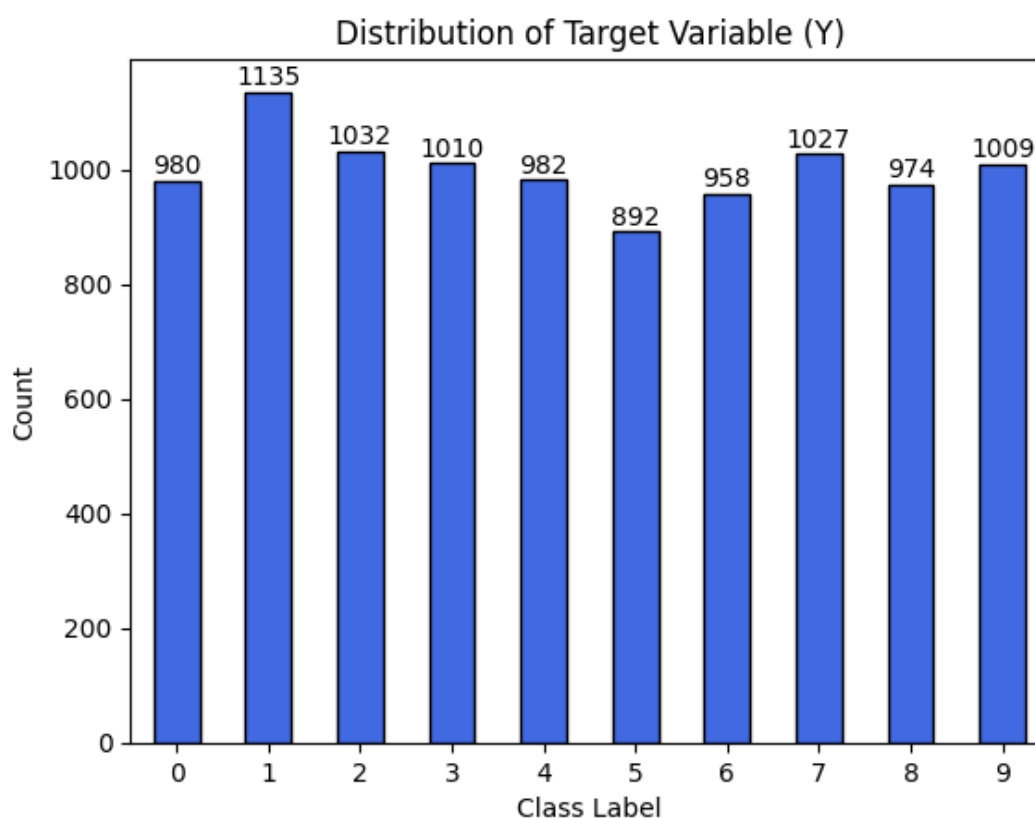
Tabel ukuran parameter menunjukkan bahwa model memiliki total parameter yang kecil untuk ukuran CNN. Jumlah parameter yang terbatas membuat kebutuhan memori parameter lebih ringan dan lebih sesuai untuk implementasi pada FPGA.

Implikasinya, penyimpanan bobot dan bias dapat dikelola sebagai data konstan atau memori internal tanpa menuntut bandwidth eksternal yang besar. Hal ini mendukung tujuan penelitian yang tidak hanya mengejar akurasi, tetapi juga kelayakan integrasi pada

Tabel 4.1. Ukuran Parameter Pada Model Python

Parameter Name	Activation	Params	Parameter Size
conv2d	relu	80	992 B
pool	-	-	-
conv2d	relu	1168	9.47 KB
pool	-	-	-
conv2d	relu	4640	36.59 KB
pool	-	-	-
flatten	-	-	-
neural	relu	528	4.47 KB
neural	relu	170	1.67 KB
softmax	-	-	-
Total		6586	53,192 KB

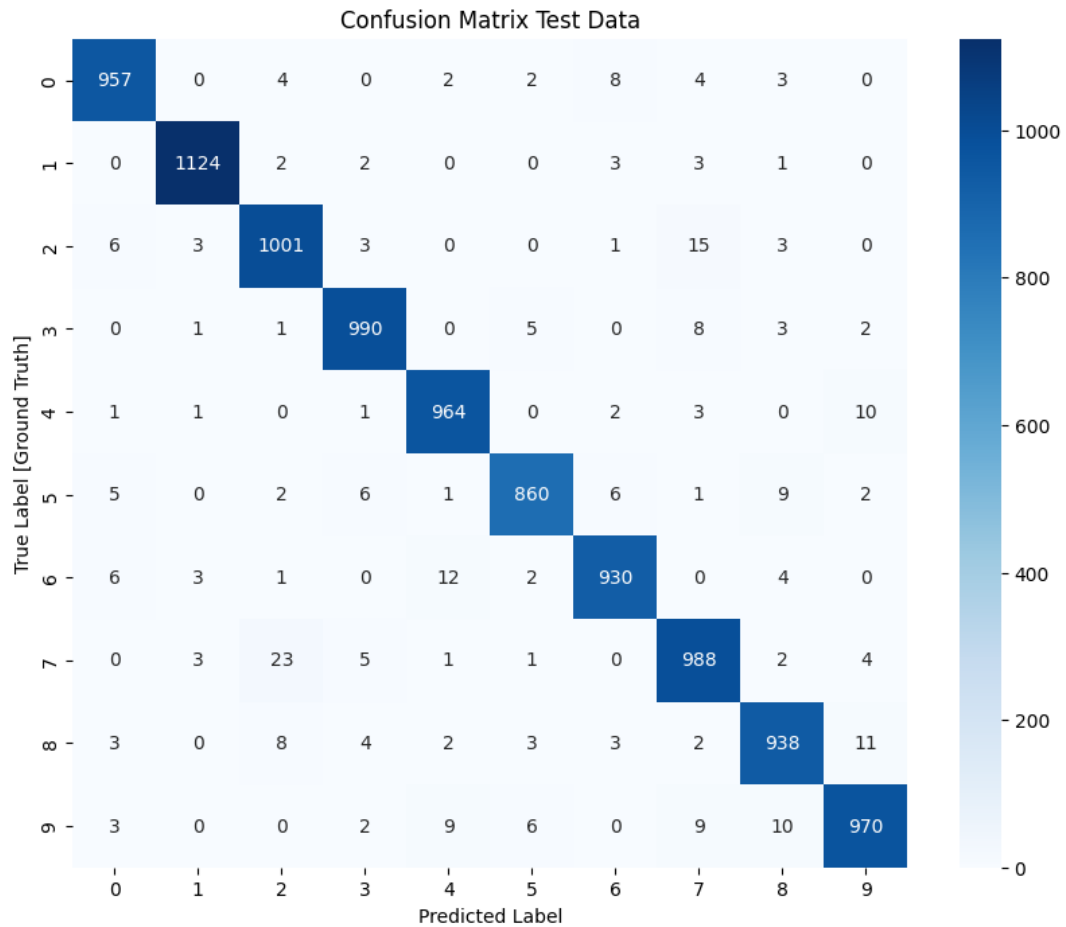
platform embedded.



Gambar 4.13. Distribusi *Test Dataset* yang digunakan

Gambar 4.13 menunjukkan distribusi data uji yang digunakan untuk mengevaluasi model. Distribusi uji yang seimbang membantu memastikan bahwa evaluasi tidak didominasi oleh satu kelas digit tertentu.

Data uji berperan sebagai baseline software sebelum model dipindahkan ke hardware. Dengan baseline ini, hasil fixed-point dan Verilog dapat dibandingkan terhadap perilaku model yang sama, bukan terhadap skenario pengujian yang berbeda.



Gambar 4.14. *Confusion Matrix testing model*

Gambar 4.14 memperlihatkan confusion matrix pada data uji. Pola diagonal yang dominan menunjukkan bahwa model mampu mengenali sebagian besar kelas dengan benar pada data yang tidak digunakan langsung sebagai data latih. Akurasi Python pada data uji adalah 97,2%.

Kesalahan di luar diagonal perlu dibaca sebagai konsekuensi dari kemiripan visual antar digit, variasi tulisan tangan, dan kapasitas model yang memang dibuat kecil agar sesuai dengan FPGA. Oleh karena itu, hasil evaluasi Python menjadi acuan realistis untuk menilai apakah implementasi hardware mempertahankan perilaku model.

4.1.5 Kuantisasi Parameter Model CNN

Tabel parameter model pada Python menampilkan bentuk tensor, rentang nilai floating-point, dan jumlah parameter pada setiap layer. Rentang nilai tersebut menja-

Tabel 4.2. Parameter Model pada Python

Parameter Name	Parameter Shape	Float Min	Float Max	Total Param
conv1_w	(8, 1, 3, 3)	-0.41649	0.22918	72
conv1_b	(8,)	-0.12903	0.20184	8
conv2_w	(16, 8, 3, 3)	-0.4300774	0.27402872	1152
conv2_b	(16,)	-0.14580758	0.16609082	16
conv3_w	(32, 16, 3, 3)	-0.27862	0.29362	4608
conv3_b	(32,)	-0.11603	0.10558	32
fc1_w	(16, 32)	-0.58693	0.56456	512
fc1_b	(16,)	-0.00994	0.01384	16
fc2_w	(10, 16)	-0.58495	0.81792	160
fc2_b	(10,)	-0.07052	0.13608	10

di dasar untuk menentukan apakah format fixed-point 16-bit mampu merepresentasikan bobot dan bias tanpa kehilangan informasi yang berlebihan.

Pada penelitian ini, format kuantisasi yang digunakan adalah Q2.14, yaitu 16 bit bertanda dengan dua bit untuk bagian integer termasuk bit tanda dan 14 bit untuk bagian fraksi. Format ini dipilih karena nilai parameter model berada pada rentang kecil dan tidak keluar dari rentang -2 sampai 2 . Dengan demikian, dua bit integer sudah cukup untuk menampung rentang nilai parameter, sedangkan 14 bit fraksi digunakan untuk mempertahankan resolusi pecahan.

Nilai parameter yang berada pada rentang relatif kecil mendukung penggunaan fractional bit $F = 14$, karena resolusi pecahan menjadi penting untuk mempertahankan perubahan kecil pada bobot. Informasi ini juga menjelaskan mengapa kuantisasi perlu dievaluasi per layer sebelum parameter dimasukkan ke implementasi Verilog.

4.2 Hasil Implementasi CNN Accelerator pada FPGA

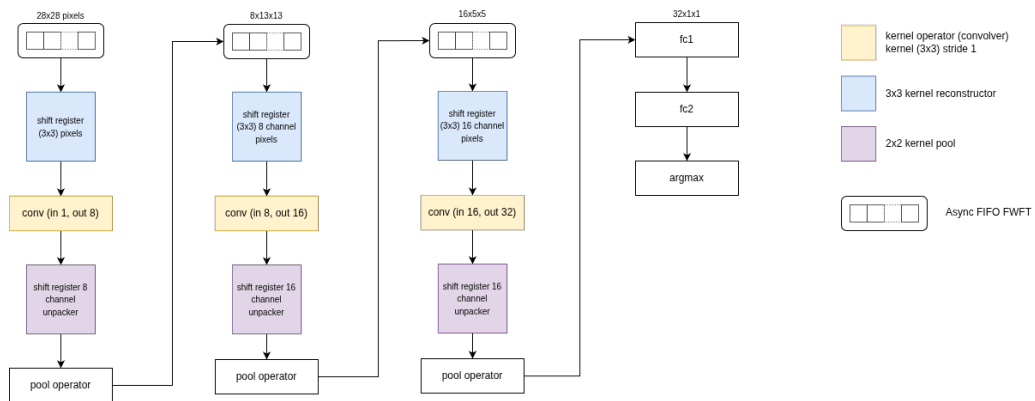
Tabel 4.3 menunjukkan bahwa seluruh parameter hasil kuantisasi memiliki nilai *Low Sat.* dan *High Sat.* sebesar nol, sehingga tidak ada bobot maupun bias yang terpotong ke batas bawah atau batas atas representasi Q2.14. Saturasi pada kuantisasi terjadi ketika nilai real berada di luar rentang representasi dan harus di-clip ke nilai minimum atau maksimum, sebagaimana prinsip *clipping* pada kuantisasi integer yang dibahas oleh Jacob et al. serta pentingnya pengendalian efek presisi terbatas pada representasi fixed-point yang dijelaskan oleh Gupta et al. [23, 24]. Dengan tidak ditemukannya saturasi, format fixed-point 16-bit Q2.14 yang menggunakan dua bit integer termasuk bit tanda dan 14 bit fraksi dapat dinyatakan cukup untuk menampung rentang parameter model yang berada dalam interval -2 sampai 2 . Implikasinya, alokasi bit dapat lebih diprioritaskan untuk bagian fraksi agar resolusi parameter tetap halus tanpa mengorbankan

Tabel 4.3. Hasil Kuantisasi Parameter

Parameter Name	Mem Row (Lines)	Low Sat.	High Sat.	Quant Min	Quant Max
conv1_w	72	0	0	-6824	3755
conv1_b	8	0	0	-2114	3307
conv2_w	1152	0	0	-7046	4490
conv2_b	16	0	0	-2389	2721
conv3_w	4608	0	0	-4565	4811
conv3_b	32	0	0	-1901	1730
fc1_w	512	0	0	-9616	9250
fc1_b	16	0	0	-163	227
fc2_w	160	0	0	-9584	13401
fc2_b	10	0	0	-1155	2230

rentang integer yang sebenarnya tidak dibutuhkan oleh bobot dan bias pada model ini.

4.2.1 Arsitektur CNN Accelerator pada FPGA



Gambar 4.15. Desain CNN Accelerator pada FPGA

Gambar 4.15 menunjukkan rancangan CNN accelerator pada FPGA. Diagram tersebut merepresentasikan pemetaan alur CNN ke blok perangkat keras, mulai dari pembentukan fitur, operasi MAC, aktivasi, pooling, hingga keluaran klasifikasi.

Secara arsitektural, desain ini menunjukkan bahwa CNN tidak dijalankan sebagai instruksi software, tetapi sebagai jalur data khusus. Pendekatan tersebut sesuai dengan karakter FPGA yang memungkinkan pipeline, pemanfaatan DSP, dan pengendalian aliran data melalui FSM serta valid-ready.

4.2.2 Hasil Penggunaan Resource CNN Accelerator

Tabel 4.4. Hasil *CNN Implement Running*

Komponen	Resource Terpakai	Persentase %
LUT	13.742	22.11
Flip-Flop	27.135	21.4
Block RAM	18	13.33
DSP Slice	232 DSP48E1	97.91

Tabel 4.4 menunjukkan bahwa penggunaan resource CNN accelerator masih dapat ditempatkan pada Arty A7-100T, tetapi dengan tekanan paling besar pada DSP. Dari tabel tersebut, LUT yang digunakan adalah 13.742 atau 22,11%, Flip-Flop 27.135 atau 21,4%, dan Block RAM 18 atau 13,33%. Ketiga resource ini masih memiliki ruang yang relatif besar, sehingga kebutuhan logika kontrol, register pipeline, dan penyimpanan data antara tidak menjadi batas utama pada rancangan CNN accelerator.

Resource yang paling dominan adalah DSP Slice, yaitu 232 DSP48E1 atau 97,91% dari kapasitas yang tersedia. Kondisi ini sesuai dengan karakter CNN accelerator karena operasi utama pada layer konvolusi dan dense adalah MAC yang membutuhkan perkalian dan akumulasi berulang. Penggunaan DSP yang hampir penuh menunjukkan bahwa rancangan memprioritaskan percepatan operasi aritmetika, sedangkan penerapan *time multiplexing* pada beberapa layer menjadi strategi untuk menjaga desain tetap muat pada FPGA. Implikasinya, pengembangan model yang lebih besar atau penambahan paralelisme perlu mempertimbangkan keterbatasan DSP sebagai faktor utama.

4.2.3 Hasil Timing CNN Accelerator

Tabel 4.5. Hasil Routing CNN Accelerator Design

Conflict Nets	Unrouted Nets	Partially Routed Nets	Fully Routed Nets
0	0	0	44973

Tabel 4.6. Hasil *CNN Accelerator Delayed Design*

WNS	TNS	WHS	WBSS	TPWS
0.121	0.00	0.010	0.00	3.75

Tabel hasil routing CNN accelerator menunjukkan bahwa tidak terdapat *conflict nets*, *unrouted nets*, maupun *partially routed nets*. Seluruh jaringan yang digunakan pada

desain berhasil dirutekan, ditunjukkan oleh nilai *fully routed nets* sebesar 44.973. Hasil ini menandakan bahwa proses implementasi fisik CNN accelerator berhasil menyambungkan seluruh jalur antarblok logika, register, memori, dan DSP tanpa menyisakan koneksi yang gagal dirutekan.

Tabel *CNN Accelerator Delayed Design* menunjukkan hasil analisis timing setelah implementasi. Nilai WNS sebesar 0,121 dan TNS sebesar 0,00 menunjukkan bahwa tidak ada jalur setup yang melanggar constraint waktu. Nilai WHS sebesar 0,010 dan WBSS sebesar 0,00 juga menunjukkan bahwa tidak ada pelanggaran hold yang signifikan pada desain. Dengan demikian, CNN accelerator telah memenuhi timing closure pada konfigurasi yang digunakan, meskipun penggunaan DSP sangat tinggi dan operasi MAC berpotensi membentuk jalur kritis yang panjang.

4.2.4 Hasil Latensi dan Throughput CNN Accelerator

Tabel 4.7. Hasil Latensi CNN Accelerator

Tahap	Jumlah Siklus	Latensi pada 100 MHz
Block 1 Output	98	0,98 μs
Block 2 Output	545	5,45 μs
Frame Done	788	7,88 μs
Block 3 Output	2457	24,57 μs
Dense Output	3802	38,02 μs

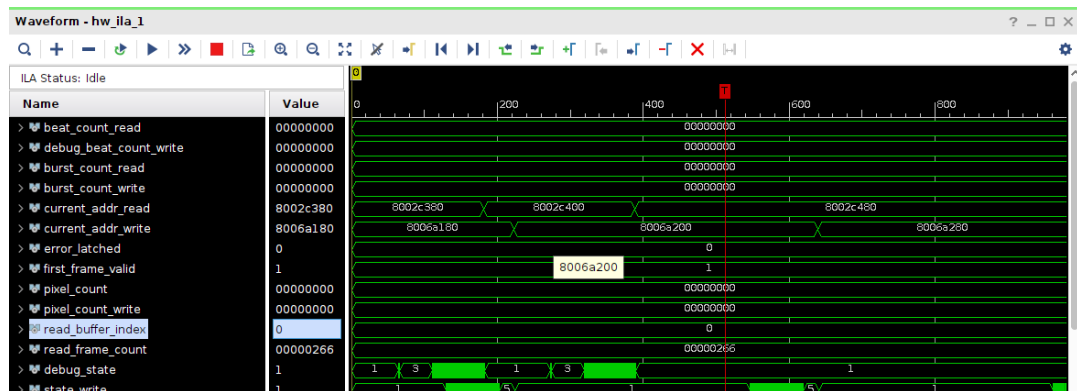
$$Throughput = \frac{1}{38,02 \times 10^{-6}} = 26302 \text{ inferensi/s} \quad (4-1)$$

Tabel latensi menunjukkan jumlah siklus yang dibutuhkan hingga keluaran pada beberapa tahap CNN tersedia. Keluaran Block 1 diperoleh pada 98 siklus, Block 2 pada 545 siklus, Frame Done pada 788 siklus, Block 3 pada 2457 siklus, dan Dense Output pada 3802 siklus. Pada clock 100 MHz, latensi akhir sampai keluaran dense adalah 38,02 μs .

Kenaikan jumlah siklus pada blok yang lebih dalam menunjukkan bahwa beban komputasi bertambah seiring peningkatan jumlah channel, filter, dan operasi MAC. Throughput yang dihitung dari latensi akhir menghasilkan 26302 inferensi/s, sehingga CNN accelerator memiliki kemampuan inferensi yang tinggi pada level komputasi. Nilai ini tetap perlu dibaca sebagai performa blok accelerator, sedangkan kinerja sistem penuh juga dipengaruhi oleh aliran data pada memori.

4.3 Hasil Pengujian Modul Perangkat Keras

4.3.1 Hasil Uji Double Frame Memory



Gambar 4.16. Hasil *debug* ILA pada *double frame memory*

Gambar 4.16 memperlihatkan hasil pengamatan melalui *Integrated Logic Analyzer* pada saat *double frame memory* berjalan. Sinyal alamat baca dan tulis bergerak pada area alamat DDR yang berbeda, sehingga proses akses memori tidak berhenti pada satu alamat tetap. Pergerakan *current address* ini menunjukkan bahwa transaksi memori berjalan setelah proses kalibrasi selesai dan sistem mampu mengakses frame yang disimpan pada DDR.

Hasil pengamatan juga menunjukkan mekanisme pertukaran buffer berjalan dengan baik. Sinyal indeks buffer tulis dan buffer baca (*buffer_write_index* dan *buffer_read_index*) berada pada buffer yang berbeda, sehingga proses tulis dan baca tidak menggunakan buffer yang sama secara bersamaan. Selain itu, sinyal error tetap bernilai 0 dan tidak terlihat indikasi error dari *async FIFO*. Dengan demikian, *double frame memory* dapat melakukan perpindahan buffer, menjaga aliran alamat, dan berjalan tanpa error FIFO pada skenario pengamatan ILA.

4.4 Hasil Integrasi Sistem

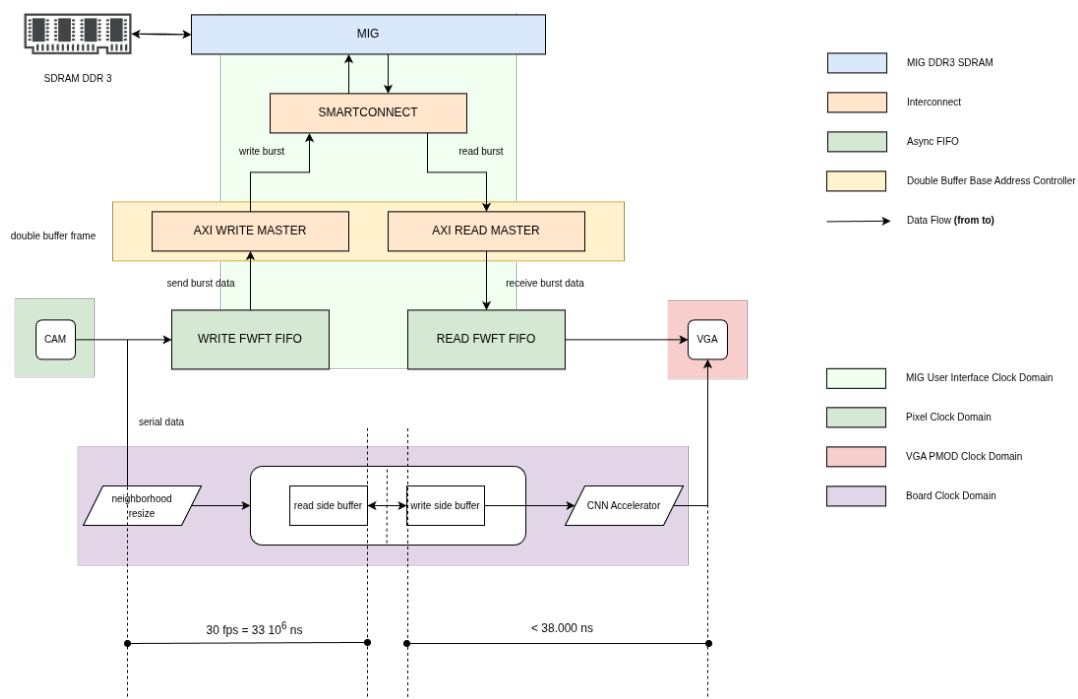
4.4.1 Hasil Integrasi Double Frame Memory

Tabel 4.8. Hasil *Double Frame Memory Implement Running*

Komponen	Resource Terpakai	Persentase %
LUT	6.969	11
Flip-Flop	6.357	5.01
Block RAM	0.5	0.37
DSP Slice	0	0.0

Tabel 4.8 menunjukkan bahwa integrasi *double frame memory* menggunakan 6.969 LUT atau 11% dari resource yang tersedia, 6.357 Flip-Flop atau 5,01%, dan 0,5 Block RAM atau 0,37%. Penggunaan DSP Slice bernilai 0 karena blok ini tidak melakukan operasi aritmetika berat seperti MAC, tetapi lebih berperan sebagai jalur kendali, pengaturan alamat, dan mekanisme buffer untuk proses baca tulis frame. Hasil tersebut menunjukkan bahwa subsistem memori relatif ringan sehingga masih menyisakan resource FPGA yang besar untuk diintegrasikan dengan CNN accelerator.

4.4.2 Hasil Integrasi CNN Accelerator dengan Double Frame Memory



Gambar 4.17. Desain Akhir Integrasi

Gambar 4.17 menunjukkan desain akhir integrasi sistem. Alur sistem menggabungkan *double frame memory*, CNN accelerator, dan argmax sebagai jalur utama pemrosesan.

Integrasi ini menjadi kontribusi utama penelitian karena sistem tidak berhenti pada pengujian accelerator terpisah. CNN ditempatkan sebagai bagian dari alur pemrosesan citra berbasis memori sehingga data dapat disimpan, dipilih dari buffer aktif, diproses, dan dikeluarkan dalam satu platform FPGA.

4.4.3 Hasil Penggunaan Resource Sistem Terintegrasi

Tabel 4.9. Detail Hasil *Implement Running*

Komponen	Resource Terpakai	Persentase %
LUT	20.755	32.74
Flip-Flop	34.502	27.21
Block RAM	19.5	14.44
DSP Slice	235 DSP48E1	97.92

Tabel 4.9 menunjukkan penggunaan resource pada sistem terintegrasi setelah *double frame memory*, CNN accelerator, dan jalur kontrol digabungkan. Resource yang digunakan meliputi 20.755 LUT atau 32,74%, 34.502 Flip-Flop atau 27,21%, dan 19,5 Block RAM atau 14,44%. Nilai ini menunjukkan bahwa kebutuhan logika, register, dan memori internal masih berada pada rentang yang dapat ditampung oleh Arty A7-100T.

DSP Slice menjadi resource terbesar, yaitu 235 unit DSP atau 97,92%. Hal ini menunjukkan bahwa CNN accelerator menjadi blok komputasi dominan karena operasi konvolusi dan dense membutuhkan banyak MAC. Dengan margin DSP yang sangat kecil, sistem masih dapat diimplementasikan pada FPGA, tetapi pengembangan model yang lebih besar atau penambahan paralelisme perlu mempertimbangkan keterbatasan DSP atau menggunakan strategi *multiplexing* yang lebih agresif.

4.4.4 Hasil Timing Sistem Terintegrasi

Tabel 4.10. Hasil Routing Design

Conflict Nets	Unrouted Nets	Partially Routed Nets	Fully Routed Nets
0	0	0	63141

Tabel 4.11. Hasil *Delayed Design*

WNS	TNS	WHS	WBSS	TPWS
0.177	0.00	0.012	0.00	11.305

4.4.5 Hasil Estimasi Daya Sistem Terintegrasi

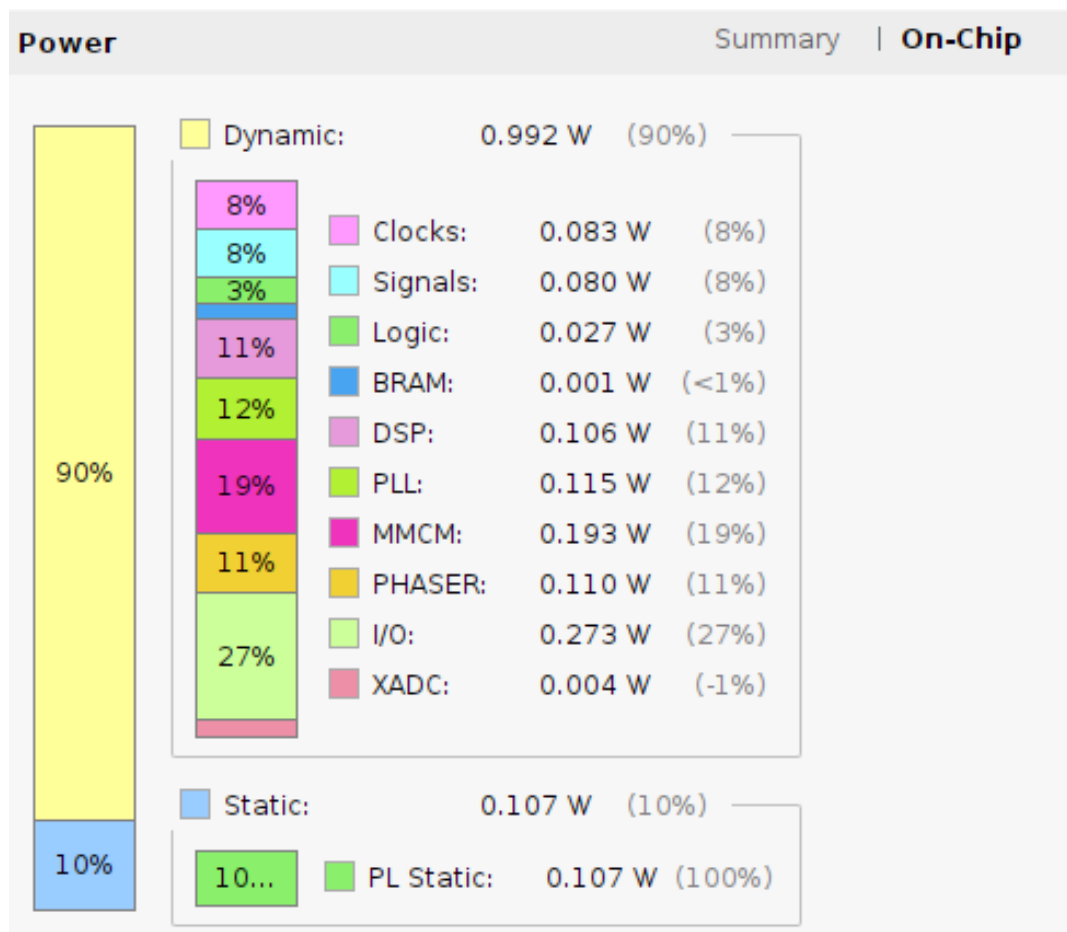
Hasil routing dan timing sistem terintegrasi menunjukkan seluruh net berhasil dirutekan dan nilai WNS tetap positif. Hal ini mengindikasikan bahwa penambahan sub-

sistem *double frame memory* dan CNN tidak menyebabkan pelanggaran timing pada constraint yang digunakan.

Timing closure pada sistem penuh penting karena integrasi sering menambah jalur kontrol dan interkoneksi yang dapat memperpanjang jalur kritis. Hasil ini menunjukkan bahwa desain akhir tidak hanya berhasil secara fungsional, tetapi juga layak dari sisi implementasi fisik pada FPGA.

Tabel 4.12. Penggunaan Daya dan Temperature

Total On Chip	Junction Temperature	Thermal Margin	Effective JA
1.099 W	30.0 C	55.0 C (11.9 W)	4.6 C/W



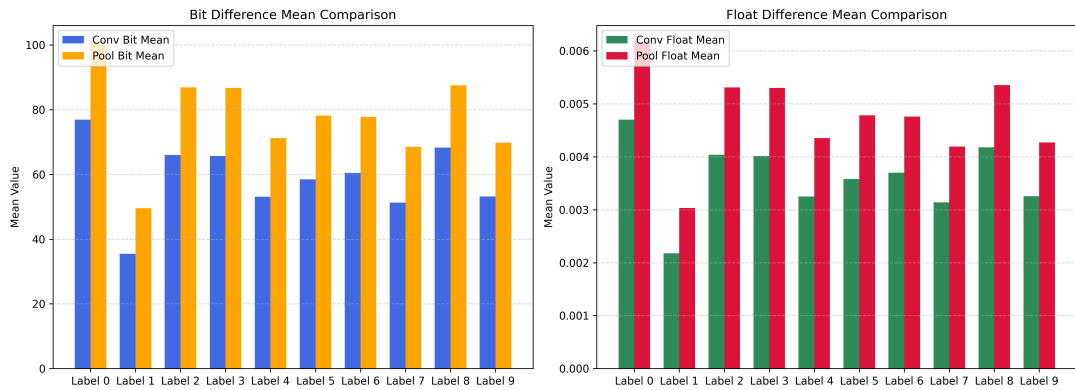
Gambar 4.18. Detail Hasil Penggunaan Daya

Tabel dan Gambar 4.18 menunjukkan estimasi daya sistem terintegrasi. Total on-chip power yang berada pada kisaran watt rendah mendukung motivasi penggunaan FPGA untuk embedded vision yang membutuhkan konsumsi daya lebih rendah dibandingkan platform komputasi umum.

Hasil daya ini sejalan dengan literatur yang menempatkan FPGA sebagai alternatif efisien untuk inferensi visi, terutama ketika jalur komputasi dapat dibuat khusus dan tidak perlu menjalankan overhead software seperti CPU atau GPU [6,9]. Namun, nilai ini tetap merupakan estimasi implementasi sehingga pengukuran daya langsung pada board dapat menjadi pekerjaan lanjutan.

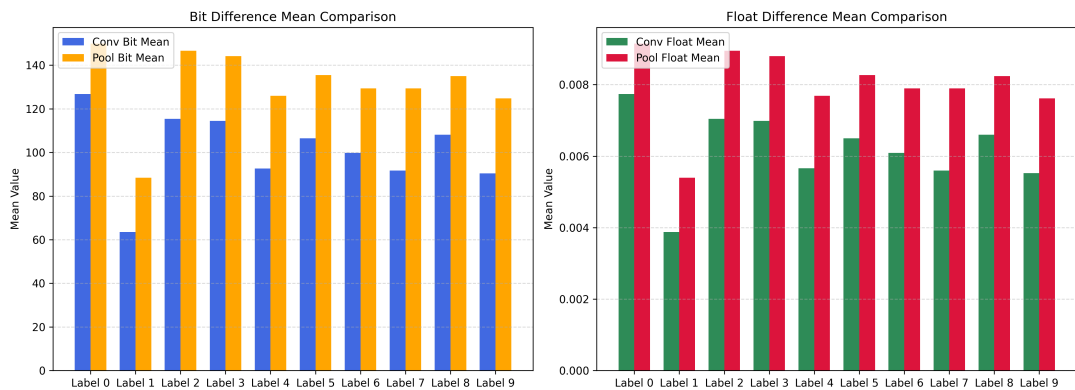
4.5 Hasil Evaluasi dan Pembahasan

4.5.1 Perbandingan Hasil Python dan Verilog



Gambar 4.19. Perbandingan *Bit* dan *Float value* pada *Conv 1* dan *Pool 1*

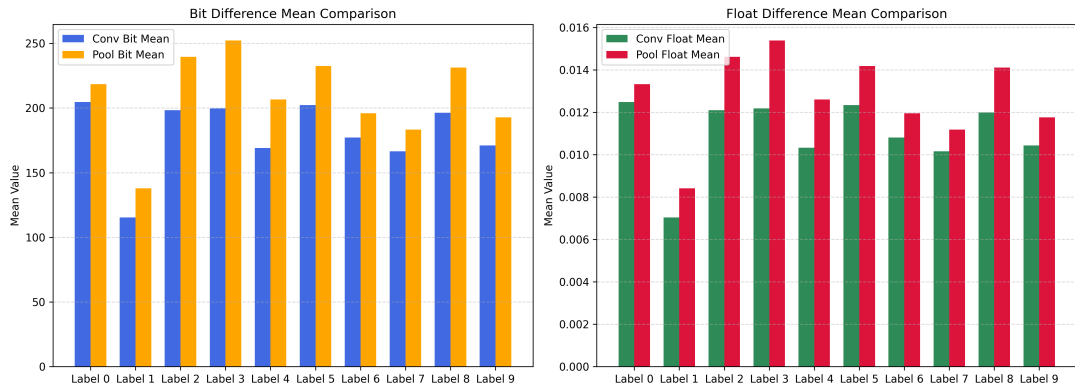
Gambar 4.19 memperlihatkan hasil perbandingan keluaran blok konvolusi pertama dan pooling pertama antara baseline Python dan simulasi Verilog Icarus. Pada label 0 sampai 9, selisih integer fixed-point dan selisih float masih relatif kecil, sehingga tahap awal jaringan belum menunjukkan penyimpangan yang besar dari implementasi referensi. Pola ini menandakan bahwa representasi fixed-point pada blok awal masih mampu menjaga keluaran tetap dekat dengan nilai acuan Python.



Gambar 4.20. Perbandingan *Bit* dan *Float value* pada *Conv 2* dan *Pool 2*

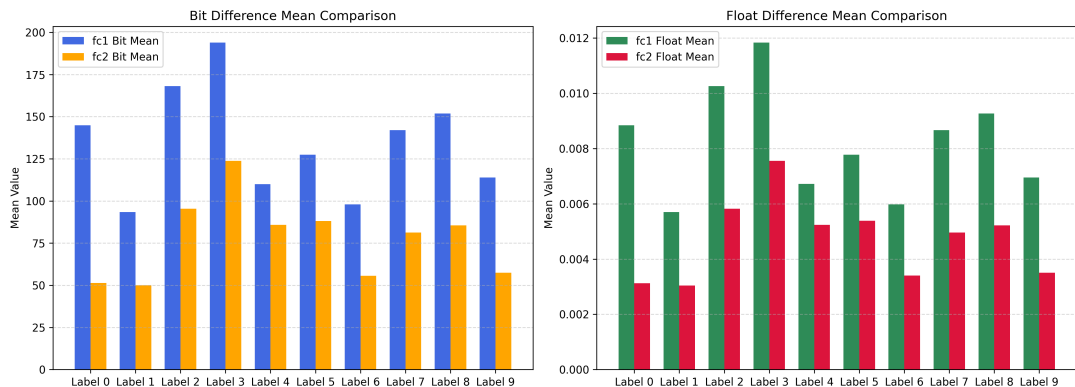
Gambar 4.20 menunjukkan bahwa pada blok konvolusi kedua dan pooling kedua, selisih mulai tampak lebih besar dibandingkan blok pertama. Hal ini konsisten dengan

akumulasi galat numerik pada layer yang lebih dalam, karena jumlah operasi MAC yang dilalui aktivasi semakin banyak. Meskipun demikian, besaran selisih float masih berada pada orde yang terkontrol sehingga perilaku layer tetap sejalan dengan baseline Python.



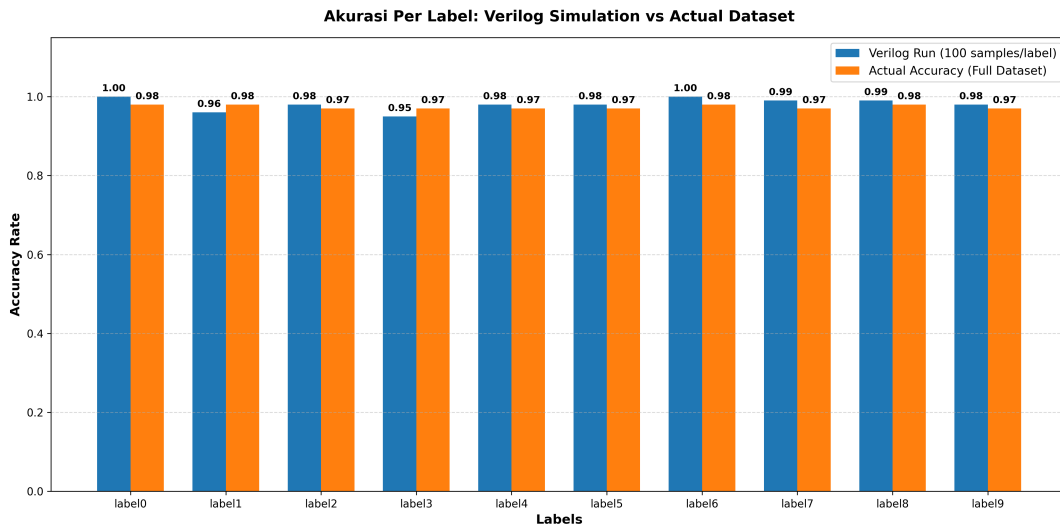
Gambar 4.21. Perbandingan *Bit* dan *Float value* pada *Conv 3* dan *Pool 3*

Gambar 4.21 memperlihatkan perbedaan yang paling menonjol di antara tiga blok konvolusi dan pooling. Wajar jika selisih pada layer ini lebih besar karena feature map yang diproses sudah melalui beberapa tahap pembulatan dan pemotongan bit sebelumnya. Selain itu, max pooling pada tahap ini juga dapat memperbesar efek kecil yang muncul pada hasil konvolusi sehingga galat numerik menjadi lebih terlihat pada beberapa label.



Gambar 4.22. Perbandingan *Bit* dan *Float value* pada *Dense 1* dan *Dense 2*

Gambar 4.22 memperlihatkan bahwa pada dense layer, FC1 cenderung memiliki selisih yang lebih besar daripada FC2. Ini menunjukkan bahwa galat fixed-point tidak terus bertambah sampai keluaran akhir, melainkan sebagian dapat teredam ketika fitur dipetakan ke sepuluh kelas keluaran. Dengan demikian, hasil akhirnya masih mempertahankan pola klasifikasi yang sama dengan baseline Python.



Gambar 4.23. Perbandingan Akurasi Verilog Simulation dan Actual Accuracy Dataset

Gambar 4.23 memperlihatkan evaluasi hasil argmax simulasi Verilog Icarus terhadap baseline Python. Evaluasi dilakukan menggunakan 100 sampel untuk setiap label, dan setiap sampel masukan yang digunakan pada Verilog dibandingkan dengan keluaran model Python untuk sampel yang sama. Pada seluruh label yang dievaluasi, implementasi Verilog menghasilkan kelas prediksi yang sama persis dengan model Python. Jika prediksi tersebut dibandingkan terhadap label aktual pada subset evaluasi, akurasi per label berada pada rentang 0,95 sampai 1,00 dengan rata-rata sekitar 98,1%. Angka ini menunjukkan performa pada subset 100 sampel per label, sedangkan akurasi Python pada keseluruhan data uji adalah 97,2%. Meskipun nilai keluaran fixed-point sedikit bergeser dari nilai Python akibat kuantisasi, pembulatan, pemotongan bit, dan akumulasi operasi MAC, pergeseran tersebut tidak mengubah hasil klasifikasi akhir berdasarkan argmax pada sampel yang dievaluasi.

4.5.2 Evaluasi Hasil Inferensi Model pada FPGA

Hasil inferensi pada FPGA perlu dibaca sebagai evaluasi kesesuaian antara model acuan Python dan realisasi hardware. Selama keputusan kelas yang dihasilkan tetap mengikuti baseline, maka perbedaan numerik antar-layer masih dapat diterima sebagai efek implementasi fixed-point.

Evaluasi ini juga menunjukkan batas penting dari penelitian: FPGA menjalankan inferensi menggunakan parameter yang sudah dilatih sebelumnya, bukan melakukan training ulang. Oleh karena itu, kualitas hasil hardware bergantung pada kualitas model Python, proses kuantisasi, dan ketepatan implementasi Verilog.

4.5.3 Pembahasan Resource, Latensi, dan Daya

Resource, latensi, dan daya menunjukkan trade-off utama pada rancangan ini. Penggunaan DSP yang hampir penuh memperlihatkan bahwa operasi MAC menjadi pusat beban komputasi, sedangkan time multiplexing membantu menjaga desain tetap muat pada FPGA dengan konsekuensi bertambahnya jumlah siklus.

Latensi akhir CNN accelerator masih berada pada skala mikrodetik berdasarkan clock 100 MHz, sehingga secara komputasi blok CNN memiliki throughput inferensi yang tinggi. Pada sistem penuh, kemampuan real-time tidak hanya ditentukan oleh accelerator, tetapi juga oleh jalur *double frame memory*. Estimasi daya yang rendah memperkuat relevansi FPGA untuk pemrosesan citra berbasis memori berganda, meskipun validasi daya langsung tetap diperlukan untuk memperoleh nilai konsumsi aktual pada board.

4.5.4 Pembahasan Ketercapaian Sistem terhadap Tujuan Penelitian

Berdasarkan hasil implementasi, tujuan perancangan CNN accelerator fixed-point 16-bit tercapai karena model berhasil dipetakan ke Verilog dan dievaluasi terhadap baseline Python. Tujuan integrasi juga tercapai melalui penggabungan *double frame memory*, CNN accelerator, dan argmax dalam desain sistem akhir.

Dari sisi performa, hasil menunjukkan bahwa evaluasi akurasi, perbedaan Python-Verilog, latency, resource, timing, dan daya telah dilakukan tanpa mengubah arsitektur model maupun hasil eksperimen. Keterbatasan utama penelitian berada pada margin DSP yang sangat kecil dan penggunaan estimasi daya dari tool, sehingga pengembangan selanjutnya perlu mempertimbangkan optimasi resource, pengukuran daya langsung, dan pengujian pada skenario citra nyata yang lebih bervariasi.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut.

1. Model CNN berhasil diimplementasikan pada board FPGA menggunakan representasi fixed-point 16-bit Q2.14, yaitu dua bit untuk bagian integer termasuk bit tanda dan 14 bit untuk bagian fraksi. Format ini digunakan karena parameter model tidak keluar dari rentang -2 sampai 2 , sehingga resolusi pecahan dapat diprioritaskan. Proses kuantisasi dilakukan terhadap parameter bobot dan bias model sehingga dapat digunakan pada modul Verilog tanpa menggunakan operasi floating-point.
2. Model Python mencapai akurasi 98,2% pada data latih dan 97,2% pada data uji. Pada evaluasi Verilog menggunakan 100 sampel untuk setiap label, kelas prediksi yang dihasilkan sama dengan keluaran model Python untuk sampel masukan yang sama. Perbedaan nilai numerik akibat kuantisasi, pembulatan, pemotongan bit, dan akumulasi operasi MAC tidak mengubah hasil klasifikasi akhir berdasarkan argmax pada sampel yang dievaluasi.
3. Hasil implementasi CNN accelerator menunjukkan bahwa sistem mampu menghasilkan keluaran inferensi dalam 3802 siklus clock. Pada frekuensi kerja 100 MHz, nilai tersebut setara dengan latensi sebesar 38,02 μs .
4. Hasil penggunaan resource menunjukkan bahwa CNN accelerator menggunakan sebagian besar DSP pada FPGA. Pada implementasi sistem penuh, penggunaan DSP mencapai 235 DSP48E1 dari total 240 DSP yang tersedia, sehingga DSP menjadi resource utama yang membatasi pengembangan model.
5. Sistem streaming citra berbasis OV7670, DDR3 SDRAM, dan VGA berhasil dirancang dan diintegrasikan sebagai bagian dari sistem pendukung akuisisi dan tampilan citra pada FPGA.
6. Hasil implementasi sistem penuh menunjukkan bahwa desain dapat melalui proses routing tanpa unrouted net dan memenuhi timing dengan nilai WNS positif. Estimasi daya total on-chip yang diperoleh adalah sebesar 1,099 W.

5.2 Saran

Berdasarkan penelitian yang telah dilakukan, terdapat beberapa saran untuk pengembangan selanjutnya sebagai berikut.

1. Optimasi penggunaan DSP perlu dilakukan karena implementasi CNN accelerator menggunakan resource DSP dalam jumlah sangat besar. Optimasi dapat dilakukan melalui penggunaan time multiplexing, pengurangan jumlah channel, atau perancangan ulang arsitektur CNN yang lebih ringan.
2. Pengujian sistem dapat dikembangkan menggunakan data citra yang diperoleh langsung dari kamera OV7670 agar sistem klasifikasi lebih merepresentasikan kondisi penggunaan secara real-time.
3. Pengukuran daya dapat dilakukan menggunakan alat ukur eksternal agar hasil konsumsi daya tidak hanya bergantung pada estimasi dari Vivado.

DAFTAR PUSTAKA

- [1] O. Elharrouss, Y. Akbari, N. Almaadeed, and S. A. Al-Maadeed, “Backbones-review: Feature extraction networks for deep learning and deep reinforcement learning approaches,” *ArXiv*, vol. abs/2206.08016, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:249712263>
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012. [Online]. Available: <https://neurips.cc>
- [3] X. Liao, Y. Li, S. Zhang, X. Wei, and J. Hu, “Predicting gpu training energy consumption in data centers using task metadata via symbolic regression,” *Energies*, vol. 19, no. 2, 2026. [Online]. Available: <https://www.mdpi.com/1996-1073/19/2/448>
- [4] S. Stirenko, Y. Rosenberg, Y. Gordienko, O. Alienin, O. Rokovyi, and N. Alienina, “Batch size influence on performance of graphic and tensor processing units during training and inference phases,” *arXiv preprint arXiv:1812.11731*, pp. 1–9, 2018. [Online]. Available: <https://arxiv.org/abs/1812.11731>
- [5] S. Grigorescu, B. Trasnea, T. Cocias, and G. Macesanu, “A survey of deep learning techniques for autonomous driving,” *Journal of Field Robotics*, vol. 37, no. 3, pp. 362–386, 2020.
- [6] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, “A survey of fpga-based neural network inference accelerators,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 12, no. 1, 2019.
- [7] K. Abdelouahab, M. Pelcat, J. Serot, and F. Berry, “Accelerating cnn inference on fpgas: A survey,” *arXiv preprint arXiv:1806.01683*, 2018.
- [8] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, “A performance study of general-purpose applications on graphics processors,” *Journal of Parallel and Distributed Computing*, vol. 69, no. 10, pp. 1370–1380, 2009.
- [9] M. Qasaimeh, K. Denolf, J. Lo, K. Vissers, M. Mattavelli, and J. Silas, “Comparing energy efficiency of cpu, gpu and fpga implementations for vision kernels,” in *2019 IEEE International Conference on Embedded Software and Systems (ICESS)*. IEEE, 2019, pp. 1–8.
- [10] Xilinx Inc., “Lowering power at 28 nm with xilinx 7 series fpgas,” White Paper: WP389 (v1.2), 2015.
- [11] R. Solovyev, A. Kustov, D. Telpukhov, V. Rukhlov, and A. Kalinin, “Fixed-point convolutional neural network for real-time video processing in FPGA,” in *2019 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (EIConRus)*. IEEE, 2019, pp. 1605–1611. [Online]. Available: <https://doi.org/10.1109/EIConRus.2019.8656778>

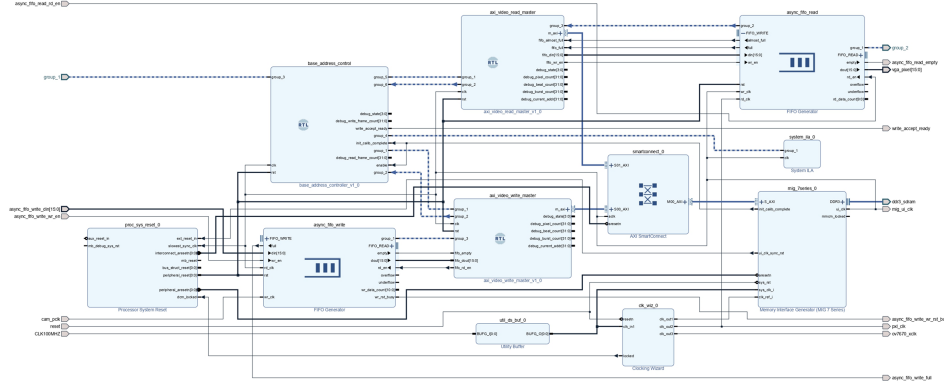
- [12] *OV7670/OV7171 CMOS VGA CameraChip Sensor Datasheet*, OmniVision Technologies, 2006, accessed: 2026-06-15. [Online]. Available: https://web.mit.edu/6.111/www/f2016/tools/OV7670_2006.pdf
- [13] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song, Y. Wang, and H. Yang, “Going deeper with embedded FPGA platform for convolutional neural network,” in *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA ’16. Association for Computing Machinery, 2016, pp. 26–35. [Online]. Available: <https://doi.org/10.1145/2847263.2847265>
- [14] A. Dua, Y. Li, and F. Ren, “Systolic-CNN: An OpenCL-defined scalable run-time-flexible FPGA accelerator architecture for accelerating convolutional neural network inference in cloud/edge computing,” 2020. [Online]. Available: <https://arxiv.org/abs/2012.03177>
- [15] S. Navaneethan, C. Thanusha, M. A. Varshini, and A. Anil, “A hardware efficient real-time video processing on FPGA with OV7670 camera interface and VGA,” in *2022 6th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*. IEEE, 2022, pp. 348–352. [Online]. Available: <https://doi.org/10.1109/ICECA55336.2022.10009129>
- [16] L. Sun, E. Sun, and Y. Yu, “Real time controllable multi channel HD digital video processing system based on FPGA and MicroBlaze,” in *Proceedings of the 2021 7th International Conference on Computing and Artificial Intelligence*, ser. ICCAI ’21. Association for Computing Machinery, 2021, pp. 183–190. [Online]. Available: <https://doi.org/10.1145/3467707.3467734>
- [17] A. Cosoroaba, “Achieving high performance DDR3 data rates,” Xilinx, White Paper WP383, Aug. 2013. [Online]. Available: <https://docs.amd.com/api/khub/documents/SKHv3pFeACeh9A7QJryuMg/content>
- [18] A. Jacobo, “FPGA_OV7670_Camera_Interface: Real-time streaming of OV7670 camera via VGA,” GitHub repository, 2021, accessed: 2026-06-15. [Online]. Available: https://github.com/AngeloJacobco/FPGA_OV7670_Camera_Interface
- [19] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: <https://www.deeplearningbook.org/>
- [20] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [21] V. Nair and G. E. Hinton, “Rectified linear units improve restricted boltzmann machines,” in *Proceedings of the 27th International Conference on Machine Learning*, 2010, pp. 807–814.
- [22] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, “Optimizing FPGA-based accelerator design for deep convolutional neural networks,” in *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA ’15. Association for Computing Machinery, 2015, pp. 161–170. [Online]. Available: <https://doi.org/10.1145/2684746.2689060>

- [23] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
- [24] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, “Deep learning with limited numerical precision,” in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 37. PMLR, 2015, pp. 1737–1746. [Online]. Available: <https://proceedings.mlr.press/v37/gupta15.html>
- [25] D. Lin, S. Talathi, and S. Annapureddy, “Fixed point quantization of deep convolutional networks,” in *Proceedings of the 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 48. PMLR, 2016, pp. 2849–2858. [Online]. Available: <https://proceedings.mlr.press/v48/linb16.html>
- [26] R. Goyal, J. Vanschoren, V. van Acht, and S. Nijssen, “Fixed-point quantization of convolutional neural networks for quantized inference on embedded platforms,” *arXiv preprint arXiv:2102.02147*, 2021. [Online]. Available: <https://arxiv.org/abs/2102.02147>
- [27] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” *arXiv preprint arXiv:1806.08342*, 2018. [Online]. Available: <https://arxiv.org/abs/1806.08342>
- [28] Xilinx, *7 Series DSP48E1 Slice User Guide*, Xilinx, 2018. [Online]. Available: https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1
- [29] F. Yan, F. Wang, C. Liu, Y. Wang, and J. Zhou, “Design of convolutional neural network processor based on fpga resource multiplexing architecture,” *Sensors*, vol. 22, no. 16, p. 5967, 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/16/5967>
- [30] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, “FINN: A framework for fast, scalable binarized neural network inference,” in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA ’17. Association for Computing Machinery, 2017, pp. 65–74. [Online]. Available: <https://doi.org/10.1145/3020078.3021744>
- [31] AMD, “68595 - DSP slice - using time division multiplexing or overclocking the DSP slices in Xilinx devices can greatly increase throughput and efficiency,” https://adaptivesupport.amd.com/s/article/68595?language=en_US, Feb. 2023, diakses pada: 16 Juni 2026.
- [32] C. E. Cummings and P. Alfke, “Simulation and synthesis techniques for asynchronous fifo design,” in *SNUG San Jose*. Sunburst Design, 2002. [Online]. Available: https://www.sunburst-design.com/papers/CummingsSNUG2002SJ_FIFO1.pdf

- [33] Arm Limited, *AMBA AXI and ACE Protocol Specification*, Arm Limited, 2020, non-Confidential. [Online]. Available: <https://developer.arm.com/documentation/ihl0022/latest>
- [34] —, *Introduction to AMBA AXI*, Arm Limited, 2020. [Online]. Available: <https://developer.arm.com/documentation/102202/latest>
- [35] Arm Ltd., *AMBA AXI and ACE Protocol Specification*, Arm Ltd., Dec. 2017. [Online]. Available: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/IHI0022H_amba_axi_protocol_spec.pdf
- [36] Xilinx, *7 Series FPGAs Memory Interface Solutions User Guide*, Xilinx, Apr. 2012. [Online]. Available: https://www.xilinx.com/support/documents/ip_documentation/mig_7series/v4_1/ug586_7Series_MIS.pdf
- [37] Digilent, *VGA Display Controller*, Digilent Inc., 2024. [Online]. Available: <https://digilent.com/reference/learn/programmable-logic/tutorials/vga-display-controller/start>
- [38] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, “A survey of fpga-based neural network inference accelerators,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 12, no. 1, pp. 1–26, 2019. [Online]. Available: <https://dl.acm.org/doi/10.1145/3289185>
- [39] hojjatk, “MNIST Dataset,” Kaggle dataset, 2026, diakses: 25 Juni 2026. [Online]. Available: <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>
- [40] H.-J. Kang, “AoCStream: All-on-chip CNN accelerator with stream-based line-buffer architecture,” in *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA ’23. Association for Computing Machinery, 2023, p. 48. [Online]. Available: <https://doi.org/10.1145/3543622.3573141>
- [41] J. F. Wakerly, *Digital Design: Principles and Practices*, 4th ed. Pearson Prentice Hall, 2005.
- [42] S. L. Harris and D. Harris, *Digital Design and Computer Architecture*, 2nd ed. Morgan Kaufmann, 2021.
- [43] J. H. Kim, B. Grady, R. Lian, J. Brothers, and J. H. Anderson, “FPGA-based CNN inference accelerator synthesized from multi-threaded C software,” in *2017 30th IEEE International System-on-Chip Conference (SOCC)*. IEEE, 2017, pp. 268–273. [Online]. Available: <https://doi.org/10.1109/SOCC.2017.8226056>
- [44] AMD Xilinx, *7 Series FPGAs Memory Interface Solutions User Guide*, Advanced Micro Devices, Inc., 2024. [Online]. Available: https://docs.amd.com/r/en-US/ug586_7Series_MIS

LAMPIRAN

L.1 Block Design Sistem OV7670, DDR3, dan VGA



Gambar L.1. Block Design Alur Inferensi Camera Sampai VGA

L.2 Kode Implementasi Kuantisasi Parameter 16 Bit

```
1
2 def quantize_signed(x, bits, frac_bits):
3     q = np.round(x * (1 << frac_bits)).astype(np.int64)
4     min_q = -(1 << (bits - 1))
5     max_q = (1 << (bits - 1)) - 1
6     sat_low = int(np.sum(q < min_q))
7     sat_high = int(np.sum(q > max_q))
8     q = np.clip(q, min_q, max_q)
9     return q.astype(np.int64), sat_low, sat_high
10
11 def to_twos_hex(v, bits):
12     v = int(v)
13     if v < 0:
14         v = (1 << bits) + v
15     return f"{v:0{bits//4}X}"
16
17 def save_mem(path, q, bits):
18     flat = q.reshape(-1)
19     with open(path, "w") as f:
20         for v in flat:
21             f.write(to_twos_hex(v, bits) + "\n")
22     print(f"saved: {path} lines={len(flat)} bits={bits}")
```

```

23
24 def report(name, arr, q, lo, hi):
25     print(f"\n{name}")
26     print("    shape:", arr.shape)
27     print("    float min/max:", float(np.min(arr)), float(np.max(
arr)))
28     print("    quant min/max:", int(np.min(q)), int(np.max(q)))
29     print("    saturation low/high:", lo, hi)
30
31

```

Kode Kuantisasi 16 bit parameter model

L.3 Kode Evaluasi Hasil Perbedaan Bit

```

1
2 import numpy as np
3 import pandas as pd
4
5 def sign_extend(arr, bit_width):
6     """
7     Untuk mengubah unsigned two's complement menjadi signed
integer.
8     Contoh kalau ACT_SIZE = 24, maka bit_width = 24.
9     """
10    arr = arr.astype(np.int64)
11
12    if bit_width is None:
13        return arr
14
15    sign_bit = 1 << (bit_width - 1)
16    full_range = 1 << bit_width
17
18    return np.where(arr >= sign_bit, arr - full_range, arr)
19
20
21 def python_float_to_fixed(arr, frac_bits=14, mode="round"):
22     """
23     Convert float Python ke fixed-point integer.
24     """
25     scale = 1 << frac_bits
26     scaled = arr * scale

```

```

27
28     if mode == "round":
29         return np.round(scaled).astype(np.int64)
30
31     elif mode == "trunc":
32         return np.trunc(scaled).astype(np.int64)
33
34     elif mode == "floor":
35         return np.floor(scaled).astype(np.int64)
36
37     else:
38         raise ValueError("mode harus 'round', 'trunc', atau '
39         floor'")
40
41 def compare_numpy_vs_verilog(
42     python_res ,
43     verilog_res ,
44     row_count ,
45     channel_count ,
46     prefix="ch" ,
47     frac_bits=14,
48     bit_width=24,
49     verilog_is_fixed=True ,
50     quant_mode="round"
51 ):
52     """
53     Membandingkan hasil NumPy Python vs hasil Verilog/Icarus.
54
55     python_res:
56         shape (row_count, channel_count)
57         biasanya float
58
59     verilog_res:
60         shape (row_count, channel_count)
61         biasanya integer fixed-point dari Verilog
62
63     Output dataframe:
64         ch0_verilog, ch0_python, ..., ch7_verilog, ch7_python,
65         float_diff, bit_diff
66     """

```

```

66
67 python_res = np.asarray(python_res, dtype=np.float64)
68 verilog_res = np.asarray(verilog_res)
69
70 expected_shape = (row_count, channel_count)
71
72 if python_res.shape != expected_shape:
73     raise ValueError(
74         f"Shape python_res salah. Dapat {python_res.shape},
75         expected {expected_shape}"
76     )
77
78 if verilog_res.shape != expected_shape:
79     raise ValueError(
80         f"Shape verilog_res salah. Dapat {verilog_res.shape},
81         expected {expected_shape}"
82     )
83
84 if verilog_is_fixed:
85     verilog_fixed = verilog_res.astype(np.int64)
86
87     # gunakan ini kalau output Verilog adalah raw two's
88     complement
89     verilog_fixed = sign_extend(verilog_fixed, bit_width)
90
91     verilog_float = verilog_fixed.astype(np.float64) / (1 <<
92     frac_bits)
93
94 else:
95     # kalau Verilog sudah menulis float
96     verilog_float = verilog_res.astype(np.float64)
97     verilog_fixed = python_float_to_fixed(
98         verilog_float,
99         frac_bits=frac_bits,
100         mode=quant_mode
101     )
102
103 python_fixed = python_float_to_fixed(
104     python_res,
105     frac_bits=frac_bits,
106     mode=quant_mode

```

```

103     )
104
105     float_diff_per_ch = np.abs(python_res - verilog_float)
106     bit_diff_per_ch = np.abs(python_fixed - verilog_fixed)
107
108     # Satu row punya banyak channel.
109     # Maka float_diff dan bit_diff di sini adalah diff maksimum
110     # dari semua channel pada row itu.
111     float_diff = np.max(float_diff_per_ch, axis=1)
112     bit_diff = np.max(bit_diff_per_ch, axis=1)
113
114     data = {}
115
116     for ch in range(channel_count):
117         data[f"{prefix}{ch}_verilog"] = verilog_float[:, ch]
118         data[f"{prefix}{ch}_python"] = python_res[:, ch]
119
120     data["float_diff"] = float_diff
121     data["bit_diff"] = bit_diff
122
123     df = pd.DataFrame(data)
124
125     return df

```

Kode evaluasi perbedaan bit Python dan Verilog

L.4 Kode Verilog AXI Read

```

1
2 `timescale 1ns / 1ps
3
4 module axi_video_read_master #(
5     parameter AXI_ADDR_WIDTH    = 32,
6     parameter AXI_DATA_WIDTH    = 64,
7     parameter AXI_ID_WIDTH      = 4,
8
9     parameter PIXEL_BITS        = 16,
10
11     parameter FRAME_WIDTH       = 640,
12     parameter FRAME_HEIGHT      = 480,
13

```

```

14 // Untuk MIG AXI 64-bit , aman pakai burst 8.
15 parameter BURST_LEN          = 8,
16
17 // Sesuai FIFO Generator kamu:
18 // wr_data_count width = 10-bit
19 parameter FIFO_COUNT_WIDTH = 10,
20
21 // FIFO depth efektif untuk count 10-bit.
22 // Walaupun di GUI kamu tulis 1028, count 10-bit praktis
dipakai sebagai 1024 level.
23 parameter FIFO_DEPTH_WORDS = 1024,
24
25 // Margin supaya tidak terlalu dekat full.
26 parameter FIFO_SAFE_MARGIN = 8,
27
28 parameter AXI_ID_VALUE      = 0
29 )(
30     input  wire                clk ,
31     input  wire                rst ,           // active-
high , sync to clk
32
33     input  wire                enable ,
34     input  wire                frame_start ,  // pulse 1
clock
35     input  wire [AXI_ADDR_WIDTH-1:0] frame_base_addr ,
36
37     output reg [PIXEL_BITS-1:0] fifo_din ,
38     output reg                fifo_wr_en ,
39     input  wire                fifo_full ,
40     input  wire                fifo_almost_full ,
41     input  wire [FIFO_COUNT_WIDTH-1:0] fifo_wr_data_count ,
42     input  wire                fifo_wr_rst_busy ,
43
44     //
=====
45     // AXI4 Read Address Channel
46     //
=====
47     output wire [AXI_ID_WIDTH-1:0] m_axi_arid ,
48     output reg [AXI_ADDR_WIDTH-1:0] m_axi_araddr ,
49     output wire [7:0] m_axi_arlen ,

```

```

50     output wire [2:0]                                m_axi_arsize ,
51     output wire [1:0]                                m_axi_arburst ,
52     output wire                                        m_axi_arlock ,
53     output wire [3:0]                                m_axi_arcache ,
54     output wire [2:0]                                m_axi_arprot ,
55     output wire [3:0]                                m_axi_arqos ,
56     output reg                                        m_axi_arvalid ,
57     input  wire                                        m_axi_arready ,
58
59     //
=====
60     // AXI4 Read Data Channel
61     //
=====
62     input  wire [AXI_ID_WIDTH-1:0]    m_axi_rid ,
63     input  wire [AXI_DATA_WIDTH-1:0]   m_axi_rdata ,
64     input  wire [1:0]                  m_axi_rresp ,
65     input  wire                        m_axi_rlast ,
66     input  wire                        m_axi_rvalid ,
67     output reg                          m_axi_rready ,
68
69     //
=====
70     // Status / debug
71     //
=====
72     output reg                          busy ,
73     output reg                          done ,
74     output reg                          error ,
75     output reg [3:0]                    debug_state ,
76     output reg [31:0]                   debug_pixel_count ,
77     output reg [31:0]                   debug_beat_count ,
78     output reg [31:0]                   debug_burst_count ,
79     output reg [AXI_ADDR_WIDTH-1:0]     debug_current_addr
80 );
81
82     //
=====
83     // Local parameters
84     //
=====

```

```

85     localparam integer BYTES_PER_BEAT      = AXI_DATA_WIDTH / 8;
86     localparam integer PIXELS_PER_BEAT     = AXI_DATA_WIDTH /
PIXEL_BITS;
87     localparam integer FRAME_PIXELS        = FRAME_WIDTH *
FRAME_HEIGHT;
88     localparam integer FRAME_BEATS         = FRAME_PIXELS /
PIXELS_PER_BEAT;
89     localparam integer PIXELS_PER_BURST    = BURST_LEN *
PIXELS_PER_BEAT;
90     localparam integer ADDR_STEP_BURST     = BURST_LEN *
BYTES_PER_BEAT;

91
92     localparam integer FIFO_START_LEVEL    =
93         FIFO_DEPTH_WORDS - PIXELS_PER_BURST - FIFO_SAFE_MARGIN;
94
95     // Untuk konfigurasi sekarang:
96     // AXI_DATA_WIDTH      = 64
97     // BYTES_PER_BEAT      = 8
98     // PIXELS_PER_BEAT     = 4
99     // BURST_LEN           = 8
100    // PIXELS_PER_BURST    = 32
101    // ADDR_STEP_BURST     = 64 byte
102
103    //
=====
104    // AXI size encoder
105    // AXI ARSIZE:
106    // 000 = 1 byte
107    // 001 = 2 byte
108    // 010 = 4 byte
109    // 011 = 8 byte  -> 64-bit
110    // 100 = 16 byte -> 128-bit
111    //
=====

112    function [2:0] axi_size_from_bytes;
113        input integer bytes;
114        begin
115            case (bytes)
116                1:    axi_size_from_bytes = 3'b000;
117                2:    axi_size_from_bytes = 3'b001;
118                4:    axi_size_from_bytes = 3'b010;

```



```

119             8:    axi_size_from_bytes = 3'b011;
120             16:    axi_size_from_bytes = 3'b100;
121             32:    axi_size_from_bytes = 3'b101;
122             64:    axi_size_from_bytes = 3'b110;
123             128:   axi_size_from_bytes = 3'b111;
124             default: axi_size_from_bytes = 3'b000;
125         endcase
126     end
127 endfunction
128
129 //
=====
130 // State encoding
131 //
=====
132 localparam [3:0]
133     S_IDLE      = 4'd0 ,
134     S_WAIT_SPACE = 4'd1 ,
135     S_AR        = 4'd2 ,
136     S_R         = 4'd3 ,
137     S_UNPACK    = 4'd4 ,
138     S_DONE      = 4'd5 ,
139     S_ERROR     = 4'd6 ;
140
141 reg [3:0] state ;
142
143 reg [AXI_ADDR_WIDTH-1:0] addr_reg ;
144
145 reg [31:0] pixel_count ;
146 reg [31:0] beat_count_total ;
147 reg [31:0] burst_count_total ;
148
149 reg [7:0] beat_index_in_burst ;
150 reg [3:0] unpack_index ;
151
152 reg [AXI_DATA_WIDTH-1:0] read_data_reg ;
153 reg                      read_last_reg ;
154
155 wire [31:0] fifo_used_words ;
156 wire        fifo_has_space_for_burst ;
157

```

```

158     assign fifo_used_words = fifo_wr_data_count;
159
160     assign fifo_has_space_for_burst =
161         (!fifo_full) &&
162         (!fifo_almost_full) &&
163         (!fifo_wr_rst_busy) &&
164         (fifo_used_words <= FIFO_START_LEVEL);
165
166     //
=====
167     // AXI constant signals
168     //
=====
169     assign m_axi_arid      = AXI_ID_VALUE;
170     assign m_axi_arlen     = BURST_LEN - 1;
171     assign m_axi_arsize    = axi_size_from_bytes(BYTES_PER_BEAT);
172     assign m_axi_arburst   = 2'b01;           // INCR burst
173     assign m_axi_arlock    = 1'b0;
174     assign m_axi_arcache   = 4'b0011;
175     assign m_axi_arprot    = 3'b000;
176     assign m_axi_arqos     = 4'b0000;
177
178     //
=====
179     // Main FSM
180     //
=====
181     always @(posedge clk) begin
182         if (rst) begin
183             state <= S_IDLE;
184
185             fifo_din <= {PIXEL_BITS{1'b0}};
186             fifo_wr_en <= 1'b0;
187
188             m_axi_araddr <= {AXI_ADDR_WIDTH{1'b0}};
189             m_axi_arvalid <= 1'b0;
190             m_axi_rready <= 1'b0;
191
192             busy <= 1'b0;
193             done <= 1'b0;
194             error <= 1'b0;

```

```

195         debug_state          <= S_IDLE;
196
197         addr_reg              <= {AXI_ADDR_WIDTH{1'b0}};
198
199         pixel_count           <= 32'd0;
200         beat_count_total      <= 32'd0;
201         burst_count_total     <= 32'd0;
202
203         beat_index_in_burst   <= 8'd0;
204         unpack_index          <= 4'd0;
205
206         read_data_reg         <= {AXI_DATA_WIDTH{1'b0}};
207         read_last_reg         <= 1'b0;
208
209         debug_pixel_count     <= 32'd0;
210         debug_beat_count      <= 32'd0;
211         debug_burst_count     <= 32'd0;
212         debug_current_addr    <= {AXI_ADDR_WIDTH{1'b0}};
213     end else begin
214         // Default pulse
215         fifo_wr_en    <= 1'b0;
216         done          <= 1'b0;
217         debug_state   <= state;
218
219         case (state)
220
221             // IDLE
222             // Menunggu frame_start dari
223             base_address_controller.
224             S_IDLE: begin
225                 busy          <= 1'b0;
226                 error         <= 1'b0;
227
228                 m_axi_arvalid <= 1'b0;
229                 m_axi_rready  <= 1'b0;
230
231                 fifo_wr_en    <= 1'b0;
232
233                 if (enable && frame_start && !
234                     fifo_wr_rst_busy) begin
235                     busy          <= 1'b1;

```

```

234
235         addr_reg                <= frame_base_addr;
236         m_axi_araddr            <= frame_base_addr;
237         debug_current_addr      <= frame_base_addr;
238
239         pixel_count             <= 32'd0;
240         beat_count_total        <= 32'd0;
241         burst_count_total       <= 32'd0;
242
243         beat_index_in_burst     <= 8'd0;
244         unpack_index            <= 4'd0;
245
246         read_data_reg           <= {AXI_DATA_WIDTH
{1'b0}};
247
248         read_last_reg           <= 1'b0;
249
250         debug_pixel_count       <= 32'd0;
251         debug_beat_count        <= 32'd0;
252         debug_burst_count       <= 32'd0;
253
254         state                    <= S_WAIT_SPACE;
255     end
256 end
257 S_WAIT_SPACE: begin
258     m_axi_rready <= 1'b0;
259
260     if (!enable) begin
261         state <= S_IDLE;
262     end else if (beat_count_total >= FRAME_BEATS
) begin
263         state <= S_DONE;
264     end else if (fifo_has_space_for_burst) begin
265         m_axi_araddr    <= addr_reg;
266         m_axi_arvalid   <= 1'b1;
267         debug_current_addr <= addr_reg;
268         state           <= S_AR;
269     end
270 end
271
272 S_AR: begin

```

```

273         if (m_axi_arvalid && m_axi_arready) begin
274             m_axi_arvalid      <= 1'b0;
275
276             beat_index_in_burst <= 8'd0;
277             unpack_index       <= 4'd0;
278
279             // Siap menerima data read dari AXI.
280             m_axi_rready       <= 1'b1;
281
282             state               <= S_R;
283         end else if (!enable) begin
284             // Belum handshake, jadi aman batal.
285             m_axi_arvalid <= 1'b0;
286             m_axi_rready  <= 1'b0;
287             busy          <= 1'b0;
288             state         <= S_IDLE;
289         end
290     end
291
292     S_R: begin
293         if (m_axi_rvalid && m_axi_rready) begin
294             m_axi_rready <= 1'b0;
295
296             read_data_reg <= m_axi_rdata;
297             read_last_reg <= m_axi_rlast;
298
299             if (m_axi_rresp != 2'b00) begin
300                 error <= 1'b1;
301                 state <= S_ERROR;
302             end else begin
303                 unpack_index <= 4'd0;
304                 state        <= S_UNPACK;
305             end
306         end
307     end
308
309     S_UNPACK: begin
310         if (!fifo_full && !fifo_wr_rst_busy) begin
311             fifo_din <= read_data_reg[unpack_index
312 *PIXEL_BITS -: PIXEL_BITS];
313             fifo_wr_en <= 1'b1;

```

```

313
314         pixel_count <= pixel_count + 32'd1;
315
316         if (unpack_index == PIXELS_PER_BEAT - 1)
317             begin
318                 unpack_index <= 4'd0;
319                 beat_count_total <= beat_count_total
320                 + 32'd1;
321
322                 // Cek RLAST:
323                 // RLAST harus muncul hanya pada
324                 beat terakhir burst.
325                 if ((read_last_reg && (
326                 beat_index_in_burst != BURST_LEN - 1)) ||
327                 (!read_last_reg && (
328                 beat_index_in_burst == BURST_LEN - 1))) begin
329                     error <= 1'b1;
330                     state <= S_ERROR;
331                 end else begin
332                     if (beat_index_in_burst ==
333                     BURST_LEN - 1) begin
334                         // Satu burst selesai.
335                         burst_count_total <=
336                         burst_count_total + 32'd1;
337
338                         addr_reg <= addr_reg +
339                         ADDR_STEP_BURST;
340                         debug_current_addr <=
341                         addr_reg + ADDR_STEP_BURST;
342
343                         beat_index_in_burst <= 8'd0;
344                         m_axi_rready <= 1'b0;
345
346                         if ((beat_count_total + 32'
347                         d1) >= FRAME_BEATS) begin
348                             state <= S_DONE;
349                         end else begin
350                             state <= S_WAIT_SPACE;
351                         end
352                     end else begin
353                         // Masih ada beat berikutnya

```

```

    dalam burst ini.

344                                     beat_index_in_burst <=
beat_index_in_burst + 8'd1;

345
346                                     // Siap menerima beat AXI
berikutnya.

347                                     m_axi_rready <= 1'b1;
348                                     state          <= S_R;
349                                     end
350                                     end
351                                     end else begin
352                                         unpack_index <= unpack_index + 4'd1;
353                                     end
354                                     end
355                                     end
356
357 S_DONE: begin
358     busy <= 1'b0;
359     done <= 1'b1;
360
361     m_axi_arvalid <= 1'b0;
362     m_axi_rready  <= 1'b0;
363     fifo_wr_en    <= 1'b0;
364
365     debug_pixel_count <= pixel_count;
366     debug_beat_count  <= beat_count_total;
367     debug_burst_count <= burst_count_total;
368
369     state <= S_IDLE;
370 end
371
372 S_ERROR: begin
373     busy <= 1'b0;
374     done <= 1'b0;
375
376     m_axi_arvalid <= 1'b0;
377     m_axi_rready  <= 1'b0;
378     fifo_wr_en    <= 1'b0;
379
380     debug_pixel_count <= pixel_count;
381     debug_beat_count  <= beat_count_total;

```

```

382         debug_burst_count <= burst_count_total;
383
384         // Tetap di error sampai reset.
385         state <= S_ERROR;
386     end
387
388     default: begin
389         state <= S_IDLE;
390     end
391
392 endcase
393 end
394 end
395
396 endmodule
397
398

```

Kode AXI Read Yang akan mengisi *FIFO VGA*

L.5 Kode Verilog AXI Write

```

1
2 `timescale 1ns / 1ps
3
4 module axi_video_write_master #(
5     parameter AXI_ADDR_WIDTH  = 32,
6     parameter AXI_DATA_WIDTH  = 64,
7     parameter AXI_ID_WIDTH    = 4,
8
9     parameter PIXEL_BITS      = 16,
10
11     parameter FRAME_WIDTH     = 640,
12     parameter FRAME_HEIGHT    = 480,
13
14     parameter BURST_LEN       = 8,
15
16     // Sesuai FIFO Generator kamu:
17     // rd_data_count width = 10-bit
18     parameter FIFO_COUNT_WIDTH = 10,
19
20     parameter AXI_ID_VALUE     = 0

```



```

21 )(
22     input  wire          clk ,
23     input  wire          rst ,          // active -
        high , sync to clk
24
25     input  wire          enable ,
26     input  wire          frame_start ,  // pulse 1
        clock
27     input  wire [AXI_ADDR_WIDTH-1:0]    frame_base_addr ,
28
29     input  wire [PIXEL_BITS-1:0]         fifo_dout ,
30     input  wire          fifo_empty ,
31     input  wire          fifo_valid ,
32     input  wire [FIFO_COUNT_WIDTH-1:0]    fifo_rd_data_count ,
33     input  wire          fifo_rd_rst_busy ,
34     output reg          fifo_rd_en ,
35
36
37     output wire [AXI_ID_WIDTH-1:0]        m_axi_awid ,
38     output reg [AXI_ADDR_WIDTH-1:0]      m_axi_awaddr ,
39     output wire [7:0]                    m_axi_awlen ,
40     output wire [2:0]                    m_axi_awsz ,
41     output wire [1:0]                    m_axi_awburst ,
42     output wire          m_axi_awlock ,
43     output wire [3:0]                    m_axi_awcache ,
44     output wire [2:0]                    m_axi_awprot ,
45     output wire [3:0]                    m_axi_awqos ,
46     output reg          m_axi_awvalid ,
47     input  wire          m_axi_awready ,
48
49
50     output reg [AXI_DATA_WIDTH-1:0]      m_axi_wdata ,
51     output wire [AXI_DATA_WIDTH/8-1:0]    m_axi_wstrb ,
52     output reg          m_axi_wlast ,
53     output reg          m_axi_wvalid ,
54     input  wire          m_axi_wready ,
55
56     //
=====
57     // AXI4 Write Response Channel
58     //

```

```

=====
59     input  wire  [AXI_ID_WIDTH-1:0]      m_axi_bid ,
60     input  wire  [1:0]                  m_axi_bresp ,
61     input  wire                                m_axi_bvalid ,
62     output reg                                m_axi_bready ,
63
64     //
=====
65     // Status / debug
66     //
=====
67     output reg                                busy ,
68     output reg                                done ,
69     output reg                                error ,
70     output reg  [3:0]                        debug_state ,
71     output reg  [31:0]                       debug_pixel_count ,
72     output reg  [31:0]                       debug_beat_count ,
73     output reg  [31:0]                       debug_burst_count ,
74     output reg  [AXI_ADDR_WIDTH-1:0]         debug_current_addr
75 );
76
77     //
=====
78     // Local parameters
79     //
=====
80     localparam integer BYTES_PER_BEAT      = AXI_DATA_WIDTH / 8;
81     localparam integer PIXELS_PER_BEAT     = AXI_DATA_WIDTH /
PIXEL_BITS;
82     localparam integer FRAME_PIXELS       = FRAME_WIDTH *
FRAME_HEIGHT;
83     localparam integer FRAME_BEATS        = FRAME_PIXELS /
PIXELS_PER_BEAT;
84     localparam integer PIXELS_PER_BURST   = BURST_LEN *
PIXELS_PER_BEAT;
85     localparam integer ADDR_STEP_BURST    = BURST_LEN *
BYTES_PER_BEAT;
86
87     // Untuk konfigurasi sekarang:
88     // AXI_DATA_WIDTH      = 64
89     // BYTES_PER_BEAT      = 8

```

```

90 // PIXELS_PER_BEAT = 4
91 // BURST_LEN      = 8
92 // PIXELS_PER_BURST = 32
93 // ADDR_STEP_BURST = 64 byte
94
95
96 function [2:0] axi_size_from_bytes;
97     input integer bytes;
98     begin
99         case (bytes)
100             1: axi_size_from_bytes = 3'b000;
101             2: axi_size_from_bytes = 3'b001;
102             4: axi_size_from_bytes = 3'b010;
103             8: axi_size_from_bytes = 3'b011;
104             16: axi_size_from_bytes = 3'b100;
105             32: axi_size_from_bytes = 3'b101;
106             64: axi_size_from_bytes = 3'b110;
107             128: axi_size_from_bytes = 3'b111;
108             default: axi_size_from_bytes = 3'b000;
109         endcase
110     end
111 endfunction
112
113 //
=====
114 // State encoding
115 //
=====
116 localparam [3:0]
117     S_IDLE      = 4'd0,
118     S_WAIT_FIFO = 4'd1,
119     S_AW        = 4'd2,
120     S_PACK      = 4'd3,
121     S_W         = 4'd4,
122     S_B         = 4'd5,
123     S_DONE      = 4'd6,
124     S_ERROR     = 4'd7;
125
126 reg [3:0] state;
127
128 reg [AXI_ADDR_WIDTH-1:0] addr_reg;

```

```

129
130     reg [31:0] pixel_count;
131     reg [31:0] beat_count_total;
132     reg [31:0] burst_count_total;
133
134     reg [7:0] beat_index_in_burst;
135     reg [3:0] pack_index;
136
137     reg [AXI_DATA_WIDTH-1:0] pack_reg;
138
139     //
=====
140     // AXI constant signals
141     //
=====
142     assign m_axi_awid      = AXI_ID_VALUE;
143     assign m_axi_awlen     = BURST_LEN - 1;
144     assign m_axi_awsiz     = axi_size_from_bytes(BYTES_PER_BEAT);
145     assign m_axi_awburst   = 2'b01;           // INCR burst
146     assign m_axi_awlock    = 1'b0;
147     assign m_axi_awcache   = 4'b0011;
148     assign m_axi_awprot    = 3'b000;
149     assign m_axi_awqos     = 4'b0000;
150
151     assign m_axi_wstrb     = {BYTES_PER_BEAT{1'b1}};
152
153     //
=====
154     // Main FSM
155     //
=====
156     always @(posedge clk) begin
157         if (rst) begin
158             state                <= S_IDLE;
159
160             fifo_rd_en           <= 1'b0;
161
162             m_axi_awaddr         <= {AXI_ADDR_WIDTH{1'b0}};
163             m_axi_awvalid        <= 1'b0;
164
165             m_axi_wdata          <= {AXI_DATA_WIDTH{1'b0}};

```

```

166         m_axi_wlast          <= 1'b0;
167         m_axi_wvalid         <= 1'b0;
168
169         m_axi_bready          <= 1'b0;
170
171         busy                   <= 1'b0;
172         done                   <= 1'b0;
173         error                   <= 1'b0;
174         debug_state            <= S_IDLE;
175
176         addr_reg                <= {AXI_ADDR_WIDTH{1'b0}};
177
178         pixel_count             <= 32'd0;
179         beat_count_total        <= 32'd0;
180         burst_count_total       <= 32'd0;
181
182         beat_index_in_burst     <= 8'd0;
183         pack_index              <= 4'd0;
184         pack_reg                <= {AXI_DATA_WIDTH{1'b0}};
185
186         debug_pixel_count       <= 32'd0;
187         debug_beat_count        <= 32'd0;
188         debug_burst_count       <= 32'd0;
189         debug_current_addr      <= {AXI_ADDR_WIDTH{1'b0}};
190     end else begin
191         // Default pulse
192         fifo_rd_en <= 1'b0;
193         done       <= 1'b0;
194         debug_state <= state;
195
196     case (state)
197
198         S_IDLE: begin
199             busy          <= 1'b0;
200             error          <= 1'b0;
201
202             m_axi_awvalid <= 1'b0;
203             m_axi_wvalid  <= 1'b0;
204             m_axi_wlast   <= 1'b0;
205             m_axi_bready  <= 1'b0;
206

```

```

207         fifo_rd_en      <= 1'b0;
208
209         if (enable && frame_start && !
fifo_rd_rst_busy) begin
210             busy          <= 1'b1;
211
212             addr_reg       <= frame_base_addr
;
213             m_axi_awaddr   <= frame_base_addr
;
214             debug_current_addr <= frame_base_addr
;
215
216             pixel_count    <= 32'd0;
217             beat_count_total <= 32'd0;
218             burst_count_total <= 32'd0;
219
220             beat_index_in_burst <= 8'd0;
221             pack_index      <= 4'd0;
222             pack_reg        <= {AXI_DATA_WIDTH
{1'b0}}};
223
224             debug_pixel_count <= 32'd0;
225             debug_beat_count  <= 32'd0;
226             debug_burst_count <= 32'd0;
227
228             state          <= S_WAIT_FIFO;
229         end
230     end
231
232     S_WAIT_FIFO: begin
233         if (!enable) begin
234             state <= S_IDLE;
235         end else if (fifo_rd_rst_busy) begin
236             state <= S_WAIT_FIFO;
237         end else if (beat_count_total >= FRAME_BEATS
) begin
238             state <= S_DONE;
239         end else if (fifo_rd_data_count >=
PIXELS_PER_BURST) begin
240             m_axi_awaddr    <= addr_reg;

```

```

241         debug_current_addr <= addr_reg;
242         m_axi_awvalid      <= 1'b1;
243         state              <= S_AW;
244     end
245 end
246
247 S_AW: begin
248     if (m_axi_awvalid && m_axi_awready) begin
249         m_axi_awvalid      <= 1'b0;
250
251         beat_index_in_burst <= 8'd0;
252         pack_index          <= 4'd0;
253         pack_reg            <= {AXI_DATA_WIDTH
{1'b0}}};
254
255         state              <= S_PACK;
256     end
257 end
258
259 S_PACK: begin
260     if (!enable) begin
261         state <= S_IDLE;
262     end else if (fifo_valid && !fifo_empty && !
fifo_rd_rst_busy) begin
263         fifo_rd_en <= 1'b1;
264
265         pack_reg[pack_index*PIXEL_BITS +:
PIXEL_BITS] <= fifo_dout;
266         pixel_count <= pixel_count + 32'd1;
267
268         if (pack_index == PIXELS_PER_BEAT - 1)
begin
269             // Kirim beat yang sudah lengkap.
270             // Karena assignment non-blocking,
slice terakhir
271             // ditulis langsung juga ke
m_axi_wdata.
272             m_axi_wdata <= pack_reg;
273             m_axi_wdata[pack_index*PIXEL_BITS +:
PIXEL_BITS] <= fifo_dout;
274

```

```

275             m_axi_wvalid <= 1'b1;
276             m_axi_wlast  <= (beat_index_in_burst
== BURST_LEN - 1);
277
278             pack_index   <= 4'd0;
279             state        <= S_W;
280         end else begin
281             pack_index <= pack_index + 4'd1;
282         end
283     end
284 end
285
286 S_W: begin
287     if (m_axi_wvalid && m_axi_wready) begin
288         m_axi_wvalid <= 1'b0;
289         m_axi_wlast  <= 1'b0;
290
291         beat_count_total <= beat_count_total +
32'd1;
292
293         if (beat_index_in_burst == BURST_LEN -
1) begin
294             beat_index_in_burst <= 8'd0;
295             m_axi_bready        <= 1'b1;
296             state               <= S_B;
297         end else begin
298             beat_index_in_burst <=
beat_index_in_burst + 8'd1;
299             pack_reg            <= {
AXI_DATA_WIDTH{1'b0}};
300             pack_index         <= 4'd0;
301             state              <= S_PACK;
302         end
303     end
304 end
305
306 S_B: begin
307     if (m_axi_bvalid && m_axi_bready) begin
308         m_axi_bready <= 1'b0;
309
310         if (m_axi_bresp != 2'b00) begin

```



```

311             error <= 1'b1;
312             state <= S_ERROR;
313         end else begin
314             burst_count_total <=
burst_count_total + 32'd1;
315             addr_reg           <= addr_reg +
ADDR_STEP_BURST;
316             debug_current_addr <= addr_reg +
ADDR_STEP_BURST;
317
318             if (beat_count_total >= FRAME_BEATS)
begin
319                 state <= S_DONE;
320             end else begin
321                 state <= S_WAIT_FIFO;
322             end
323         end
324     end
325 end
326
327 S_DONE: begin
328     busy <= 1'b0;
329     done <= 1'b1;
330
331     debug_pixel_count <= pixel_count;
332     debug_beat_count  <= beat_count_total;
333     debug_burst_count <= burst_count_total;
334
335     state <= S_IDLE;
336 end
337
338 S_ERROR: begin
339     busy <= 1'b0;
340     done <= 1'b0;
341
342     m_axi_awvalid <= 1'b0;
343     m_axi_wvalid  <= 1'b0;
344     m_axi_wlast   <= 1'b0;
345     m_axi_bready  <= 1'b0;
346     fifo_rd_en    <= 1'b0;
347

```

```

348         debug_pixel_count <= pixel_count;
349         debug_beat_count  <= beat_count_total;
350         debug_burst_count <= burst_count_total;
351
352         // Tetap di error sampai reset.
353         state <= S_ERROR;
354     end
355
356     default: begin
357         state <= S_IDLE;
358     end
359
360 endcase
361 end
362 end
363
364 endmodule
365
366

```

Kode AXI Write Yang akan membaca *FIFO Camera* dan menulis ke *DDR3 SDRAM*

L.6 Kode Verilog *Base Address Controller*

```

1
2 `timescale 1ns / 1ps
3
4 module base_address_controller #(
5     parameter ADDR_WIDTH = 32,
6
7     parameter [ADDR_WIDTH-1:0] BUFFER0_BASE_ADDR = 32'h8000_0000
8     ,
9     parameter [ADDR_WIDTH-1:0] BUFFER1_BASE_ADDR = 32'h8010_0000
10    ,
11    parameter [31:0] FRAME_BYTES = 32'd614400
12 ) (
13     input  wire          clk ,
14     input  wire          rst ,
15
16     input  wire          enable ,
17     input  wire          init_calib_complete ,

```

```

17
18     input  wire          write_start_allowed ,
19     input  wire          read_start_allowed ,
20
21     input  wire          axi_write_busy ,
22     input  wire          axi_write_done ,
23     input  wire          axi_write_error ,
24
25     input  wire          axi_read_busy ,
26     input  wire          axi_read_done ,
27     input  wire          axi_read_error ,
28
29     output wire          axi_write_enable ,
30     output reg          axi_write_frame_start ,
31     output reg [ADDR_WIDTH-1:0] axi_write_frame_base_addr ,
32
33     output wire          axi_read_enable ,
34     output reg          axi_read_frame_start ,
35     output reg [ADDR_WIDTH-1:0] axi_read_frame_base_addr ,
36
37     output reg [3:0]      debug_state ,
38     output reg          debug_write_buffer_index ,
39     output reg          debug_read_buffer_index ,
40     output reg          debug_first_frame_valid ,
41     output reg          debug_error_latched ,
42     output reg [31:0]    debug_write_frame_count ,
43     output reg [31:0]    debug_read_frame_count ,
44     output wire          write_accept_ready
45 );
46
47     localparam S_IDLE   = 4'd0;
48     localparam S_RUN    = 4'd1;
49     localparam S_ERROR  = 4'd15;
50
51     reg write_buf;
52     reg read_buf;
53
54     reg pending_swap;
55     reg pending_buf;
56
57     reg axi_write_done_d;

```

```

58     reg axi_read_done_d;
59
60     wire axi_write_done_pulse;
61     wire axi_read_done_pulse;
62
63     assign axi_write_done_pulse = axi_write_done & ~
axi_write_done_d;
64     assign axi_read_done_pulse  = axi_read_done  & ~
axi_read_done_d;
65
66     assign axi_write_enable =
67         enable &&
68         init_calib_complete &&
69         !debug_error_latched;
70
71     assign axi_read_enable =
72         enable &&
73         init_calib_complete &&
74         debug_first_frame_valid &&
75         !debug_error_latched;
76
77     assign write_accept_ready =
78         enable &&
79         init_calib_complete &&
80         !debug_error_latched;
81
82     function [ADDR_WIDTH-1:0] buffer_base_addr;
83         input buffer_index;
84         begin
85             if (buffer_index == 1'b0)
86                 buffer_base_addr = BUFFER0_BASE_ADDR;
87             else
88                 buffer_base_addr = BUFFER1_BASE_ADDR;
89         end
90     endfunction
91
92     always @(posedge clk) begin
93         if (rst) begin
94             debug_state                <= S_IDLE;
95
96             write_buf                  <= 1'b0;

```

```

97         read_buf                                     <= 1'b0;
98
99         pending_swap                                 <= 1'b0;
100        pending_buf                                   <= 1'b0;
101
102        axi_write_frame_start                         <= 1'b0;
103        axi_read_frame_start                         <= 1'b0;
104
105        axi_write_frame_base_addr <= BUFFER0_BASE_ADDR;
106        axi_read_frame_base_addr <= BUFFER0_BASE_ADDR;
107
108        axi_write_done_d                             <= 1'b0;
109        axi_read_done_d                             <= 1'b0;
110
111        debug_write_buffer_index <= 1'b0;
112        debug_read_buffer_index  <= 1'b0;
113        debug_first_frame_valid  <= 1'b0;
114        debug_error_latched      <= 1'b0;
115        debug_write_frame_count  <= 32'd0;
116        debug_read_frame_count   <= 32'd0;
117    end else begin
118        axi_write_frame_start <= 1'b0;
119        axi_read_frame_start  <= 1'b0;
120
121        axi_write_done_d <= axi_write_done;
122        axi_read_done_d  <= axi_read_done;
123
124        if (!enable || !init_calib_complete) begin
125            debug_state                                     <= S_IDLE;
126
127            write_buf                                     <= 1'b0;
128            read_buf                                     <= 1'b0;
129
130            pending_swap                                 <= 1'b0;
131            pending_buf                                   <= 1'b0;
132
133            axi_write_frame_base_addr <= BUFFER0_BASE_ADDR;
134            axi_read_frame_base_addr  <= BUFFER0_BASE_ADDR;
135
136            debug_write_buffer_index <= 1'b0;
137            debug_read_buffer_index  <= 1'b0;

```

```

138         debug_first_frame_valid    <= 1'b0;
139         debug_error_latched        <= 1'b0;
140     end else begin
141         if (axi_write_error || axi_read_error) begin
142             debug_error_latched <= 1'b1;
143             debug_state         <= S_ERROR;
144         end else begin
145             case (debug_state)
146
147                 S_IDLE: begin
148                     write_buf          <= 1'b0;
149                     read_buf           <= 1'b0;
150
151                     pending_swap       <= 1'b0;
152                     pending_buf        <= 1'b0;
153
154                     axi_write_frame_base_addr <=
155 BUFFER0_BASE_ADDR;
156
157                     axi_read_frame_base_addr <=
158 BUFFER0_BASE_ADDR;
159
160                     debug_write_buffer_index <= 1'b0;
161                     debug_read_buffer_index  <= 1'b0;
162                     debug_first_frame_valid  <= 1'b0;
163
164                     debug_state <= S_RUN;
165                 end
166
167                 S_RUN: begin
168                     //
169                     =====
170                     // WRITE SIDE
171                     // Start write hanya saat awal frame
172 kamera.
173
174                     // Kalau pending_swap masih 1,
175 berarti frame baru
176                     // belum sempat diambil oleh VGA,
177 jadi jangan overwrite.
178                     //
179                     =====
180                     if (write_start_allowed &&

```

```

172         !axi_write_busy &&
173         !pending_swap) begin
174
175         axi_write_frame_base_addr <=
buffer_base_addr( write_buf);
176         axi_write_frame_start      <= 1'
b1;
177     end
178
179     // Write selesai:
180     // frame baru sudah siap, tapi BELUM
swap.
181     // Swap harus menunggu awal frame
VGA.
182     if (axi_write_done_pulse) begin
183         pending_swap      <= 1'b1;
184         pending_buf      <=
write_buf;
185         debug_first_frame_valid <= 1'b1;
186         debug_write_frame_count <=
debug_write_frame_count + 32'd1;
187     end
188
189     //
=====
190     // READ SIDE
191     // Start read hanya saat awal frame
VGA.
192     //
=====
193     if (read_start_allowed &&
194         !axi_read_busy &&
195         debug_first_frame_valid) begin
196
197         if (pending_swap) begin
198             // Ambil buffer yang baru
selesai ditulis
199             read_buf <= pending_buf;
200
201             axi_read_frame_base_addr <=
buffer_base_addr(pending_buf);

```

```

202             axi_read_frame_start      <=
1'b1;

203
204             // Setelah buffer pending
dipakai untuk read,
205             // buffer satunya menjadi
target write berikutnya.
206             write_buf <= ~pending_buf;
207
208             pending_swap <= 1'b0;
209         end else begin
210             // Tidak ada frame baru.
211             // Ulangi frame lama supaya
VGA tetap stabil.
212             axi_read_frame_base_addr <=
buffer_base_addr(read_buf);
213             axi_read_frame_start      <=
1'b1;
214         end
215     end
216
217     if (axi_read_done_pulse) begin
218         debug_read_frame_count <=
debug_read_frame_count + 32'd1;
219     end
220
221     debug_write_buffer_index <=
write_buf;
222     debug_read_buffer_index  <= read_buf
;
223 end
224
225 S_ERROR: begin
226     axi_write_frame_start <= 1'b0;
227     axi_read_frame_start  <= 1'b0;
228     debug_error_latched   <= 1'b1;
229     debug_state            <= S_ERROR;
230 end
231
232 default: begin
233     debug_state <= S_IDLE;

```



```

234                                     end
235
236                                     endcase
237                                 end
238                            end
239                        end
240                    end
241
242    endmodule
243
244

```

Kode *Base Address Controller* untuk mengatur *double buffer memory*

L.7 Kode Verilog *Camera Capture*

```

1
2  `timescale 1ns / 1ps
3
4  module camera_read(
5      input  wire      p_clock ,
6      input  wire      vsync ,
7      input  wire      href ,
8      input  wire [7:0] p_data ,
9
10     output reg [15:0] pixel_data  = 16'd0 ,
11     output reg          pixel_valid = 1'b0 ,
12     output reg          frame_done  = 1'b0 ,
13
14     output reg [9:0]  cam_x        = 10'd0 ,
15     output reg [9:0]  cam_y        = 10'd0 ,
16     output reg          frame_start = 1'b0
17 );
18
19     localparam WAIT_FRAME_START = 2'd0;
20     localparam ROW_CAPTURE      = 2'd1;
21
22     reg [1:0] FSM_state = WAIT_FRAME_START;
23     reg          pixel_half = 1'b0;
24
25     reg [9:0] x_cnt = 10'd0;
26     reg [9:0] y_cnt = 10'd0;

```

```

27
28 // =====
29 // ROI Border 280x280 Center
30 // Frame camera diasumsikan 640x480
31 // ROI: x = 180..459, y = 100..379
32 // =====
33 localparam [9:0] ROI_X0 = 10'd180;
34 localparam [9:0] ROI_Y0 = 10'd100;
35 localparam [9:0] ROI_X1 = 10'd459;
36 localparam [9:0] ROI_Y1 = 10'd379;
37
38 localparam [9:0] BORDER_THICK = 10'd2;
39
40 wire in_roi_x;
41 wire in_roi_y;
42 wire roi_border;
43
44 assign in_roi_x = (x_cnt >= ROI_X0) && (x_cnt <= ROI_X1);
45 assign in_roi_y = (y_cnt >= ROI_Y0) && (y_cnt <= ROI_Y1);
46
47 assign roi_border =
48     in_roi_x &&
49     in_roi_y &&
50     (
51         (x_cnt < ROI_X0 + BORDER_THICK) ||
52         (x_cnt > ROI_X1 - BORDER_THICK) ||
53         (y_cnt < ROI_Y0 + BORDER_THICK) ||
54         (y_cnt > ROI_Y1 - BORDER_THICK)
55     );
56
57 always @(posedge p_clock) begin
58     // default pulse
59     pixel_valid <= 1'b0;
60     frame_start <= 1'b0;
61     frame_done <= 1'b0;
62
63     case (FSM_state)
64
65         WAIT_FRAME_START: begin
66             pixel_half <= 1'b0;
67

```

```

68         x_cnt <= 10'd0;
69         y_cnt <= 10'd0;
70         cam_x <= 10'd0;
71         cam_y <= 10'd0;
72
73         // OV7670 aktif capture ketika VSYNC low
74         if (!vsync) begin
75             FSM_state <= ROW_CAPTURE;
76             frame_start <= 1'b1;    // benar-benar pulse
77         end
78     end
79
80     ROW_CAPTURE: begin
81         if (vsync) begin
82             FSM_state <= WAIT_FRAME_START;
83             frame_done <= 1'b1;    // pulse 1 p_clock
84             saat frame selesai
85
86             pixel_half <= 1'b0;
87             x_cnt <= 10'd0;
88             y_cnt <= 10'd0;
89             cam_x <= 10'd0;
90             cam_y <= 10'd0;
91         end else begin
92             if (href) begin
93                 pixel_half <= ~pixel_half;
94
95                 if (!pixel_half) begin
96                     // byte pertama RGB565
97                     pixel_data[15:8] <= p_data;
98                 end else begin
99                     // byte kedua RGB565
100                     if (roi_border) begin
101                         // Green ROI border RGB565
102                         pixel_data <= 16'h07E0;
103                     end else begin
104                         pixel_data[7:0] <= p_data;
105                     end
106
107                     pixel_valid <= 1'b1;

```

```

107
108         cam_x <= x_cnt;
109         cam_y <= y_cnt;
110
111         if (x_cnt == 10'd639) begin
112             x_cnt <= 10'd0;
113
114             if (y_cnt == 10'd479)
115                 y_cnt <= 10'd0;
116             else
117                 y_cnt <= y_cnt + 1'b1;
118             end else begin
119                 x_cnt <= x_cnt + 1'b1;
120             end
121         end
122     end else begin
123         // penting supaya byte alignment tidak
124         geser antar baris
125         pixel_half <= 1'b0;
126     end
127 end
128
129     default: begin
130         FSM_state <= WAIT_FRAME_START;
131     end
132
133 endcase
134 end
135
136 endmodule
137
138

```

Kode Pixel Capture untuk OV7670

L.8 Kode Verilog VGA Stream

```

1
2 `timescale 1ns / 1ps
3
4 module vga_from_fifo_rgb565 (

```

```

5     input  wire          vga_clk ,
6     input  wire          rst ,
7
8     input  wire [15:0] fifo_dout ,
9     input  wire          fifo_empty ,
10    input  wire          fifo_valid ,
11    input  wire          fifo_rd_rst_busy ,
12    output wire          fifo_rd_en ,
13
14    input  wire [3:0]  class_out ,
15
16    // =====
17    // GRAYSCALE MODE SWITCH
18    // -----
19    // 0 = invert grayscale
20    // 1 = red threshold mode
21    // =====
22    input  wire          sw_gray_scale ,
23
24    output reg [3:0]  vga_r ,
25    output reg [3:0]  vga_g ,
26    output reg [3:0]  vga_b ,
27    output reg          vga_hs ,
28    output reg          vga_vs ,
29
30    output reg          vga_vblank_start = 1'b0,
31    output reg          vga_frame_start  = 1'b0
32 );
33
34    // =====
35    // VGA 640x480 @ 60 Hz Timing
36    // =====
37    localparam H_VISIBLE = 640;
38    localparam H_FRONT   = 16;
39    localparam H_SYNC     = 96;
40    localparam H_BACK     = 48;
41    localparam H_TOTAL    = H_VISIBLE + H_FRONT + H_SYNC + H_BACK
42    ;
43
44    localparam V_VISIBLE = 480;
45    localparam V_FRONT   = 10;

```

```

45     localparam V_SYNC      = 2;
46     localparam V_BACK      = 33;
47     localparam V_TOTAL     = V_VISIBLE + V_FRONT + V_SYNC + V_BACK
;

48
49     reg [9:0] h_cnt = 10'd0;
50     reg [9:0] v_cnt = 10'd0;
51
52     wire active_video;
53     assign active_video = (h_cnt < H_VISIBLE) && (v_cnt <
V_VISIBLE);

54
55     always @(posedge vga_clk) begin
56         if (rst) begin
57             h_cnt <= 10'd0;
58             v_cnt <= 10'd0;
59         end else begin
60             if (h_cnt == H_TOTAL - 1) begin
61                 h_cnt <= 10'd0;
62
63                 if (v_cnt == V_TOTAL - 1)
64                     v_cnt <= 10'd0;
65                 else
66                     v_cnt <= v_cnt + 1'b1;
67             end else begin
68                 h_cnt <= h_cnt + 1'b1;
69             end
70         end
71     end

72
73     // =====
74     // VGA sync active-low
75     // =====
76     always @(posedge vga_clk) begin
77         if (rst) begin
78             vga_hs <= 1'b1;
79             vga_vs <= 1'b1;
80         end else begin
81             vga_hs <= ~((h_cnt >= H_VISIBLE + H_FRONT) &&
82                 (h_cnt <  H_VISIBLE + H_FRONT + H_SYNC))
;

```

```

83
84         vga_vs <= ~((v_cnt >= V_VISIBLE + V_FRONT) &&
85                     (v_cnt < V_VISIBLE + V_FRONT + V_SYNC))
86     ;
87     end
88
89     always @(posedge vga_clk) begin
90         if (rst) begin
91             vga_vblank_start <= 1'b0;
92             vga_frame_start  <= 1'b0;
93         end else begin
94             vga_vblank_start <= 1'b0;
95             vga_frame_start  <= 1'b0;
96
97             if ((h_cnt == 10'd0) && (v_cnt == 10'd0))
98                 vga_frame_start <= 1'b1;
99
100            if ((h_cnt == 10'd0) && (v_cnt == V_VISIBLE))
101                vga_vblank_start <= 1'b1;
102        end
103    end
104
105    // =====
106    // FIFO Read
107    // =====
108    assign fifo_rd_en = active_video &&
109                    !fifo_empty &&
110                    !fifo_rd_rst_busy;
111
112    // =====
113    // RGB565 to RGB444 untuk tampilan kamera normal
114    // =====
115    wire [3:0] pixel_r;
116    wire [3:0] pixel_g;
117    wire [3:0] pixel_b;
118
119    assign pixel_r = fifo_valid ? fifo_dout[15:12] : 4'h0;
120    assign pixel_g = fifo_valid ? fifo_dout[10:7]  : 4'h0;
121    assign pixel_b = fifo_valid ? fifo_dout[4:1]   : 4'h0;
122

```

```

123 // =====
124 // Synchronize sw_gray_scale to VGA clock domain
125 // =====
126 (* ASYNC_REG = "TRUE" *) reg sw_meta = 1'b0;
127 (* ASYNC_REG = "TRUE" *) reg sw_vga = 1'b0;
128
129 always @(posedge vga_clk) begin
130     if (rst) begin
131         sw_meta <= 1'b0;
132         sw_vga <= 1'b0;
133     end else begin
134         sw_meta <= sw_gray_scale;
135         sw_vga <= sw_meta;
136     end
137 end
138
139 // =====
140 // MINI ROI GENERATOR
141 // =====
142 localparam SRC_ROI_SIZE = 10'd280;
143 localparam SRC_ROI_X0 = 10'd180;
144 localparam SRC_ROI_Y0 = 10'd100;
145
146 wire src_roi_area;
147 assign src_roi_area = active_video &&
148                     fifo_valid &&
149                     (h_cnt >= SRC_ROI_X0) &&
150                     (h_cnt < SRC_ROI_X0 + SRC_ROI_SIZE)
151 &&
152                     (v_cnt >= SRC_ROI_Y0) &&
153                     (v_cnt < SRC_ROI_Y0 + SRC_ROI_SIZE);
154
155 wire [9:0] src_roi_local_x;
156 wire [9:0] src_roi_local_y;
157
158 assign src_roi_local_x = h_cnt - SRC_ROI_X0;
159 assign src_roi_local_y = v_cnt - SRC_ROI_Y0;
160
161 wire [5:0] pick_x;
162 wire [5:0] pick_y;

```



```

163     assign pick_x = pick_280_to_28(src_roi_local_x);
164     assign pick_y = pick_280_to_28(src_roi_local_y);
165
166     wire sample_x_valid;
167     wire sample_y_valid;
168     wire [4:0] sample_x;
169     wire [4:0] sample_y;
170
171     assign sample_x_valid = pick_x[5];
172     assign sample_y_valid = pick_y[5];
173     assign sample_x      = pick_x[4:0];
174     assign sample_y      = pick_y[4:0];
175
176     wire write_sample;
177     assign write_sample = src_roi_area && sample_x_valid &&
sample_y_valid;
178
179     wire [9:0] sample_y_ext;
180     wire [9:0] sample_x_ext;
181     wire [9:0] mini_wr_addr;
182
183     assign sample_y_ext = {5'd0, sample_y};
184     assign sample_x_ext = {5'd0, sample_x};
185
186     assign mini_wr_addr = (sample_y_ext << 5) -
187                          (sample_y_ext << 2) +
188                          sample_x_ext;
189
190     // =====
191     // RGB565 -> grayscale mode
192     // =====
193     wire [4:0] r5;
194     wire [5:0] g6;
195     wire [4:0] b5;
196
197     assign r5 = fifo_dout[15:11];
198     assign g6 = fifo_dout[10:5];
199     assign b5 = fifo_dout[4:0];
200
201     wire [7:0] r8;
202     wire [7:0] g8;

```

```

203     wire [7:0] b8;
204
205     assign r8 = {r5, r5[4:2]};
206     assign g8 = {g6, g6[5:4]};
207     assign b8 = {b5, b5[4:2]};
208
209     // Mode 0: invert grayscale
210     wire [7:0] invert_gray8;
211     assign invert_gray8 = 8'hFF - g8;
212
213     // Mode 1: red threshold mode
214     localparam [7:0] RED_LOW_TH    = 8'd64;
215     localparam [7:0] RED_HIGH_TH   = 8'd192;
216     localparam [7:0] MID_LOW_OUT   = 8'd100;
217     localparam [7:0] MID_HIGH_OUT  = 8'd200;
218
219     wire [7:0] red_delta;
220     wire [15:0] red_mul100;
221     wire [7:0] red_scaled_0_99;
222     reg [7:0] red_gray8;
223
224     assign red_delta = r8 - RED_LOW_TH;
225
226     assign red_mul100 = ({8'd0, red_delta} << 6) +
227                        ({8'd0, red_delta} << 5) +
228                        ({8'd0, red_delta} << 2);
229
230     assign red_scaled_0_99 = red_mul100[14:7];
231
232     always @(*) begin
233         if (r8 < RED_LOW_TH) begin
234             red_gray8 = 8'd0;
235         end else if (r8 >= RED_HIGH_TH) begin
236             red_gray8 = 8'd255;
237         end else begin
238             red_gray8 = MID_LOW_OUT + red_scaled_0_99;
239
240             if (red_gray8 > MID_HIGH_OUT)
241                 red_gray8 = MID_HIGH_OUT;
242         end
243     end

```

```

244
245     wire [7:0] selected_gray8;
246     assign selected_gray8 = (sw_vga == 1'b0) ? invert_gray8 :
red_gray8;
247
248     // =====
249     // Mini buffer 28x28
250     // =====
251     localparam MINI_SIZE      = 10'd28;
252     localparam MINI_MEM_SIZE = 784;
253
254     (* ram_style = "distributed" *) reg [7:0] mini_mem [0:
MINI_MEM_SIZE-1];
255     reg mini_valid = 1'b0;
256
257     always @(posedge vga_clk) begin
258         if (rst) begin
259             mini_valid <= 1'b0;
260         end else begin
261             if (write_sample) begin
262                 mini_mem[mini_wr_addr] <= selected_gray8;
263
264                 if ((sample_x == 5'd27) && (sample_y == 5'd27))
265                     mini_valid <= 1'b1;
266             end
267         end
268     end
269
270     // =====
271     // Area tampilan mini ROI kanan-bawah
272     // =====
273     localparam MINI_MARGIN = 10'd16;
274     localparam MINI_X0     = 10'd596;
275     localparam MINI_Y0     = 10'd436;
276
277     wire mini_area;
278     assign mini_area = active_video &&
279                     (h_cnt >= MINI_X0) &&
280                     (h_cnt <  MINI_X0 + MINI_SIZE) &&
281                     (v_cnt >= MINI_Y0) &&
282                     (v_cnt <  MINI_Y0 + MINI_SIZE);

```

```

283
284     wire [4:0] mini_x;
285     wire [4:0] mini_y;
286     wire [9:0] mini_x_ext;
287     wire [9:0] mini_y_ext;
288     wire [9:0] mini_rd_addr;
289
290     assign mini_x = h_cnt - MINI_X0;
291     assign mini_y = v_cnt - MINI_Y0;
292
293     assign mini_x_ext = {5'd0, mini_x};
294     assign mini_y_ext = {5'd0, mini_y};
295
296     assign mini_rd_addr = (mini_y_ext << 5) -
297                           (mini_y_ext << 2) +
298                           mini_x_ext;
299
300     wire [7:0] mini_pixel8;
301     wire [3:0] mini_gray4;
302
303     assign mini_pixel8 = mini_valid ? mini_mem[mini_rd_addr] :
304     8'd0;
305
306     // =====
307     // Synchronize class_out to VGA clock domain
308     // =====
309     reg [3:0] class_meta = 4'd0;
310     reg [3:0] class_vga  = 4'd0;
311
312     always @(posedge vga_clk) begin
313         if (rst) begin
314             class_meta <= 4'd0;
315             class_vga  <= 4'd0;
316         end else begin
317             class_meta <= class_out;
318             class_vga  <= class_meta;
319         end
320     end
321
322     // =====

```

```

323 // Text Overlay: "PREDICTED: X"
324 // =====
325 localparam TEXT_SCALE = 2;
326 localparam FONT_W     = 8;
327 localparam FONT_H     = 8;
328 localparam CHAR_W     = FONT_W * TEXT_SCALE;
329 localparam CHAR_H     = FONT_H * TEXT_SCALE;
330 localparam TEXT_LEN   = 12;
331
332 localparam TEXT_X0 = 16;
333 localparam TEXT_Y0 = 16;
334
335 wire text_area;
336 assign text_area = active_video &&
337                  (h_cnt >= TEXT_X0) &&
338                  (h_cnt < TEXT_X0 + TEXT_LEN * CHAR_W) &&
339                  (v_cnt >= TEXT_Y0) &&
340                  (v_cnt < TEXT_Y0 + CHAR_H);
341
342 wire [9:0] text_local_x;
343 wire [9:0] text_local_y;
344
345 assign text_local_x = h_cnt - TEXT_X0;
346 assign text_local_y = v_cnt - TEXT_Y0;
347
348 wire [3:0] text_char_idx;
349 wire [2:0] font_col;
350 wire [2:0] font_row;
351
352 assign text_char_idx = text_local_x [9:4];
353 assign font_col      = text_local_x [3:1];
354 assign font_row      = text_local_y [3:1];
355
356 wire [7:0] current_char;
357 wire [7:0] font_bits;
358 wire      font_pixel;
359
360 assign current_char = get_text_char(text_char_idx , class_vga
);
361 assign font_bits    = font8x8(current_char , font_row);
362 assign font_pixel   = text_area && ((font_bits & (8'h80 >>

```

```

font_col)) != 8'd0));

363
364 // =====
365 // Final Pixel Priority
366 // =====
367 always @(posedge vga_clk) begin
368     if (rst) begin
369         vga_r <= 4'h0;
370         vga_g <= 4'h0;
371         vga_b <= 4'h0;
372     end else begin
373         if (!active_video) begin
374             vga_r <= 4'h0;
375             vga_g <= 4'h0;
376             vga_b <= 4'h0;
377         end else if (mini_area) begin
378             vga_r <= mini_gray4;
379             vga_g <= mini_gray4;
380             vga_b <= mini_gray4;
381         end else if (font_pixel) begin
382             vga_r <= 4'hF;
383             vga_g <= 4'hF;
384             vga_b <= 4'hF;
385         end else begin
386             vga_r <= pixel_r;
387             vga_g <= pixel_g;
388             vga_b <= pixel_b;
389         end
390     end
391 end

392
393 // =====
394 // Function: mapping 280 -> 28
395 // =====
396 function [5:0] pick_280_to_28;
397     input [9:0] v;
398     begin
399         case (v)
400             10'd0:    pick_280_to_28 = {1'b1, 5'd0};
401             10'd10:   pick_280_to_28 = {1'b1, 5'd1};
402             10'd20:   pick_280_to_28 = {1'b1, 5'd2};

```

```

403         10'd30: pick_280_to_28 = {1'b1, 5'd3};
404         10'd40: pick_280_to_28 = {1'b1, 5'd4};
405         10'd50: pick_280_to_28 = {1'b1, 5'd5};
406         10'd60: pick_280_to_28 = {1'b1, 5'd6};
407         10'd70: pick_280_to_28 = {1'b1, 5'd7};
408         10'd80: pick_280_to_28 = {1'b1, 5'd8};
409         10'd90: pick_280_to_28 = {1'b1, 5'd9};
410         10'd100: pick_280_to_28 = {1'b1, 5'd10};
411         10'd110: pick_280_to_28 = {1'b1, 5'd11};
412         10'd120: pick_280_to_28 = {1'b1, 5'd12};
413         10'd130: pick_280_to_28 = {1'b1, 5'd13};
414         10'd140: pick_280_to_28 = {1'b1, 5'd14};
415         10'd150: pick_280_to_28 = {1'b1, 5'd15};
416         10'd160: pick_280_to_28 = {1'b1, 5'd16};
417         10'd170: pick_280_to_28 = {1'b1, 5'd17};
418         10'd180: pick_280_to_28 = {1'b1, 5'd18};
419         10'd190: pick_280_to_28 = {1'b1, 5'd19};
420         10'd200: pick_280_to_28 = {1'b1, 5'd20};
421         10'd210: pick_280_to_28 = {1'b1, 5'd21};
422         10'd220: pick_280_to_28 = {1'b1, 5'd22};
423         10'd230: pick_280_to_28 = {1'b1, 5'd23};
424         10'd240: pick_280_to_28 = {1'b1, 5'd24};
425         10'd250: pick_280_to_28 = {1'b1, 5'd25};
426         10'd260: pick_280_to_28 = {1'b1, 5'd26};
427         10'd270: pick_280_to_28 = {1'b1, 5'd27};
428         default: pick_280_to_28 = {1'b0, 5'd0};
429     endcase
430 end
431 endfunction
432
433 // =====
434 // Text Character Selector: "PREDICTED: X"
435 // =====
436 function [7:0] get_text_char;
437     input [3:0] idx;
438     input [3:0] digit;
439     begin
440         case (idx)
441             4'd0: get_text_char = 8'h50;
442             4'd1: get_text_char = 8'h52;
443             4'd2: get_text_char = 8'h45;

```

```

444         4'd3:  get_text_char = 8'h44;
445         4'd4:  get_text_char = 8'h49;
446         4'd5:  get_text_char = 8'h43;
447         4'd6:  get_text_char = 8'h54;
448         4'd7:  get_text_char = 8'h45;
449         4'd8:  get_text_char = 8'h44;
450         4'd9:  get_text_char = 8'h3A;
451         4'd10: get_text_char = 8'h20;
452         4'd11: begin
453             if (digit <= 4'd9)
454                 get_text_char = 8'h30 + {4'b0000, digit
};
455             else
456                 get_text_char = 8'h3F;
457         end
458         default: get_text_char = 8'h20;
459     endcase
460 end
461 endfunction
462
463 // =====
464 // Minimal 8x8 Font ROM
465 // =====
466 function [7:0] font8x8;
467     input [7:0] ch;
468     input [2:0] row;
469     begin
470         case (ch)
471
472             8'h20: font8x8 = 8'b00000000;
473
474             8'h3A: begin
475                 case (row)
476                     3'd0: font8x8 = 8'b00000000;
477                     3'd1: font8x8 = 8'b00011000;
478                     3'd2: font8x8 = 8'b00011000;
479                     3'd3: font8x8 = 8'b00000000;
480                     3'd4: font8x8 = 8'b00000000;
481                     3'd5: font8x8 = 8'b00011000;
482                     3'd6: font8x8 = 8'b00011000;
483                     3'd7: font8x8 = 8'b00000000;

```



```

484         endcase
485     end
486
487     8'h3F: begin
488         case (row)
489             3'd0: font8x8 = 8'b00111100;
490             3'd1: font8x8 = 8'b01100110;
491             3'd2: font8x8 = 8'b00000110;
492             3'd3: font8x8 = 8'b00001100;
493             3'd4: font8x8 = 8'b00011000;
494             3'd5: font8x8 = 8'b00000000;
495             3'd6: font8x8 = 8'b00011000;
496             3'd7: font8x8 = 8'b00000000;
497         endcase
498     end
499
500     8'h30: begin
501         case (row)
502             3'd0: font8x8 = 8'b00111100;
503             3'd1: font8x8 = 8'b01100110;
504             3'd2: font8x8 = 8'b01101110;
505             3'd3: font8x8 = 8'b01110110;
506             3'd4: font8x8 = 8'b01100110;
507             3'd5: font8x8 = 8'b01100110;
508             3'd6: font8x8 = 8'b00111100;
509             3'd7: font8x8 = 8'b00000000;
510         endcase
511     end
512
513     8'h31: begin
514         case (row)
515             3'd0: font8x8 = 8'b00011000;
516             3'd1: font8x8 = 8'b00111000;
517             3'd2: font8x8 = 8'b00011000;
518             3'd3: font8x8 = 8'b00011000;
519             3'd4: font8x8 = 8'b00011000;
520             3'd5: font8x8 = 8'b00011000;
521             3'd6: font8x8 = 8'b01111110;
522             3'd7: font8x8 = 8'b00000000;
523         endcase
524     end

```

```

525
526      8'h32: begin
527          case (row)
528              3'd0: font8x8 = 8'b00111100;
529              3'd1: font8x8 = 8'b01100110;
530              3'd2: font8x8 = 8'b00000110;
531              3'd3: font8x8 = 8'b00001100;
532              3'd4: font8x8 = 8'b00110000;
533              3'd5: font8x8 = 8'b01100000;
534              3'd6: font8x8 = 8'b01111110;
535              3'd7: font8x8 = 8'b00000000;
536          endcase
537      end
538
539      8'h33: begin
540          case (row)
541              3'd0: font8x8 = 8'b00111100;
542              3'd1: font8x8 = 8'b01100110;
543              3'd2: font8x8 = 8'b00000110;
544              3'd3: font8x8 = 8'b00011100;
545              3'd4: font8x8 = 8'b00000110;
546              3'd5: font8x8 = 8'b01100110;
547              3'd6: font8x8 = 8'b00111100;
548              3'd7: font8x8 = 8'b00000000;
549          endcase
550      end
551
552      8'h34: begin
553          case (row)
554              3'd0: font8x8 = 8'b00001100;
555              3'd1: font8x8 = 8'b00011100;
556              3'd2: font8x8 = 8'b00101100;
557              3'd3: font8x8 = 8'b01001100;
558              3'd4: font8x8 = 8'b01111110;
559              3'd5: font8x8 = 8'b00001100;
560              3'd6: font8x8 = 8'b00001100;
561              3'd7: font8x8 = 8'b00000000;
562          endcase
563      end
564
565      8'h35: begin

```

```

566         case (row)
567             3'd0: font8x8 = 8'b01111110;
568             3'd1: font8x8 = 8'b01100000;
569             3'd2: font8x8 = 8'b01111100;
570             3'd3: font8x8 = 8'b00000110;
571             3'd4: font8x8 = 8'b00000110;
572             3'd5: font8x8 = 8'b01100110;
573             3'd6: font8x8 = 8'b00111100;
574             3'd7: font8x8 = 8'b00000000;
575         endcase
576     end
577
578     8'h36: begin
579         case (row)
580             3'd0: font8x8 = 8'b00111100;
581             3'd1: font8x8 = 8'b01100110;
582             3'd2: font8x8 = 8'b01100000;
583             3'd3: font8x8 = 8'b01111100;
584             3'd4: font8x8 = 8'b01100110;
585             3'd5: font8x8 = 8'b01100110;
586             3'd6: font8x8 = 8'b00111100;
587             3'd7: font8x8 = 8'b00000000;
588         endcase
589     end
590
591     8'h37: begin
592         case (row)
593             3'd0: font8x8 = 8'b01111110;
594             3'd1: font8x8 = 8'b00000110;
595             3'd2: font8x8 = 8'b00001100;
596             3'd3: font8x8 = 8'b00011000;
597             3'd4: font8x8 = 8'b00110000;
598             3'd5: font8x8 = 8'b00110000;
599             3'd6: font8x8 = 8'b00110000;
600             3'd7: font8x8 = 8'b00000000;
601         endcase
602     end
603
604     8'h38: begin
605         case (row)
606             3'd0: font8x8 = 8'b00111100;

```

```

607         3'd1: font8x8 = 8'b01100110;
608         3'd2: font8x8 = 8'b01100110;
609         3'd3: font8x8 = 8'b00111100;
610         3'd4: font8x8 = 8'b01100110;
611         3'd5: font8x8 = 8'b01100110;
612         3'd6: font8x8 = 8'b00111100;
613         3'd7: font8x8 = 8'b00000000;
614     endcase
615 end
616
617 8'h39: begin
618     case (row)
619         3'd0: font8x8 = 8'b00111100;
620         3'd1: font8x8 = 8'b01100110;
621         3'd2: font8x8 = 8'b01100110;
622         3'd3: font8x8 = 8'b00111110;
623         3'd4: font8x8 = 8'b00000110;
624         3'd5: font8x8 = 8'b01100110;
625         3'd6: font8x8 = 8'b00111100;
626         3'd7: font8x8 = 8'b00000000;
627     endcase
628 end
629
630 8'h43: begin
631     case (row)
632         3'd0: font8x8 = 8'b00111100;
633         3'd1: font8x8 = 8'b01100110;
634         3'd2: font8x8 = 8'b01100000;
635         3'd3: font8x8 = 8'b01100000;
636         3'd4: font8x8 = 8'b01100000;
637         3'd5: font8x8 = 8'b01100110;
638         3'd6: font8x8 = 8'b00111100;
639         3'd7: font8x8 = 8'b00000000;
640     endcase
641 end
642
643 8'h44: begin
644     case (row)
645         3'd0: font8x8 = 8'b01111000;
646         3'd1: font8x8 = 8'b01101100;
647         3'd2: font8x8 = 8'b01100110;

```

```

648             3'd3: font8x8 = 8'b01100110;
649             3'd4: font8x8 = 8'b01100110;
650             3'd5: font8x8 = 8'b01101100;
651             3'd6: font8x8 = 8'b01111000;
652             3'd7: font8x8 = 8'b00000000;
653         endcase
654     end
655
656     8'h45: begin
657         case (row)
658             3'd0: font8x8 = 8'b01111110;
659             3'd1: font8x8 = 8'b01100000;
660             3'd2: font8x8 = 8'b01100000;
661             3'd3: font8x8 = 8'b01111100;
662             3'd4: font8x8 = 8'b01100000;
663             3'd5: font8x8 = 8'b01100000;
664             3'd6: font8x8 = 8'b01111110;
665             3'd7: font8x8 = 8'b00000000;
666         endcase
667     end
668
669     8'h49: begin
670         case (row)
671             3'd0: font8x8 = 8'b01111110;
672             3'd1: font8x8 = 8'b00011000;
673             3'd2: font8x8 = 8'b00011000;
674             3'd3: font8x8 = 8'b00011000;
675             3'd4: font8x8 = 8'b00011000;
676             3'd5: font8x8 = 8'b00011000;
677             3'd6: font8x8 = 8'b01111110;
678             3'd7: font8x8 = 8'b00000000;
679         endcase
680     end
681
682     8'h50: begin
683         case (row)
684             3'd0: font8x8 = 8'b01111100;
685             3'd1: font8x8 = 8'b01100110;
686             3'd2: font8x8 = 8'b01100110;
687             3'd3: font8x8 = 8'b01111100;
688             3'd4: font8x8 = 8'b01100000;

```

```

689         3'd5: font8x8 = 8'b01100000;
690         3'd6: font8x8 = 8'b01100000;
691         3'd7: font8x8 = 8'b00000000;
692     endcase
693 end
694
695 8'h52: begin
696     case (row)
697         3'd0: font8x8 = 8'b01111100;
698         3'd1: font8x8 = 8'b01100110;
699         3'd2: font8x8 = 8'b01100110;
700         3'd3: font8x8 = 8'b01111100;
701         3'd4: font8x8 = 8'b01111000;
702         3'd5: font8x8 = 8'b01101100;
703         3'd6: font8x8 = 8'b01100110;
704         3'd7: font8x8 = 8'b00000000;
705     endcase
706 end
707
708 8'h54: begin
709     case (row)
710         3'd0: font8x8 = 8'b01111110;
711         3'd1: font8x8 = 8'b00011000;
712         3'd2: font8x8 = 8'b00011000;
713         3'd3: font8x8 = 8'b00011000;
714         3'd4: font8x8 = 8'b00011000;
715         3'd5: font8x8 = 8'b00011000;
716         3'd6: font8x8 = 8'b00011000;
717         3'd7: font8x8 = 8'b00000000;
718     endcase
719 end
720
721     default: font8x8 = 8'b00000000;
722 endcase
723 end
724 endfunction
725
726 endmodule
727

```

Kode untuk menampilkan data hasil vga ke monitor